

TRABAJO PRÁCTICO PROFESIONAL INGENIERÍA ELECTRÓNICA

Autores:

Larraya, Juan Cruz - 104162 - jlarraya@fi.uba.ar

Stirparo, Nicolás - 105885 - nstirparo@fi.uba.a

Paulino Alaya, Diego-104311 - dpaulino@fi.uba.ar

Director:

Enrique Jorge castelli

Codirectores:

Fernando Filippetti

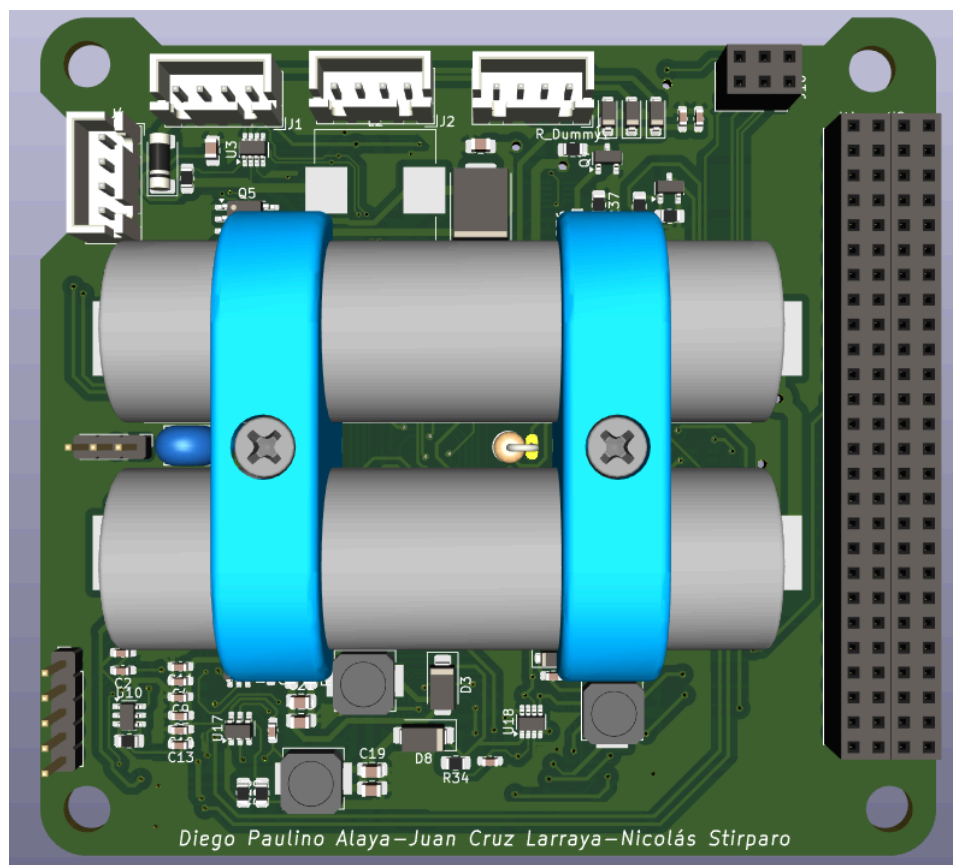
Monica Viviana Nitoli

Jurados:

Jorge Nesto Zanabria

Fernando Oscar Bonelli

Jonas Damian Pfefferman



Sistema de Gestión de Energía para CubeSat

Versión	Autor	Descripción del cambio	Estado	Fecha
1.0	DP	Documentación inicial	Finalizado	15-10-2024
1.1	DP	Informe final	Finalizado	23-09-2025

Contenido

Introducción.....	4
Motivación.....	4
Objetivo.....	4
Alcance del proyecto.....	4
1. Análisis de factibilidad.....	5
1.1. Descripción de la necesidad detectada.....	5
1.2. Descripción de la solución propuesta.....	7
1.3. Soluciones existentes.....	9
1.4. FODA.....	10
1.5. Factibilidad tecnológica.....	11
1.6. Descripción detallada del producto a desarrollar.....	13
1.6.1. Entrada de Energía:.....	13
1.6.2. Almacenamiento de Energía:.....	14
1.6.3. Unidad de Gestión de Energía:.....	15
1.6.4. Monitorización y Control:.....	18
1.6.5. Protección y Seguridad:.....	18
1.6.6. Software de Gestión:.....	18
1.7. Consideraciones Importantes:.....	19
1.8. Análisis de normas vigentes.....	19
1.9. Cronograma de desarrollo.....	22
2. Consideraciones técnicas.....	23
2.1. Protocolo I ² C en el Sistema de Gestión de Energía.....	23
3. Desarrollo de prototipo.....	25
3.1. BMS.....	26
Primer Diseño del BMS.....	26
3.1.1. Segundo Diseño del BMS.....	26
3.1.2. Diseño final del BMS.....	27
4. Primer Diseño del Circuito Completo.....	29
4.1. Rediseño de módulo PPC.....	29
Primer Diseño del PPC.....	31
Diseño final del PPC.....	32
4.2. Primer Rediseño de módulo BMS.....	32
Análisis Detallado de la Evolución del BMS.....	32
Transición 1: De Módulo Autónomo (bq76905) a Supervisado (bq29330).....	32
Mejoras Buscadas (Del Diseño 1 al 2).....	32

4.3. Segundo Rediseño de módulo BMS.....	34
Limitaciones Críticas del Diseño 2.....	34
Mejoras Obtenidas con el Diseño 3 (A Medida).....	34
4.4. Rediseño de módulo PDU.....	35
Optimización de la Unidad de Distribución de Potencia (PDU).....	35
Diseño Inicial: Salidas Independientes.....	35
Diseño Final: Canales con Redundancia 	36
5. Versión Final.....	37
6. Controladores.....	39
Arquitectura de Control y Microcontroladores Seleccionados.....	39
ATmega328P (Controlador Lógico Principal).....	39
STM32F103C8T6 (Co-Procesador de Hardware).....	40
7. Descripción Funcional.....	41
7.1. Características Generales.....	41
7.2. Diagrama en Bloques.....	41
7.3. Microcontrolador.....	42
7.4. Paneles solares.....	42
8. Especificaciones.....	42
8.1. PPC.....	42
8.2. BMS.....	43
8.3. Reguladores con PDU.....	43
8.4. Paneles – P110 GOMspace.....	44
8.5. Baterías - Battery 18650 Samsung.....	44
9. Conexiones.....	45
10. Desempeño.....	45
10.1. Medición de Señales con ADC:.....	45
10.2. Medición de Temperatura.....	46
10.3. Carga Completa de las Baterías.....	48
10.4. Descarga Completa de las Baterías.....	49
10.5. Balanceo de Baterías:.....	52
11. Interfaz de control y envío de datos.....	54
11.1. PPC.....	55
11.2. BMS.....	56
11.3. PDU.....	58
11.4. Aclaraciones:.....	62
12. Dimensiones Físicas.....	62
13. Apéndice.....	64
Apéndice 1.....	64
Apéndice 2.....	65
Apéndice 4.....	66
Apéndice 5.....	67
14. Fallos Encontrados y Soluciones Implementadas:.....	72

16. Resultados del Desarrollo a Nivel técnico:	73
17. Conclusión:	73

Introducción

El EPS es uno de los subsistemas más críticos de un satélite, ya que de su correcto funcionamiento depende la operación continua de los demás módulos, tales como el sistema de comunicaciones, el control de actitud y el payload. Su rol principal es gestionar la energía captada por los paneles solares, regular y distribuir las tensiones necesarias, cargar las baterías y garantizar tanto la protección de las mismas como la seguridad de los circuitos asociados. Además, debe ser capaz de sensor parámetros eléctricos y térmicos para reportarlos al computador a bordo y así optimizar la eficiencia energética durante la misión.

En este trabajo práctico profesional, se desarrolla un sistema de gestión de energía (EPS) para un CubeSat, diseñado para cumplir con las especificaciones requeridas en tensiones de salida, tasas de datos para comunicación con periféricos y vida útil extendida.

Motivación

Este proyecto surge de la escasa oferta de módulos EPS en Sudamérica y de los elevados costos de los productos disponibles en el mercado internacional, que en muchos casos carecen de documentación abierta para su integración. Ante esta situación, se plantea la oportunidad de desarrollar un sistema propio que no solo satisfaga las necesidades técnicas del CubeSat propuesto, sino que también contribuya al crecimiento de la industria espacial argentina mediante la generación de conocimiento y tecnología local.

Objetivo

Como resultado final, se entregará un prototipo operativo junto con la documentación técnica y las mediciones necesarias para su validación además de un circuito esquemático para realizar copias del circuito industrialmente a escala en un PCB bajo la norma PC104.

Alcance del proyecto

El alcance del proyecto no contempla el desarrollo de baterías, paneles solares ni la estructura mecánica del satélite, enfocándose exclusivamente en el sistema electrónico de gestión de energía preparado para operar en condiciones hostiles como las del espacio.

Debido a limitaciones presupuestarias, no se llevó a cabo el proceso de **certificación espacial** del módulo. Sin embargo, el diseño se realizó siguiendo criterios compatibles con condiciones extremas de temperatura, radiación y vibraciones propias del espacio, adoptando lineamientos de robustez y redundancia en los subsistemas electrónicos.

En las secciones posteriores se detallan las normas internacionales aplicables al diseño, ensamblaje y verificación de hardware aeroespacial, que constituyen la referencia metodológica sobre la cual se basó el desarrollo, aunque sin completar la validación experimental exigida por los estándares de certificación

1. Análisis de factibilidad

1.1. Descripción de la necesidad detectada

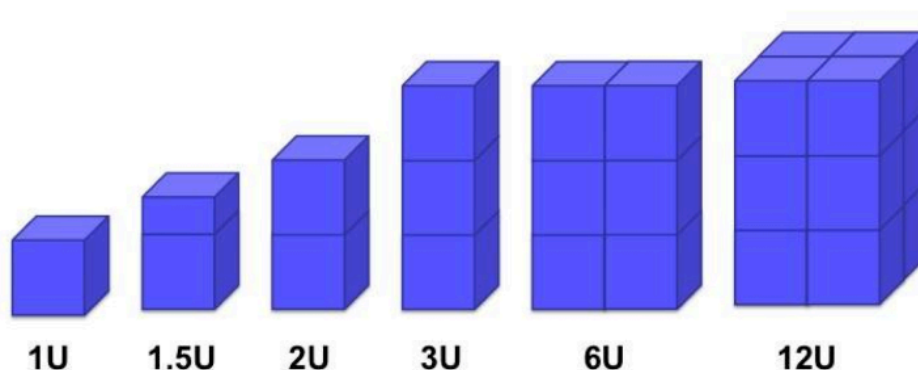
A través de lo desarrollado en la materia industrias y productos de ingeniería electrónica (86.45) se realizó la investigación de los CubeSat, estos son una categoría de nanosatélites que adoptan una forma y un tamaño estandarizados, así mismo, estos se encuentran en la órbita LEO gracias a que brinda mayor disponibilidad, menor coste y la radiación a la que se someten es significativamente menor frente a las demás órbitas.

El tamaño estándar de CubeSat se conoce como “una unidad” o “1U” que mide 10 x 10 x 10 cm y es extensible a tamaños más grandes; 1.5, 2, 3, 6 e incluso 12U.

Desarrollados originalmente en 1999, los CubeSats se crearon con la finalidad de ofrecer una plataforma accesible tanto para la educación como para la exploración espacial. El desarrollo de estos nanosatélites ha avanzado hasta constituir una industria independiente; donde gobiernos, empresas y entidades académicas colaboran para impulsar capacidades cada vez más avanzadas. Hoy en día se han convertido en una solución rentable para llevar a cabo investigaciones científicas, demostrar nuevas tecnologías y explorar conceptos de misión

avanzados como constelaciones de satélites en red (varias unidades cubesat desagregadas) con un mismo propósito.

Para nuestro desarrollo buscamos hacer un sistema de gestión de energía que pueda cumplir con las especificaciones propuestas, ya sea, en las distintas tensiones a suministrar, la tasa de información para la comunicación con los demás periféricos o en cuanto a la vida útil del sistema. Para este proyecto, por cuestiones económicas se está evaluando la certificación o garantía del funcionamiento del módulo, pero al margen de esto su diseño estará preparado para un ambiente hostil como lo es el espacio. Más adelante en el informe se especifican las normas a seguir para el diseño, ensamblaje y verificación del módulo.



El CubeSat se compone de varios subsistemas interconectados, cada uno desempeñando un papel fundamental en su funcionamiento. Se destacan los siguientes:

- Sistema de Guía y Control
- Sistema de Comunicaciones
- Sistema de Suministro de Energía
- Sistema de Misión (Payload)

El sistema de Guía y Control está compuesto por dos subsistemas: el ADCS y el OBC.

El ADCS (Attitude Determination and Control System) se encarga de la determinación y control de la orientación del CubeSat en el espacio, utilizando una combinación de sensores y actuadores para mantener

el satélite en la orientación deseada. En general, el control de actitud de un satélite es necesario no solo para lograr los objetivos de la misión, sino también para sobrevivir en el entorno espacial, principalmente en términos de generación de energía y control térmico. Las operaciones satelitales son autónomas y se basan fundamentalmente en la interacción de subsistemas electrónicos interconectados que intercambian datos según los requisitos y objetivos de la misión.

El OBC (On Board Computer) está desarrollado alrededor de un microcontrolador, realiza todas las tareas de computación y organiza la recopilación de datos.

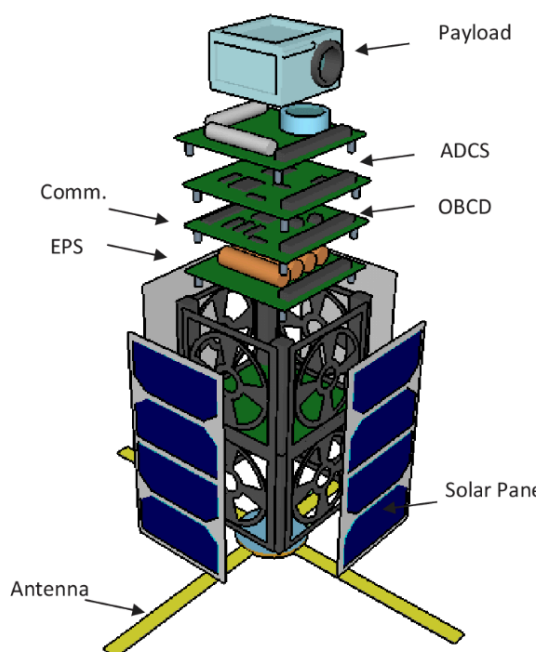
Para nuestro desarrollo buscamos hacer un sistema de gestión de energía que pueda cumplir con las especificaciones propuestas, ya sea, en las distintas tensiones a suministrar, la tasa de información para la comunicación con los demás periféricos o en cuanto a la vida útil del sistema.

Dado el estudio de los subsistemas del CubeSat observamos que uno de los componentes más críticos y fundamentales para el funcionamiento del satélite es el sistema de gestión de energía. Al analizar a la competencia descubrimos que había una escasa oferta de piezas en sudamérica, que la mayoría de los proveedores estaban ubicados en Europa del este y unos pocos en Estados Unidos.

En general estos sistemas no brindan abiertamente una gran documentación de su producto dificultando la comparativa. De esta manera visto la poca oferta y lo elevado muchas veces de los valores de esos módulos decidimos llevar a cabo el propio siendo esta una gran oportunidad para investigar y desarrollar tecnología en la Argentina.

1.2. Descripción de la solución propuesta

Para nuestro desarrollo buscamos hacer un sistema de gestión de energía que pueda cumplir con las especificaciones propuestas, ya sea, en las distintas tensiones a suministrar, la tasa de información para la comunicación con los demás periféricos o en cuanto a la vida útil del sistema.



El sistema consiste en tomar la energía que generan los paneles solares, regular la tensión, cargar las baterías y alimentar a los distintos subsistemas del satélite como pueden ser el sistema de control y el de comunicación. Además de esto el módulo debe contar con protecciones para las baterías y sus circuitos internos, así como también realizar mediciones de parámetros como la temperatura, tensiones y corrientes para comunicarlas al resto de los subsistemas.

En si el sistema está compuesto por.

- **Protector de baterías (BMS):** El BMS por sus siglas en inglés battery management system es el encargado de monitorear, comunicar y asegurar el correcto funcionamiento de cada una de las celdas que componen a la batería.
- **Cargador de baterías:** Es el circuito encargado de suministrar una corriente eléctrica a las baterías. Para ello controla la tensión de entrada (dada por los paneles) para cargar la batería en etapas (corriente cte, tensión cte y batería cargada).

Además, el sistema de carga permite extraer la máxima potencia posible de los paneles solares através de:

- **Método de carga MPPT (Maximum Power Point Tracking):** Es un sistema que permite extraer la máxima potencia posible de los paneles solares.

Comparado con otros métodos de carga, como el PWM (Pulse Width Modulation), el MPPT ofrece una mayor eficiencia y rendimiento, ya que optimiza constantemente la operación del panel solar para adaptarse a las condiciones de iluminación cambiantes.

- **Reguladores de tensión del tipo DC-DC:** Estos son necesarios para obtener distintos valores de tensión constante y suministrar a los distintos dispositivos con ella, tanto a los internos del módulo como al resto del satélite.
- **Circuito de gestión de energía:** Este permite administrar la alimentación a los distintos módulos del dispositivo permitiendo un manejo de la energía más eficiente en función de las necesidades.
- **Medición y comunicación:** Se sensan parámetros como las tensiones de las baterías, las corrientes a las salidas, las temperaturas de ambiente y las baterías para comunicarlás a los distintos módulos.

El entregable final del proyecto será un PCB formato cubesat operativo, la documentación técnica necesaria para quien lo deba usar y las mediciones realizadas incluyendo normas y protocolos. No son parte del desarrollo, las baterías, los paneles ni la estructura mecánica del satélite.

1.3. Soluciones existentes

A la fecha, se muestran algunos productos que se encuentran disponibles en el mercado:

	Endurosat EPS I	Kongsberg nanoavionics EPS 1.0	Kongsberg nanoavionics EPS 2.0	ISISPACE Compact EPS (tipo C)	Gomspace NanoPower P31u
Tensión entrada (panel solar) [V]	Más de 5.5	2.6-15	5-42	3.5 - 13	0 - 8.5
Tensión salida [V]	3.3 - 5	3.3-5-configurable entre 3 y 12	3.3 - 5 - 12 - 28	3.3 - 5 - Vbat - 28.2	3.3 - 5 - Vbat
Entradas de paneles	6	8	12	6 a 10	6
Potencia salida [W]	10.2 o 20.4	20(3.3-5V), 25 sal sin reg y 175 sal sin reg con pack de bat extra	33(3.3V)-40(5V)-60(12V)-75(28V)-600 sal sin reg con pack de bat extras	28.8 o 57.6	36
Número de salidas/convertidores	52 salidas	18/4	20/4 o 80	9/3 a 17/6	6/2
Comunicación	-	soporta CAN con protocolo CSP	soporta CAN con protocolo CSP	I2C	I2C
Peso [gr]	230	-	-	365	-
Precio	\$4,500	-	-	€4,400	

1.4. FODA

1.4.1. Fortalezas:

- Lugares de mucha trayectoria en el país donde podemos realizar test y simulaciones de nuestro proyecto.
- Experiencia previa en diseño y ensamblaje de fuentes DC-DC buck.
- Gente involucrada (tutores) con conocimiento y experiencia trabajando con paneles y baterías.
- Existencia de software de simulación para cubesat con el que modelar los entornos espaciales y de lanzamiento.

1.4.2. Oportunidades:

- Incentivo por parte de la FIUBA formar estudiantes en tecnología satelital, generar proyectos de investigación y desarrollo
- Industria espacial argentina creciente.
- Importaciones más flexibles a futuro, además de otras políticas favorables de comercio exterior.
- Industria espacial en auge a lo largo del mundo gracias al desarrollo de la internet satelital (starlink) entre otros proyectos con nuevos paradigmas.

1.4.3. Debilidades:

- Falta de experiencia trabajando con sistemas embebidos y circuitos en el ámbito espacial.
- Se debe negociar el testeo con empresas/entidades que dispongan de los aparatos necesarios para realizar las mediciones en condiciones espaciales.

1.4.4. Amenazas:

- Dificultad para conseguir fondos para un proyecto estudiantil en el ámbito espacial.

1.5. Factibilidad tecnológica

a) La mejor solución de compromiso entre la tecnología existente y la posible de implementar exitosamente en el país.

Usar integrados de primeras marcas con la documentación adecuada de su funcionamiento y con componentes comerciales. Delegar la fabricación de la placa en una empresa especializada para un acabado más profesional. Realizar testeos en tierra del correcto funcionamiento del circuito a distintas temperaturas además de simulaciones computarizadas de la placa en entorno espacial.

Así mismo cabe la posibilidad de testear el módulo en un ambiente controlado con las características lo más similares al espacio posible (CONAE, CNA, CONEA, etc) o en un ambiente espacial por medio de un socio que quiera lanzar un satélite y usar nuestro módulo a modo de prueba.

b) Analizar los riesgos de seleccionar u optar por una determinada tecnología, averiguar los antecedentes de aplicación, el ciclo de vida de los componentes a utilizar y las tecnologías emergentes que puedan superar por costo o prestación a las actuales.

- MPPT vs PWM para carga de batería.
- Tecnología MOS vs TBJ para distribución de energía en PDU.
- Protocolos de comunicación para telemetría de estado de vida (Fuel Gauge).
- Relación Li-on vs Lipo.
- Cantidad y tensiones de entrada/ salida.
- Microcontrolador vs integrados dedicados para batería.
- Cantidad de reguladores de entrada/salida.
- Cuántas baterías en serie y paralelo puede manejar el BMS.

c) Analizar el estado del arte y las nuevas tecnologías que se vienen desarrollando en los últimos años.

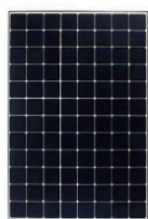
Actualmente, debido a los costos y a los avances tecnológicos en cuanto al tamaño de los módulos necesarios para un satélite, se empezaron a desarrollar los PocketQube, los cuales son de menor tamaño a los CubeSats, 5x5x5 cm es su tamaño normalizado, Sin embargo, en este proyecto se desarrollará un cubesat debido a que la posibilidad de éxito es mayor frente a la de un Pocket Cube.

1.6. Descripción detallada del producto a desarrollar

Un módulo de gestión de energía para un CubeSat es crucial para garantizar un suministro eléctrico confiable y eficiente en el espacio.

1.6.1. Entrada de Energía:

- Paneles Solares: Instalados en el CubeSat para capturar la energía solar mientras el satélite orbita. Principalmente son de arseniuro de galio y entregan tensiones de 3V, 5V y 8V. Además de que pueden colocarse en serie o paralelo para obtener distintas entradas al EPS.



Silicio monocristalino: poseen una estructura cristalina uniforme, donde todos los átomos están perfectamente alineados, formando un solo cristal.



Silicio policristalino: presentan una estructura cristalina ordenada por regiones, en la que sus átomos presentan distintas direcciones.



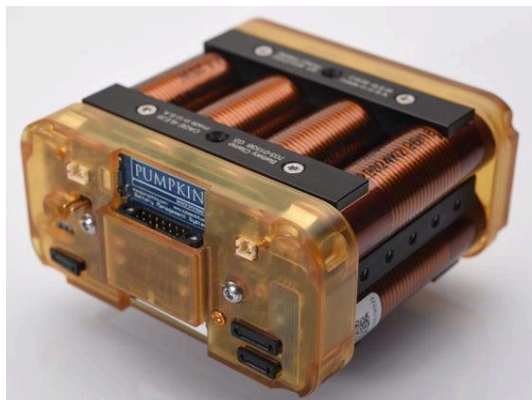
Silicio amorfo: el material semiconductor se deposita como película fina en distintos soportes, lo que permite producir módulos rígidos o flexibles.

1.6.2. Almacenamiento de Energía:

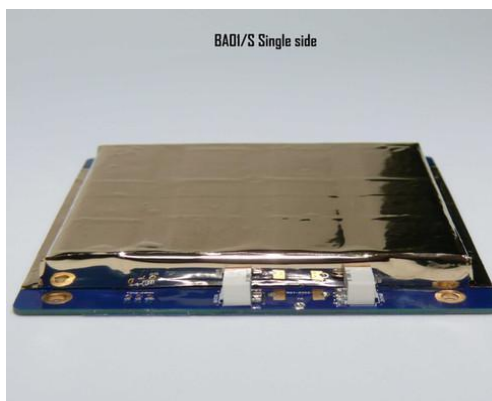
- **Baterías:** Almacenan la energía generada por los paneles solares para su uso durante eclipses solares o cuando el satélite esté en la sombra de la Tierra. Deben seguir funcionando correctamente en el entorno

espacial, para eso son testeadas de antemano. Estas son de litio ya sea polímero o ion de litio dependiendo el fabricante.

Observación: Tanto las baterías como los paneles solares son componentes que se relacionan directamente con el sistema de gestión de energía pero no están incluidos y no son parte del proyecto.



Batería de Ión litio

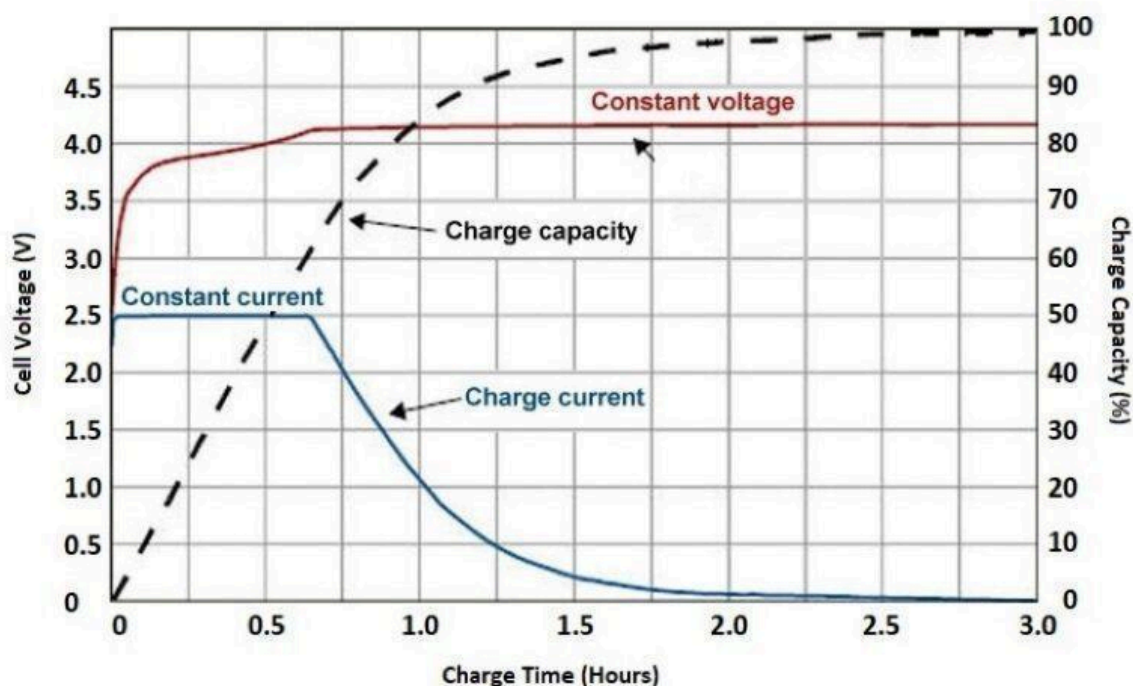


Batería de polímero de litio

1.6.3. Unidad de Gestión de Energía

- Controlador de Carga: Regula y controla cómo se cargan las baterías desde los paneles solares para evitar sobrecargas o descargas excesivas. Este puede desarrollarse con un microcontrolador o un integrado dedicado. Los métodos más típicos son MTTP o PWM. El proceso de carga de una batería consta de:

Batería de Ión litio Batería de polímero de litio Para nuestro desarrollo buscamos hacer un sistema de gestión de energía que pueda cumplir con las especificaciones propuestas, ya sea, en las distintas tensiones a suministrar, la tasa de información para la comunicación con los demás periféricos o en cuanto a la vida útil del sistema.

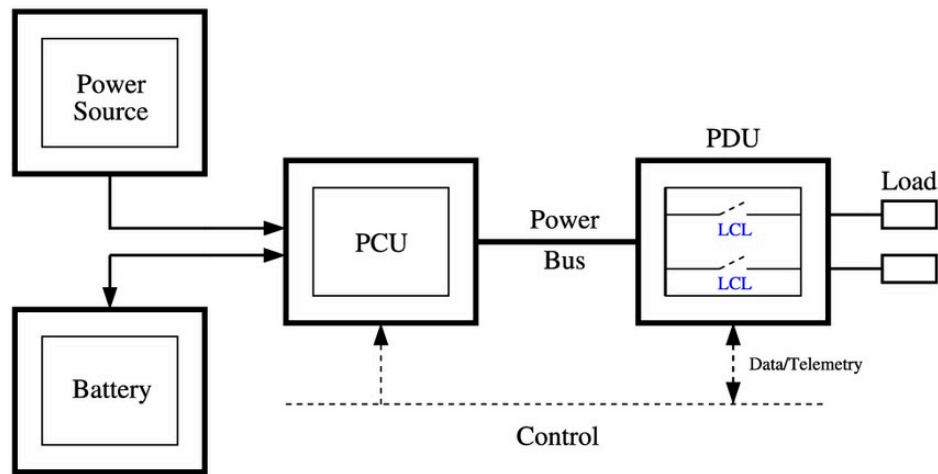
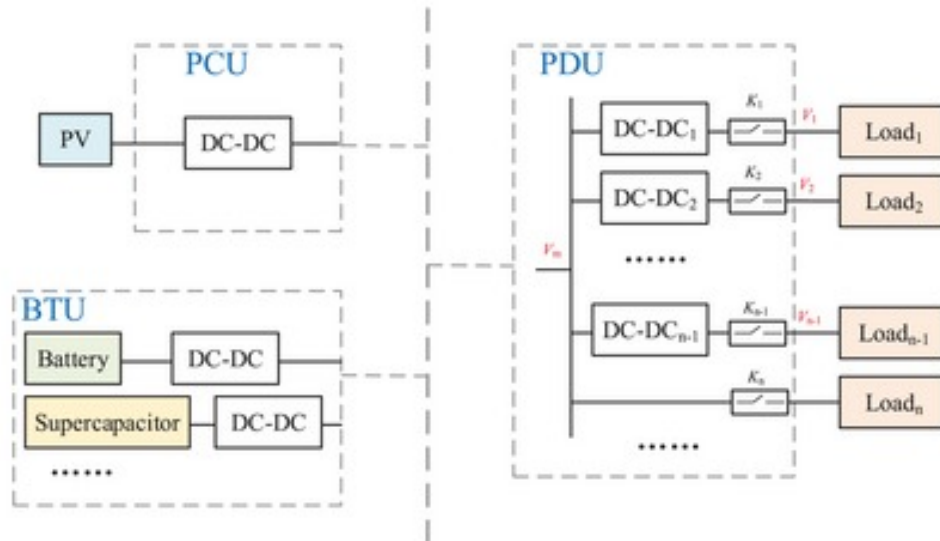


Para el cargador de baterías de litio se utiliza un cargador de 3 etapas. La primera a Corriente Constante (CC) y cuando la batería esté casi cargada, comienza la segunda etapa a tensión constante (CV) suministrando a la batería una corriente muy baja hasta que corta y pasa al 3 estado de carga completa.

- Conversores DC-DC: Transforman y regulan la tensión para adaptarla a las necesidades de los distintos subsistemas del satélite. Típicamente conocidas como fuentes switching compuestas por un transistor que actúa como llave, en función del tiempo que esté abierto y cerrado este se genera un pwm que posteriormente es filtrado por un filtro de segundo orden tipo LC. Son de alta eficiencia y brindan la posibilidad de aislar eléctricamente el circuito de entrada del de salida por medio de un transformador (en reemplazo del inductor).

Para suministrar una tensión más alta que la de entrada se carga el inductor y por medio de la energía almacenada variando el pwm pueden obtenerse tensiones mayores, esta configuración es conocida como step-up a diferencia de la reductora conocida como step-down o BUCK.

- **Sistema de Distribución de Energía (PDU):** Canaliza la energía a los diferentes sistemas y subsistemas del CubeSat, asegurando un suministro estable y protegiendo contra cortocircuitos u otros fallos.



1.6.4. Monitorización y Control:

- **Sensores:** Para medir la corriente, voltaje y temperatura de los paneles solares, baterías y otros componentes.
- **Microcontrolador/Microprocesador:** Controla el flujo de energía, supervisa los sensores y toma decisiones para optimizar la gestión de energía.

1.6.5. Protección y Seguridad:

- **Circuitos de Protección:** Evitan sobrecargas, cortocircuitos y condiciones peligrosas que podrían dañar los componentes.
- **Sistemas de Respuesta ante Fallos:** Protocolos para responder a situaciones de emergencia, como la desconexión de sistemas no esenciales en caso de escasez de energía.

1.6.6. Software de Gestión:

- **Algoritmos de Gestión de Energía:** Programas informáticos que controlan el flujo de energía basados en la información recopilada por los sensores y las condiciones operativas.

1.7. Consideraciones Importantes:

- **Eficiencia Energética:** La gestión de energía debe optimizar la eficiencia para maximizar la vida útil de las baterías y el rendimiento general del CubeSat.
- **Resistencia al Entorno Espacial:** Los componentes deben ser capaces de resistir radiación, cambios de temperatura y otras condiciones extremas del espacio.
- **Limitaciones de Espacio y Peso:** Los CubeSats tienen restricciones severas de tamaño y peso, lo que hace que la selección y diseño de componentes sea crítica.
- **Fiabilidad y Redundancia:** Se deben considerar sistemas de respaldo y redundancia para asegurar la operatividad del satélite en caso de fallos.

1.8. Análisis de normas vigentes

- **Standard PC/104:** Establece un conjunto de especificaciones para la fabricación de hardware informático en forma de placas apilables.
 - Define un formato estándar para las placas (9.144 x 9.652 pulgadas) y conectores para permitir la interconexión entre diferentes placas.
 - Facilita el diseño modular y la interoperabilidad entre componentes electrónicos en sistemas embebidos, incluidos los CubeSats.
- **IPC-A-610 Class 3 Assembly:** Especifica los requisitos para la inspección de componentes electrónicos durante el proceso de ensamblaje.

○ Clasifica los estándares de calidad en tres clases, siendo la Clase 3 la más exigente en términos de calidad y confiabilidad.

○ Define criterios detallados para la aceptación o rechazo de soldaduras, conexiones y otros aspectos de ensamblaje electrónico.

● **ECSS-E-ST-10-03C:** Es utilizado por la Agencia Espacial Europea (ESA) y sus contratistas para establecer los requisitos técnicos para el diseño, desarrollo y operación de satélites y módulos espaciales. Cubre aspectos como la gestión de riesgos, el diseño de hardware y software, los sistemas de control de calidad, la verificación y validación, entre otros.

● **GEVS: GSFC-STD-7000A (NASA Goddard Space Flight Center General Environmental**

Verification Standard): establece los requisitos de verificación ambiental para componentes, subsistemas y sistemas espaciales.

○ Define los procedimientos y criterios para la verificación de equipos espaciales en cuanto a su resistencia y capacidad de operación en diferentes condiciones ambientales, como la vibración, la temperatura, la radiación, entre otros.

Este estándar examina:

● **Peso y tamaño (volumen):** En función del tipo de satélite se establecen límites de peso y tamaño, esto es vital para el dispenser (encargado de transportar y poner en órbita el satélite una vez que el cohete llegó a la altura adecuada). Para nuestro desarrollo buscamos hacer un sistema de gestión de energía que pueda cumplir con las especificaciones propuestas, ya sea, en las distintas tensiones a suministrar la tasa de información para la comunicación con los demás periféricos o en cuanto a la vida útil del sistema. Para este proyecto, por cuestiones económicas se está evaluando la certificación o garantía del funcionamiento del módulo, pero al margen de esto su diseño estará preparado para un ambiente hostil como lo es el espacio. Más adelante en el informe se especifican las normas a seguir para el diseño, ensamblaje y verificación del módulo.

● **Aceleración, vibración y choque:** Por la alta aceleración y vibraciones que se producen en el lanzamiento, los conectores con clasificación de espacio suelen especificar tornillos de fijación o diseños de enclavamiento siempre que sea posible para garantizar una conexión segura.

● **Desgasificación:** El calor y las condiciones de vacío del espacio aumentan la tasa de desgasificación de los conectores. Materiales como los elastómeros y los plásticos pueden liberar lentamente compuestos

orgánicos volátiles (COV) que fueron disueltos, atrapados, congelados o absorbidos en el material como gas o vapor. Incluso el epoxi y otros adhesivos habituales pueden liberar estos COV, lo que obliga a utilizar adhesivos especiales. Los COV pueden dar lugar a una contaminación que puede afectar gravemente al rendimiento de los equipos de misión crítica al interferir con los instrumentos delicados y las superficies ópticas. En el caso de un conector con clasificación espacial, los COV se "expulsan" del material mediante la cocción de los conectores a una temperatura elevada en un horno sellado al vacío.

- **Magnetismo residual:** Puede interferir en el funcionamiento de los circuitos y subsistemas cercanos, provocando lecturas erróneas de los sensores de precisión. Para minimizarlo puede ser necesario utilizar materiales no magnéticos cuando sea posible, como las aleaciones de cobre.

- **Rango de temperaturas:** El rango ampliado para los conectores con clasificación espacial suele ser de -65°C a $+150^{\circ}\text{C}$. Sin embargo, los ciclos térmicos también son motivo de preocupación: las tensiones repetidas que resultan de dichos ciclos pueden inducir microfisuras y finalmente, fracturas por fatiga. Algunos satélites están diseñados para girar con el fin de igualar su temperatura media entre los lados orientados al sol y los que están a la sombra. Esta es una solución inadecuada para los satélites más grandes, ya que la superficie y la subsuperficie pueden seguir estando sometidas a ciclos térmicos importantes en comparación con sus interiores más profundos. En los satélites pequeños, como los CubeSats, casi todos los componentes están relativamente cerca de la superficie.

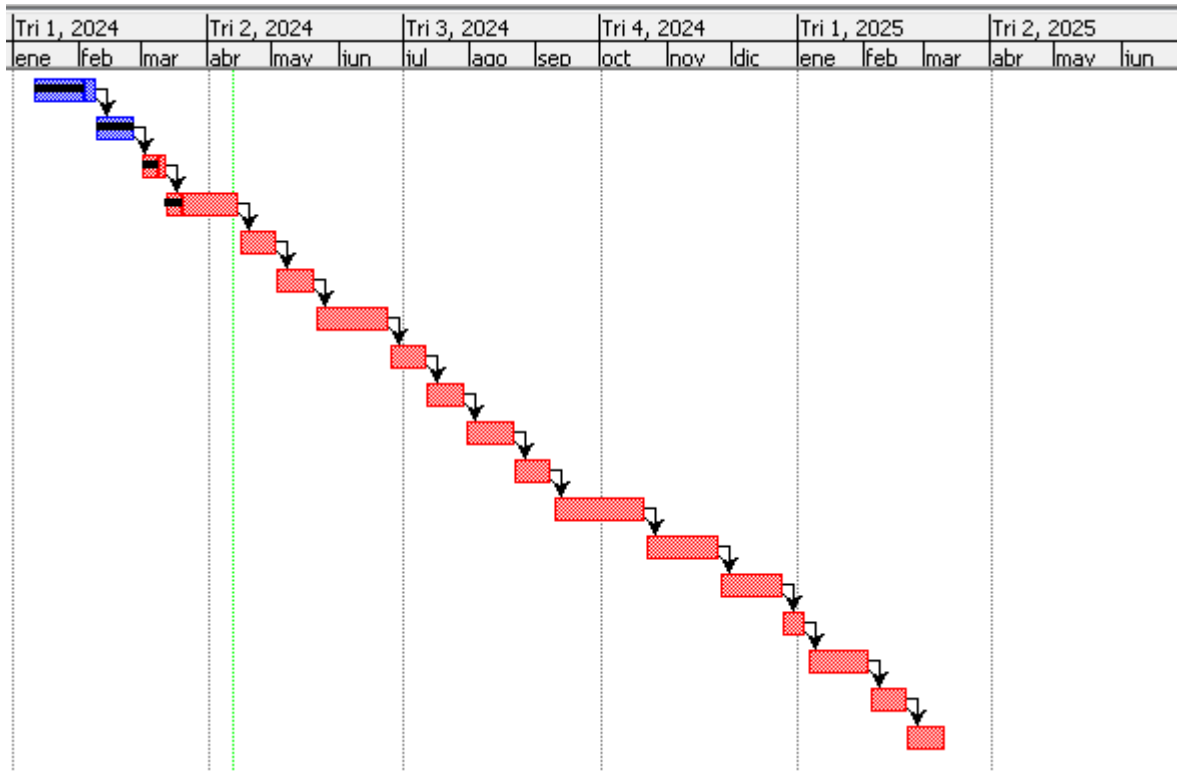
- **Radiación cósmica:** Aumenta a medida que aumenta la altitud de funcionamiento del satélite y se reduce la atmósfera protectora de la Tierra. Los efectos de esta radiación inevitable son similares en cierto modo a los efectos de las interferencias electromagnéticas (EMI). Aunque la carcasa metálica del satélite proporcionará cierto nivel de protección, puede ser necesario incluir un blindaje adicional en las placas de circuitos o en los cables que son susceptibles de recibir impactos de radiaciones.

- **Salto de corriente:** Es una descarga eléctrica continua de alta corriente desde un conductor hasta la superficie metálica más cercana. Las descargas eléctricas se producen a diferentes valores de tensión en función de la densidad de las moléculas de aire, siendo el vacío del espacio el caso extremo, por lo que los conectores deben tener una tensión nominal adecuada a la altitud.

- **Consideraciones físicas:** La orientación del conector y su cable es fundamental. Obviamente, los satélites están densamente empaquetados, y los populares pero diminutos CubeSats están llevando esta densidad a un nuevo nivel.

1.9. Cronograma de desarrollo

	Nombre	Duracion
1	Estudio del tema	25 days
2	Estudio de los modulos del sistema de gestion	15 days
3	Presentacion de informe a la comision curricular	10 days
4	Estudio de las características técnicas	30 days
5	Investigacion de disposicion de componenetes	15 days
6	Investigar donde se pueden hacer los ensayos	15 days
7	Definición de características tecnicas	30 days
8	Definición de componentes y tecnologías	15 days
9	Simulaciones en software	15 days
10	Desarrollo del plano electrico	20 days
11	Documentación preliminar a fabricación	15 days
12	Programacion y test de circuitos y micros	37 days
13	Pruebas del desempeño del circuitos	30 days
14	Correcciones en base a pruebas finales	25 days
15	Plano con dimensiones de la placa	10 days
16	Testeos	25 days
17	Generación con las especificaciones reales	15 days
18	Generación de Documentación (datasheet, manual de usuario, dimensiones, graficos)	15 days



2. Consideraciones técnicas

2.1. Protocolo I²C en el Sistema de Gestión de Energía

El **bus I²C (Inter-Integrated Circuit)** fue seleccionado como protocolo de comunicación principal en nuestro proyecto debido a su simplicidad y eficiencia para interconectar múltiples dispositivos dentro de un entorno embebido de recursos limitados, como lo es un CubeSat.

Características técnicas

- Utiliza **solo dos líneas físicas**:
 - **SDA (Serial Data Line)**: línea de datos.
 - **SCL (Serial Clock Line)**: línea de reloj.
- Permite conexión de múltiples dispositivos en un esquema **multi maestro/multi esclavo**.
- Soporta hasta **400 kHz** de frecuencia en modo Fast (compatible con nuestro diseño).
- Requiere **pocas resistencias pull-up**, lo que simplifica el cableado y reduce el consumo.

Ventajas para el CubeSat

- **Minimización de pines:** reduce la cantidad de líneas necesarias, liberando pines del microcontrolador para otras funciones.
- **Escalabilidad:** permite agregar sensores o periféricos sin modificar la topología principal.
- **Eficiencia en espacio y peso:** factores críticos en un satélite 1U, donde la densidad de componentes es elevada.

Aplicaciones en nuestro proyecto

El bus I²C se emplea en distintos bloques del sistema:

1. Gestión de energía (EPS)

- 1.1. Comunicación entre el **microcontrolador** y el **gestor de baterías** (p. ej. BQ76905 o BQ29330), para leer tensiones de celdas, corrientes y temperaturas, además de controlar el estado de carga/descarga.

2. Monitoreo de tensión y corriente con INA219

- 2.1. INA219: medición de tensión y corriente en buses de alimentación críticos, lo que permite:
 - 2.1.1. Supervisar el consumo en tiempo real de los subsistemas.
 - 2.1.2. Detectar sobrecorrientes o pérdidas de eficiencia.
 - 2.1.3. Implementar rutinas de protección y control inteligente de carga.

Este protocolo es fundamental para el desarrollo de nuestro proyecto ya que nos permitio dejar de depender el una arquitectura independiente de los divisores de tensión y amplificadores operacionales conectados directamente a las entradas ADC del microcontrolador e implementar una **arquitectura de monitoreo digital y distribuida basada en el bus I²C**.

2.2. MPPT

Algoritmo de Seguimiento del Punto de Máxima Potencia (MPPT)

El Módulo de Acondicionamiento de Potencia (PPC) implementa un algoritmo de **Seguimiento del Punto de Máxima Potencia**, conocido como **MPPT** por sus siglas en inglés (Maximum Power Point Tracking). Este no es un componente físico, sino una pieza de software (firmware) que controla de manera inteligente el convertidor boost para maximizar la energía cosechada de los paneles solares.

El Problema: La Eficiencia Variable del Panel Solar

Un panel solar no se comporta como una batería; no entrega una potencia constante. Su capacidad para generar energía depende drásticamente de las condiciones externas, como la **intensidad de la luz solar (irradiancia)** y la **temperatura**.

La relación entre la tensión y la corriente que un panel puede entregar se describe en su curva I-V. Si multiplicamos estos dos valores (Potencia = Tensión x Corriente), obtenemos la curva de potencia (P-V), que tiene un "pico" o "rodilla" muy definido. Este pico es el **Punto de Máxima Potencia (MPP)**: el punto de operación exacto donde el panel es más eficiente y entrega la máxima energía posible.

El problema es que este punto no es estático. Si la luz del sol disminuye o el panel se calienta, toda la curva cambia y el MPP se desplaza. Conectar un panel solar directamente a una batería es ineficiente, porque la tensión de la batería fuerza al panel a operar en un punto que casi nunca coincide con su MPP, desperdiciando una gran cantidad de energía.

¿Cómo funciona? El Algoritmo "Perturbar y Observar"

El MPPT soluciona este problema. El algoritmo que implementamos es el conocido como **"Perturbar y Observar" (P&O)**, que funciona de la siguiente manera:

1. **Medir (Observar):** El sistema mide la tensión y la corriente actuales que provienen del panel solar y calcula la potencia instantánea (P1).
2. **Ajustar (Perturbar):** El microcontrolador realiza un pequeño cambio en el **ciclo de trabajo (duty cycle)** del PWM que controla el convertidor boost. Por ejemplo, lo aumenta ligeramente.
3. **Volver a medir:** Inmediatamente después del ajuste, vuelve a medir la potencia (P2).
4. **Decidir:**
 - Si $P2 > P1$, significa que el cambio fue positivo y nos acercamos al pico de potencia. Por lo tanto, el algoritmo continuará haciendo cambios en la misma dirección (seguirá aumentando el duty cycle).
 - Si $P2 < P1$, significa que el cambio fue negativo y nos pasamos del pico. El algoritmo entonces invierte la dirección del cambio (comenzará a disminuir el duty cycle).

Este ciclo se repite cientos de veces por segundo, haciendo que el sistema "oscile" constantemente alrededor del punto exacto de máxima potencia, persiguiendo sin descanso.

Analogía simple: Es como sintonizar una radio antigua. Giras el dial lentamente en una dirección. Mientras el sonido se haga más claro, sigues girando en esa dirección. Si empieza a sonar peor, es que te pasaste, entonces giras el dial en la dirección contraria. El MPPT hace exactamente eso, pero buscando la "señal de potencia" más clara.

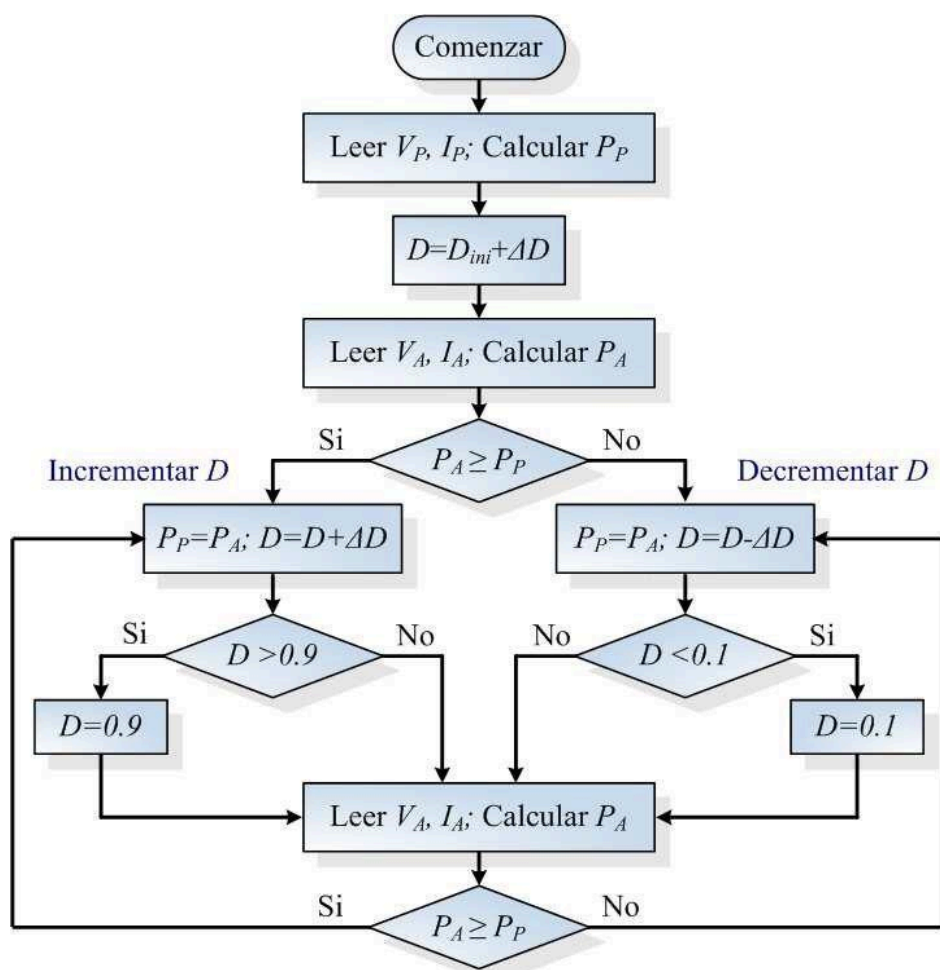
Beneficios Clave para un CubeSat en Órbita

Para CubeSat, el MPPT, es una **necesidad crítica** para un aprovechamiento pleno de la energía de los paneles.

Un CubeSat en órbita no mantiene una orientación fija respecto al sol; está constantemente **girando y rotando sobre sus ejes (tumbling)**. Esto provoca que:

- **La Iluminación es Dinámica:** El ángulo con el que la luz solar incide en cada panel cambia a cada segundo. Pasa de recibir luz solar directa y plena a recibirla de forma oblicua y débil en una fracción de minuto.
- **Las Condiciones Cambian Rápidamente:** Cada vez que la orientación cambia, la irradiancia y la temperatura del panel se alteran, lo que, como vimos, **desplaza constantemente el Punto de Máxima Potencia**.

El algoritmo MPPT es la única forma de adaptarse a esta caótica realidad. De manera autónoma y en tiempo real, el sistema se reajusta continuamente para exprimir cada miliwatt de energía disponible, sin importar la orientación del satélite. Esto asegura que las baterías se carguen de la forma más rápida y eficiente posible, maximizando la energía disponible para los experimentos científicos, las comunicaciones y el resto de los subsistemas.



3. Desarrollo de prototipo

El presente informe detalla el proceso de diseño, desarrollo y validación de un Sistema de Potencia Eléctrica (EPS) para un nanosatélite estándar CubeSat de una unidad (1U). Lejos de ser un proceso lineal, el proyecto se caracterizó por una marcada evolución, donde los desafíos técnicos y las restricciones prácticas se convirtieron en los principales catalizadores para la innovación y la optimización profunda del diseño.

El punto de inflexión que definió la trayectoria del proyecto fue el fallo funcional del primer prototipo. Este había sido concebido como una solución monolítica y final, diseñado rigurosamente bajo la normativa CubeSat y enviado a manufactura profesional. Sin embargo, su fracaso en las pruebas iniciales, sumado a las estrictas limitaciones presupuestarias de un proyecto académico que hacían inviable la fabricación de nuevas versiones completas, nos obligó a realizar un cambio estratégico fundamental.

Se abandonó el enfoque de "todo en uno" para adoptar una **metodología de desarrollo modular e incremental**. Esta decisión, nacida de la necesidad, resultó ser una de las más fructíferas. Trabajar con prototipos de bajo costo para cada subsistema no solo facilitó la depuración y viabilizar económicamente el proyecto, sino que también nos permitió analizar críticamente y rediseñar componentes clave con un nivel de profundidad que el enfoque inicial no permitía.

En las siguientes secciones, se explorará en detalle cómo esta filosofía de trabajo iterativo nos llevó a implementar optimizaciones sustanciales en los subsistemas más críticos del EPS:

- **La Optimización del Sistema de Telemetría:** Se describe la transición desde un esquema inicial que consumía una gran cantidad de recursos del microcontrolador mediante pines ADC, hacia una arquitectura de bus **PC** con sensores dedicados **INA219**. Esta mejora simplificó drásticamente el hardware y aumentó la precisión y escalabilidad del monitoreo.
- **El Desarrollo de un Sistema de Gestión de Batería (BMS) a Medida:** Se detallará la evolución desde el uso de circuitos integrados comerciales, primero autónomos y luego supervisados, hasta la creación de una solución completamente personalizada basada en un microcontrolador. Este desarrollo final nos otorgó un control sin precedentes, una mayor flexibilidad y la capacidad de implementar una interfaz de usuario amigable mediante comandos **UART**.

El prototipo puede dividirse en tres bloques principales que trabajando en conjunto cumplen los requerimientos del sistema:

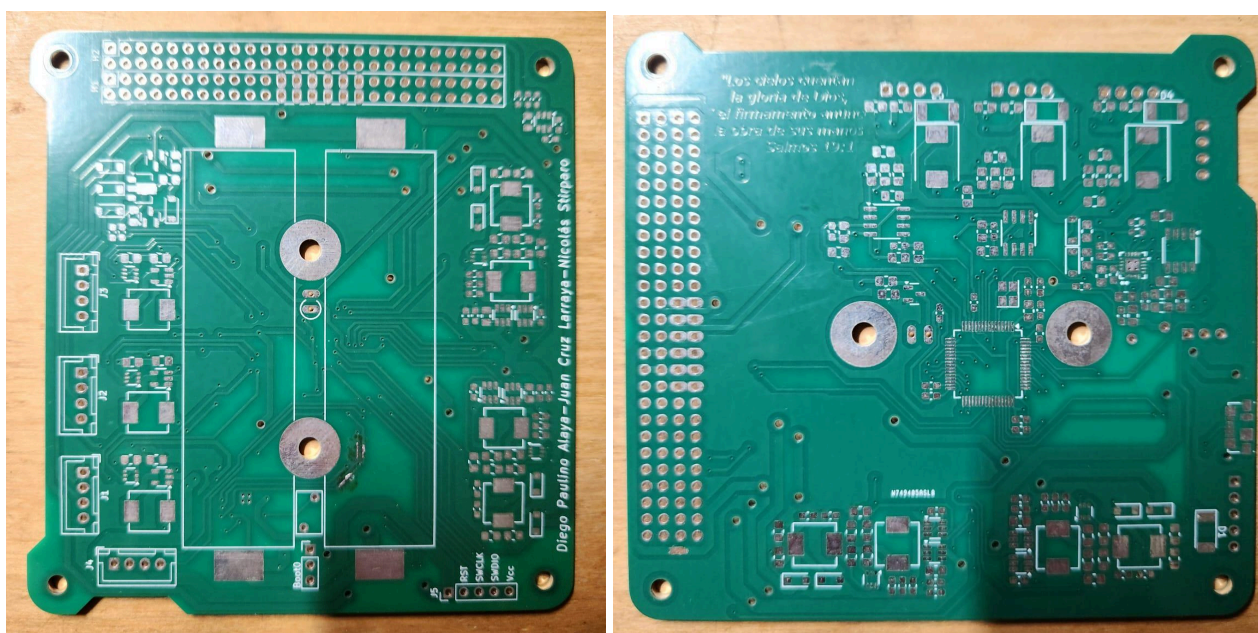
- **BMS:** El Battery Management System o BMS se encarga de proteger a las celdas de litio para garantizar que las mismas funcionen dentro de su área segura de operación. Manteniendo un control de corrientes de carga y descarga, tensiones de celdas máxima y mínima, temperatura y contador de carga.
- **PDU:** El Power Distribution Unit o PDU se encarga de proveer a los sistemas alimentados por la EPS las tensiones necesarias mientras se protege el suministro contra cortocircuitos e incompatibilidades electromagnéticas.
- **PPC:** El Photovoltaic Panel Control o PPC se encarga de, mediante el algoritmo de trackeo del punto de máxima potencia o MPPT, aprovechar al máximo la energía recolectada por los paneles solares para alimentar con la misma la carga de las baterías y las salidas siempre que se logre un superávit energético.

4. Primer Diseño del Circuito Completo

Restricciones y Reorientación Estratégica

La fabricación de un PCB de alta densidad y múltiples capas representó una inversión económica significativa. Dado el **contexto académico y estudiantil** del proyecto, el alto costo (aproximado de 500 USD por placa) hizo inviable la producción de nuevas iteraciones completas para cada ajuste de diseño.

Esta limitación nos obligó a reevaluar la metodología de desarrollo. Se tomó la decisión estratégica de abandonar el enfoque monolítico en las fases de prueba y adoptar un **enfoque de validación modular e incremental**.



4.1. Rediseño de módulo PPC

Dentro de la reorientación metodológica, uno de los subsistemas que requirió un rediseño completo fue el **Módulo de Acondicionamiento y Control de Potencia (PPC)**, cuya función principal es la de actuar como convertidor boost para maximizar la transferencia de energía desde los paneles solares.

Análisis del Fallo en el Diseño Inicial

El primer prototipo del PPC, aunque había sido validado exitosamente en pruebas preliminares utilizando módulos convertidores disponibles en el mercado local, falló al ser integrado en la placa final. El análisis posterior reveló varias debilidades fundamentales en su arquitectura:

- **Sensibilidad y Rango Operativo Limitado:** El diseño se basaba en un lazo de control analógico. El microcontrolador no controlaba directamente el switching del convertidor, sino que ajustaba una tensión en el pin de feedback del circuito integrado boost. Este método demostró ser extremadamente

sensible a la precisión de los sensores y solo funcionaba de manera estable dentro de un rango operativo muy estrecho.

- **Discrepancia de Componentes:** Se constató que las características del circuito integrado boost utilizado en la manufactura profesional difieren de las del módulo usado para la validación inicial. Esta discrepancia, combinada con la fragilidad del lazo de control, hizo que el sistema fuera incapaz de regular la potencia de manera fiable.
- **Control Indirecto:** La naturaleza indirecta del control hacía que el sistema fuera poco predecible y difícil de sintonizar. No se tenía una autoridad directa sobre el parámetro más importante del convertidor: el ciclo de trabajo (duty cycle).

La Nueva Arquitectura: Simplicidad y Control Digital Directo

Para solventar estas deficiencias, se optó por una arquitectura radicalmente más **simple, robusta y controlable**. El nuevo diseño elimina por completo el complejo lazo de realimentación analógico y adopta un enfoque de **control digital directo**.

La nueva topología se compone de:

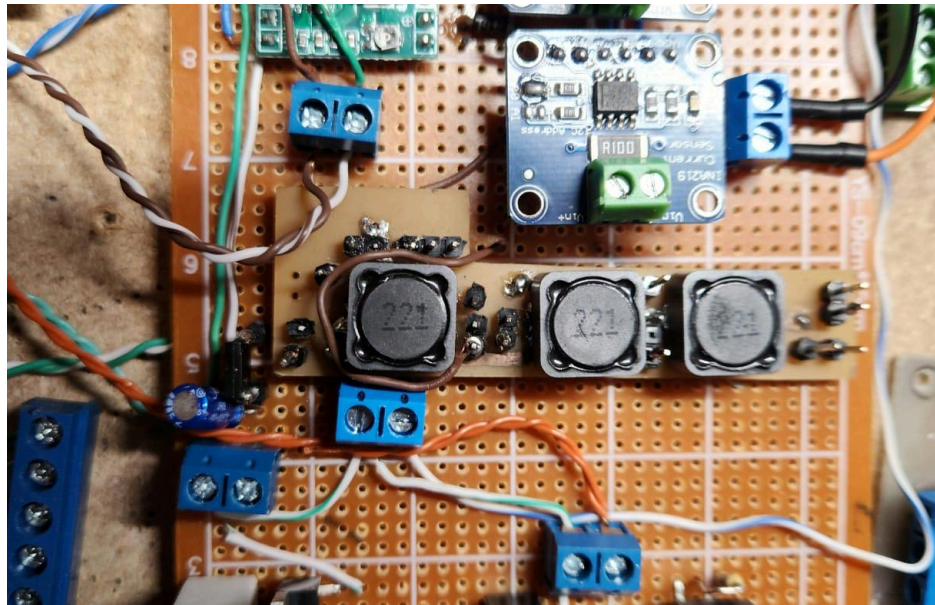
- Un **microcontrolador** que genera directamente las señales de Modulación por Ancho de Pulso (PWM).
- **Gate Drivers** dedicados que toman estas señales PWM de bajo poder y las amplifican para poder excitar de manera eficiente los MOSFETs de la etapa de potencia.

Este cambio representa una transición fundamental en la filosofía de control. Se pasó de un método indirecto a uno donde el firmware tiene **autoridad total y directa sobre el ciclo de trabajo (Duty Cycle)** del convertidor boost.

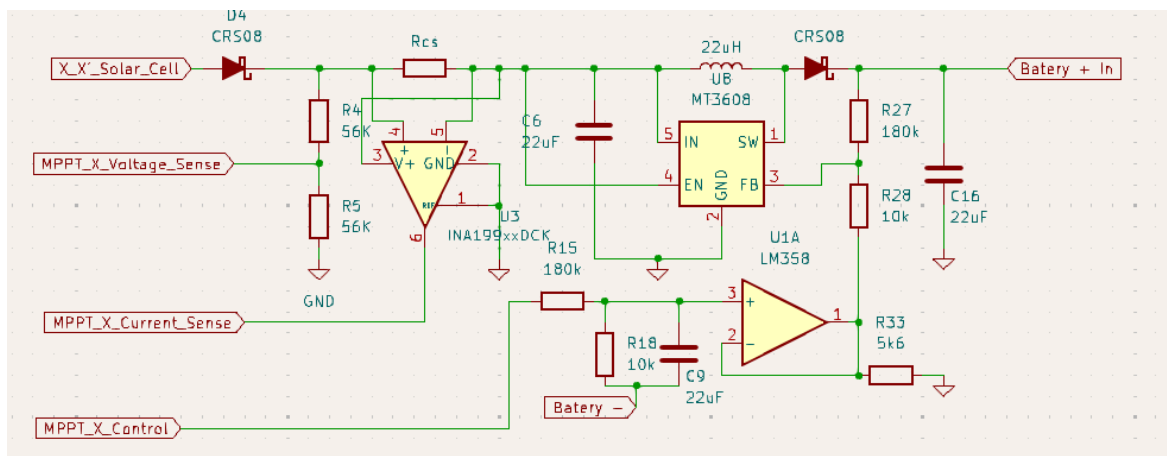
Ventajas del Nuevo Enfoque:

- **Robustez y Fiabilidad:** Al eliminar el sensible lazo analógico, el sistema es mucho menos vulnerable a variaciones de componentes y ruido de sensores.
- **Control Total y Predecible:** El comportamiento del convertidor está directamente dictado por el duty cycle que establece el software, lo que facilita enormemente la implementación y el ajuste de algoritmos de seguimiento del punto de máxima potencia (MPPT).
- **Simplicidad del Hardware:** El circuito se vuelve más simple, con menos componentes críticos, lo que reduce los posibles puntos de fallo.
- **Amplio Rango Operativo:** El control digital directo permite que el sistema opere de manera estable en un rango mucho más amplio de condiciones de entrada (tensión e irradiancia solar).

Este rediseño no fue una simple corrección, sino una evolución hacia una solución de ingeniería mucho más sólida y adecuada para una aplicación crítica como la de un satélite.



Primer Diseño del PPC



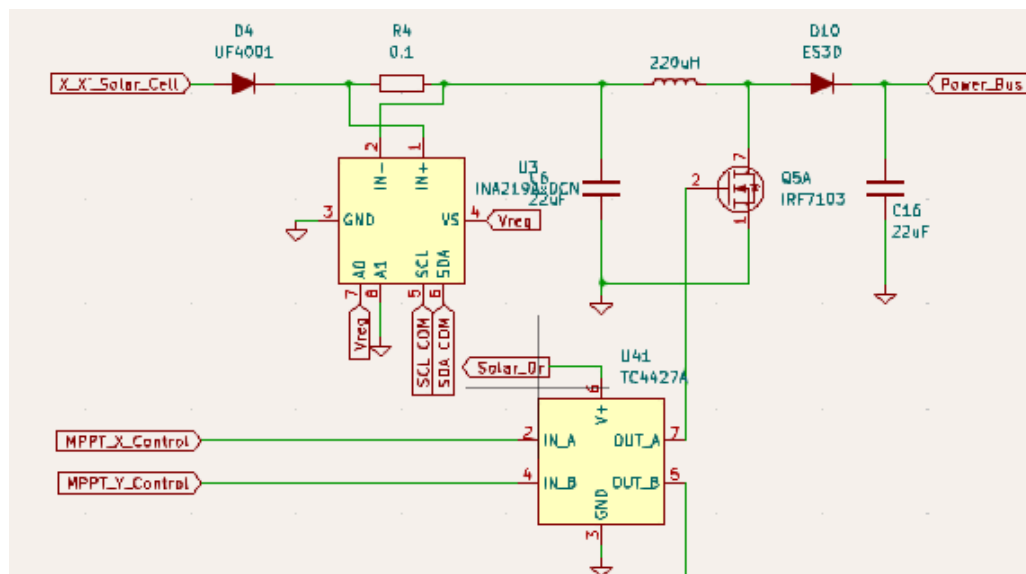
Monitoreo:

Al realizarse censados por ADC para tensión y corriente, arrastró las mismas susceptibilidades mencionadas anteriormente en el módulo PPC.

Conversión:

El objetivo de este diseño era permitir una recuperación en caso de una caída por parte del microcontrolador principal del sistema. Sin embargo el sistema se mostró muy susceptible a los controles de flujo por parte del sistema BMS además de un comportamiento poco fiable en su control desde la realimentación.

Diseño final del PPC



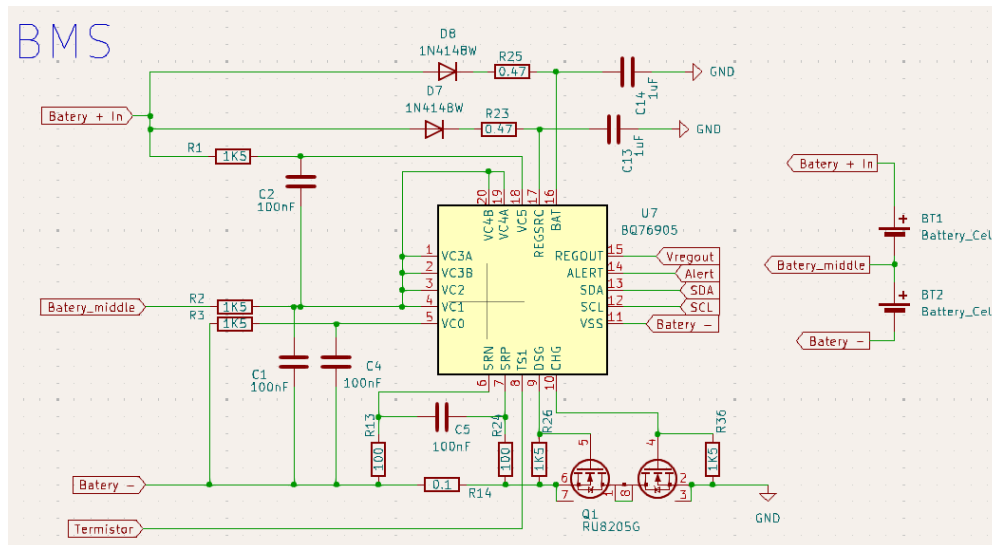
Monitoreo:

Al realizarse censados in situ mediante el INA219, se eliminaron susceptibilidades a ruido y se permitió agrupar el monitoreo de PPC en un solo bus I2C junto con las demás mediciones del sistema, volviéndolo así más escalable que el modelo anterior que se veía limitado por la cantidad de canales ADC del microcontrolador central y aumentaba el consumo del mismo.

Conversión:

Al constar de menos componentes y de un diseño más sencillo, el sistema de conversión se vuelve mucho más robusto. El mismo, al ser monitoreado desde su entrada y teniendo una topología boost, bombea corriente a las baterías buscando el punto de máxima potencia de los paneles solares.

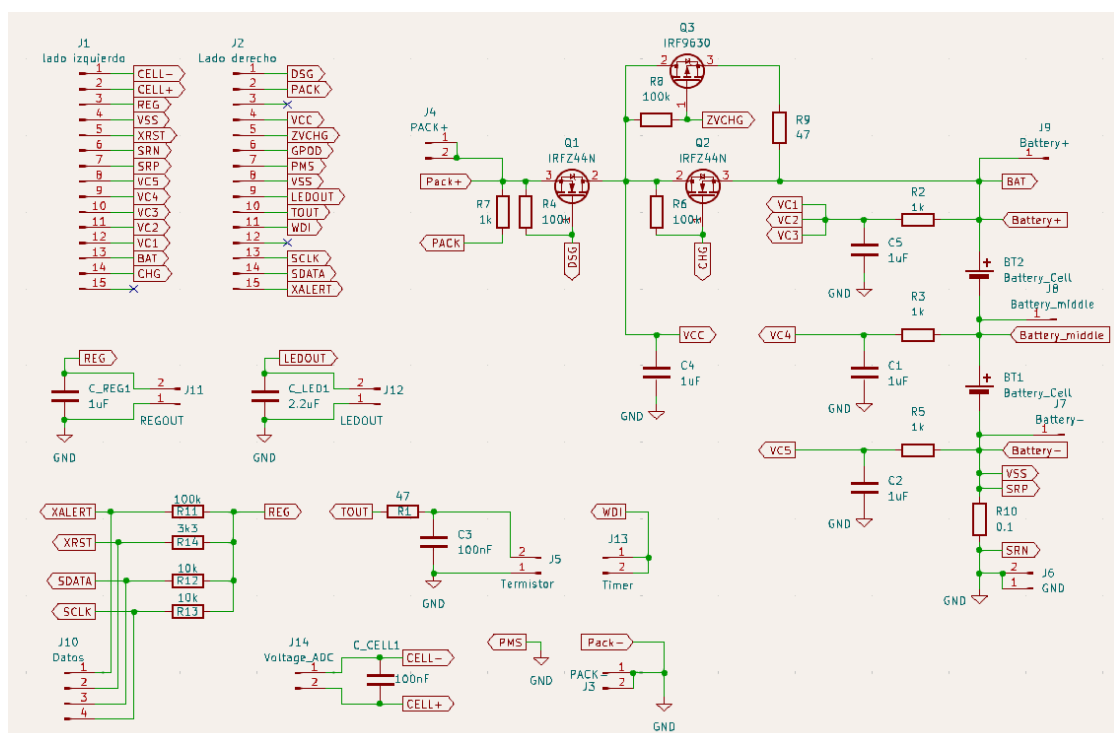
4.2. Primer Rediseño de módulo BMS



Análisis Detallado de la Evolución del BMS

El desarrollo del Sistema de Gestión de Batería (BMS) fue un micro-proyecto en sí mismo, marcado por una evolución en tres etapas. Cada transición fue impulsada por las limitaciones críticas descubiertas en el diseño anterior, llevándonos desde componentes comerciales restrictivos hasta una solución final totalmente personalizada y robusta.

Transición 1: De Módulo Autónomo (bq76905) a Supervisado (bq29330)



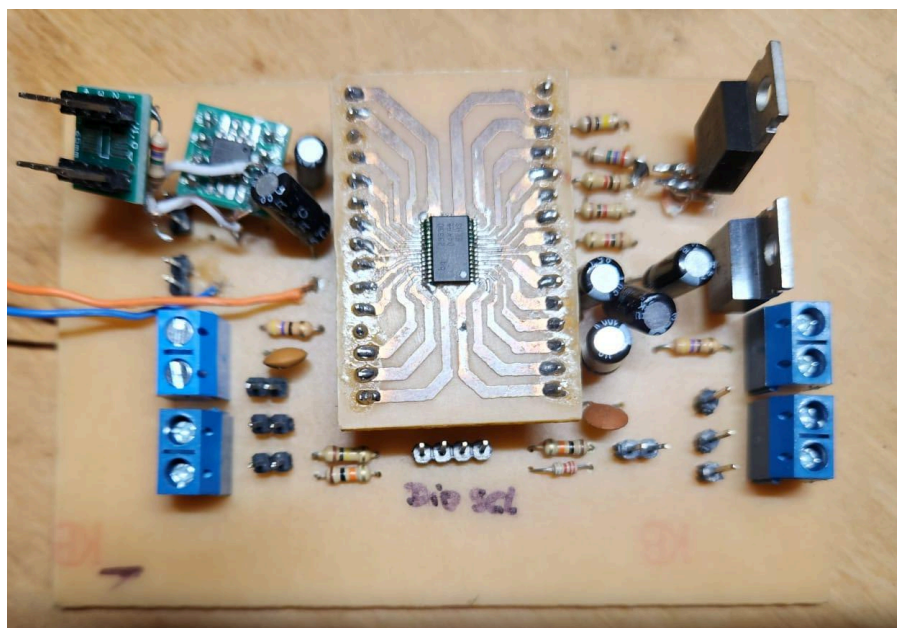
La primera versión del BMS se basó en el circuito integrado **bq76905**. La principal motivación para su elección fue la simplicidad: se presentaba como una solución "todo en uno" que manejaba las protecciones y el balanceo de forma autónoma.

Mejoras Buscadas (Del Diseño 1 al 2)

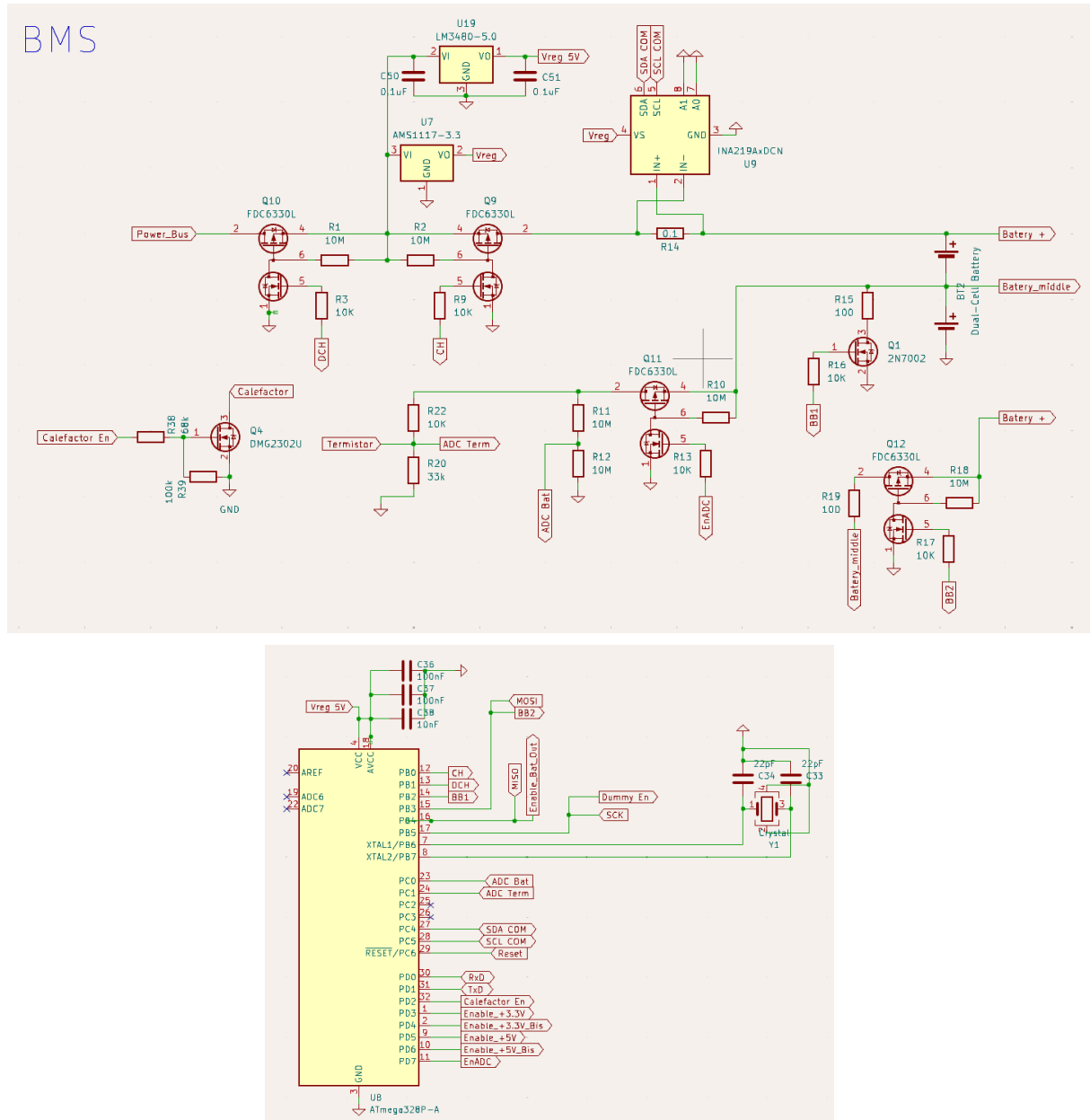
El problema fundamental del primer enfoque era su naturaleza de **caja negra** y su topología de protección. No teníamos control real sobre su comportamiento ni una forma eficiente de extraer datos, y su diseño presentaba riesgos. La transición al **bq29330** fue un intento deliberado de resolver esto. El objetivo era claro:

- **Corregir un Fallo Crítico de Seguridad:** El bq76905 realizaba el corte de protección por el lado negativo de la batería (low-side switching). En un sistema complejo como un satélite con una tierra común, esto representa un riesgo fundamental. Si cualquier otro subsistema creaba una ruta alternativa a tierra (por ejemplo, a través del chasis), la protección del BMS podía ser **completamente anulada**, dejando a las baterías indefensas ante una sobrecarga o descarga profunda. El nuevo diseño debía implementar high side switching (corte por el positivo) para ser intrínsecamente más seguro.
- **Obtener Control Granular:** A diferencia del primer chip, el BQ29330 estaba diseñado para ser gobernado por un microcontrolador externo. Esto, en teoría, nos daría la capacidad de ajustar umbrales, implementar lógicas de carga personalizadas y tener un control preciso sobre el proceso de balanceo.
- **Acceso a Telemetría Detallada:** Se buscaba poder monitorear el voltaje de cada celda individualmente y otros parámetros internos, algo que el primer diseño no permitía de forma sencilla.
- **Flexibilidad:** Pasar a un sistema supervisado nos abriría la puerta a adaptar el BMS a futuras revisiones o cambios en el paquete de baterías sin necesidad de cambiar el hardware principal.

En esencia, el paso al Diseño 2 fue un movimiento estratégico para corregir un grave fallo de seguridad y salir de un sistema rígido para pasar a uno que prometía la **flexibilidad y el control** que el proyecto demandaba.



4.3. Segundo Rediseño de módulo BMS



Si bien el BQ29330 parecía la solución ideal en papel, su implementación práctica reveló una serie de obstáculos insuperables que lo convirtieron en un callejón sin salida técnica.

Limitaciones Críticas del Diseño 2

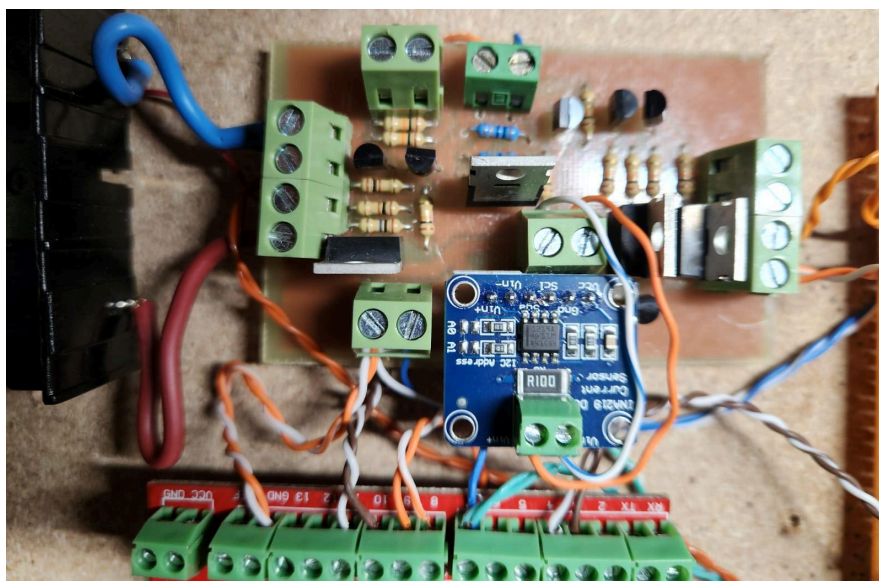
El segundo diseño fracasó no por su concepto, sino por su ejecución y las características del componente elegido. Los problemas eran de tal magnitud que hacían inviable su uso en un prototipo fiable:

- **Obsolescencia y Mal Soporte:** El principal problema fue el estado del componente. El **BQ29330** es un circuito integrado antiguo, con **documentación escasa e imprecisa**. Además, se descubrió que estaba diseñado para operar con un controlador específico de Texas Instruments que también se encontraba discontinuado. Intentar integrarlo con nuestro microcontrolador fue una tarea de ingeniería inversa muy compleja y poco fiable.
- **Dificultades de Prototipado Físico:** Al no poder fabricar una placa profesional para esta etapa modular, tuvimos que montarlo a mano. El chip tenía un encapsulado SMD muy pequeño con un alto número de pines. A esto se sumó que **no existían en el mercado nacional adaptadores de su encapsulado a formato THT**, lo que nos obligó a intentar soldar cables directamente a sus diminutos pines, un proceso extremadamente propenso a errores, cortocircuitos y daños en el componente.
- **Inestabilidad Operativa:** Durante las pocas pruebas que logramos realizar, el circuito presentaba un comportamiento errático, incluyendo **problemas con su watchdog interno**. Esto provocaba reinicios o bloqueos inesperados, una característica inaceptable para el BMS, que es el principal sistema de seguridad de las baterías.

Mejoras Obtenidas con el Diseño 3 (A Medida)

El fracaso categórico del segundo enfoque nos llevó a la solución final: diseñar un BMS desde cero con un microcontrolador dedicado. Este diseño no solo solucionó todos los problemas anteriores, sino que superó las expectativas iniciales:

- **Control Total del Firmware:** Eliminamos por completo la dependencia de un chip de terceros con documentación pobre. Toda la lógica reside en nuestro propio código, lo que nos da un **conocimiento absoluto de su funcionamiento** y la capacidad de depurar eficientemente.
- **Diseño Orientado a la Disponibilidad:** Se seleccionaron componentes (microcontrolador, MOSFETs, etc.) modernos, bien documentados y disponibles en encapsulados manejables para el prototipado manual o para los cuales existían adaptadores comerciales, eliminando el principal obstáculo físico del diseño anterior.
- **Fiabilidad Garantizada:** Al controlar el software, implementamos un sistema robusto y predecible. Los mecanismos de seguridad, como el watchdog, fueron diseñados y probados por nosotros, garantizando un comportamiento estable.
- **Interfaz de Alto Nivel:** Como se mencionó anteriormente, el control total nos permitió crear una interfaz por comandos **UART**, haciendo que la configuración y operación del BMS sea intuitiva y segura, algo que habría sido casi imposible de lograr con los complejos registros del BQ29330.



4.4. Rediseño de módulo PDU

Optimización de la Unidad de Distribución de Potencia (PDU)

A diferencia de otros subsistemas que requirieron rediseños completos debido a fallos funcionales, la Unidad de Distribución de Potencia (PDU) operó correctamente desde el primer prototipo. Sin embargo, durante la fase de refinamiento del diseño, se implementó una modificación estratégica con el objetivo de **aumentar la fiabilidad y la tolerancia a fallos del sistema**, adoptando prácticas comunes en la industria aeroespacial.

Diseño Inicial: Salidas Independientes

La primera arquitectura de la PDU estaba concebida de una manera sencilla y directa, proporcionando **cinco salidas de potencia independientes** para alimentar los distintos componentes del CubeSat:

- Dos salidas reguladas de 3.3V.
- Dos salidas reguladas de 5V.
- Una salida directa de la tensión de la batería (Vbat).

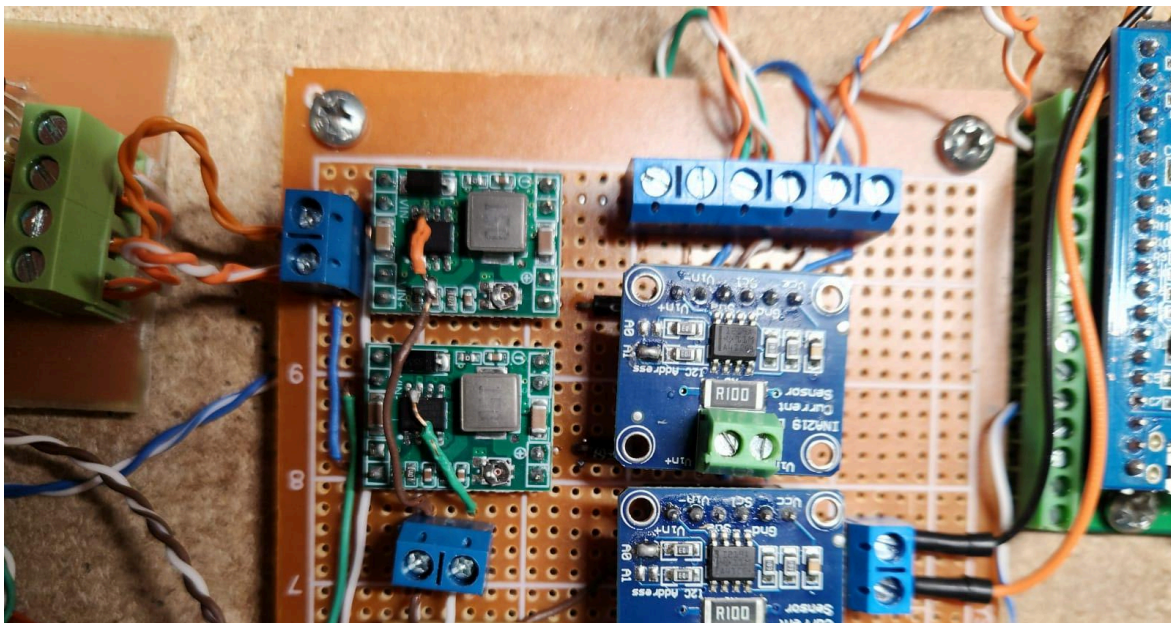
Este esquema era funcional, pero cada salida dependía de un único regulador. Un fallo en cualquiera de estos componentes comprometería permanentemente la alimentación de ese bus de tensión.

Diseño Final: Canales con Redundancia

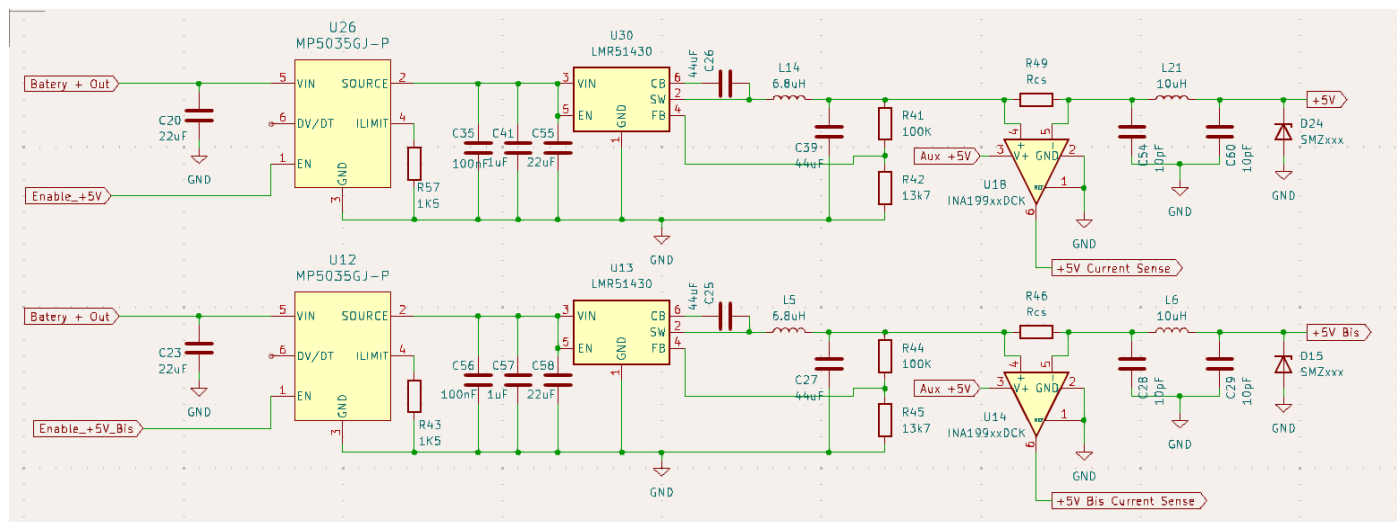
Para mitigar el riesgo de un punto único de fallo, se reestructuró la PDU hacia una arquitectura basada en **canales de potencia con redundancia**. Las cinco salidas se consolidaron en tres canales principales:

1. **Canal de 3.3V con Redundancia:** En lugar de dos salidas separadas, se implementó un canal de 3.3V alimentado por **dos reguladores de tensión idénticos**. Uno opera como la fuente **principal**, mientras que el segundo permanece en espera como **respaldo (backup)**. En caso de detectarse un fallo en el regulador principal (por ejemplo, una subtensión o sobretensión), el sistema tiene la capacidad de conmutar al regulador de respaldo, asegurando la continuidad del suministro de 3.3V.
2. **Canal de 5V con Redundancia:** Se aplicó la misma filosofía, utilizando dos reguladores de 5V para crear un canal de alimentación tolerante a fallos.
3. **Salida de Tensión de Batería (Vbat):** Esta salida se mantuvo sin cambios, ya que su función es proveer la tensión directa y no regulada de la batería, por lo que no depende de un regulador que pueda fallar y solo posee un smart switch de corriente para cortar por hardware de manera rápida.

Esta modificación, aunque no fue motivada por un error, representa un **salto cualitativo en la robustez del diseño**. Transforma a la PDU de una simple placa de distribución a un sistema de potencia resiliente, incrementando significativamente la probabilidad de supervivencia de la misión ante un eventual fallo de hardware en los reguladores de tensión.



Primer Diseño del PDU



Para dar seguridad al suministro de energía del resto del satélite, el PDU inicial contaba con fuentes duplicadas para cada valor de tensión ofrecido en la especificación. Esto incluía redundancia al sistema pero no permitía a la EPS controlar cómo se utilizaban esos respaldos.

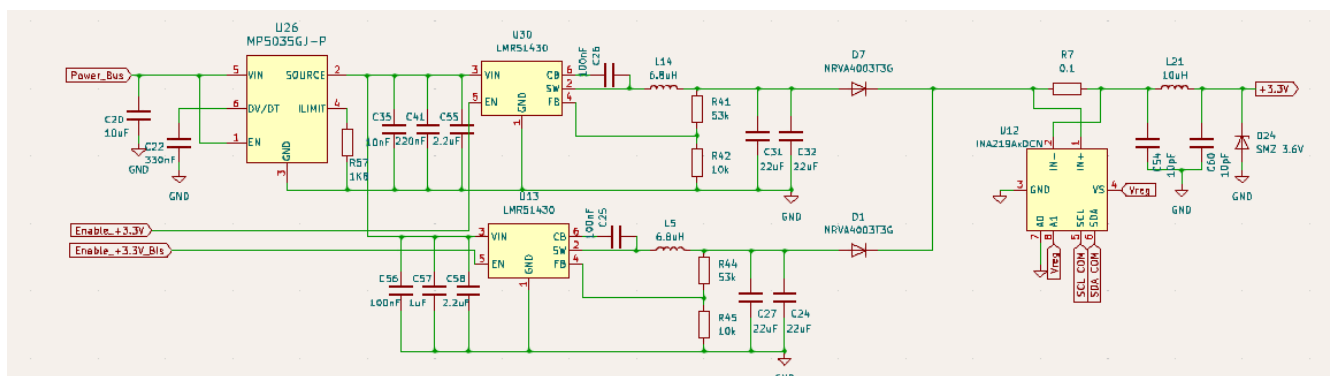
Monitoreo:

El monitoreo de tensión y corriente se realizaba mediante ADC's del microcontrolador central, esto se volvió problemático al operar junto con el BMS original ya que al realizar sus interrupciones de suministro por el negativo, el resto del circuito perdía su referencia por la tensión añadida sobre la llave de control de flujo.

Protecciones:

5. Sobreconsumo: al monitorearse las corrientes de salida es posible realizar cortes por sobreconsumo si algún sistema del satélite empieza a manifestar consumos anormales.
6. Cortocircuito: Los smart switch MP5035GJ-P permiten fijar una limitación de corriente para añadir a las salidas una protección de reacción rápida a excesos de corriente. El mismo cuenta con un modo Hicup al detectar un cortocircuito que permite reintentar una devolución de corriente y un corte inmediato si el comportamiento anormal resulta persistente.
7. Compatibilidad electromagnética: Los filtros Pi permiten incluir una protección bidireccional contra ruido de alta frecuencia. Permite proteger tanto a la EPS del resto del circuito como al resto del circuito de cualquier ruido de alta frecuencia que pueda generarse en la EPS.
8. Sobretensión: Se incluye un zener de tensión levemente por encima de la especificada para incluir, además del monitoreo y deshabilitación, una protección rígida que evite excesos de tensión en las salidas.

8.1.1. Diseño final del PDU



En el diseño final del PDU se incluyeron varias mejoras que lo convierten en una alternativa superadora del diseño inicial.

Se decidió aplicar una redundancia controlada mediante la habilitación de una fuente de backup para cada salida especificada manteniendo las protecciones ya implementadas y/o suplantadas por versiones superadoras.

Monitoreo:

Mediante el uso de sensores INA219, no solo se redujo la cantidad de valores a conservar a la mitad por la unificación de salidas sino que además se logró reducir su susceptibilidad al ruido mediante mediciones in situ realizadas por el integrado y comunicadas por I2C.

Protecciones:

9. Sobreconsumo: Al monitorear mediante el uso de sensores digitales que realizan mediciones in situ, las protecciones de sobreconsumo son menos susceptibles a ruido y permiten utilizar una sola medicion
10. Cortocircuito: Los smart switch MP5035GJ-P se mantienen y conservan sus beneficios mientras su cantidad se reduce a la mitad al haber unificado las fuentes en una sola salida respaldada
11. Compatibilidad electromagnética: Los filtros Pi se conservan pero su cantidad se reduce a la mitad al haber unificado las fuentes en una sola salida respaldada.
12. Sobretensión: Los zener se conservan pero su cantidad se reduce a la mitad al haber unificado las fuentes en una sola salida respaldada.

13. Versión Final

Reingeniería del Subsistema de Adquisición de Datos

Un pilar fundamental en la operación de una EPS es su capacidad para monitorear en tiempo real los parámetros críticos de energía. Durante el desarrollo del prototipo, este subsistema experimentó una reingeniería completa para optimizar su eficiencia y fiabilidad.

Enfoque Inicial: Mapeo Directo por ADC

El concepto de diseño preliminar se basaba en un enfoque tradicional de adquisición de datos. La medición de los niveles de tensión y corriente en los distintos puntos del sistema (entradas de la PPC, estado del BMS y salidas reguladas) se realizaba a través de divisores de tensión y amplificadores operacionales conectados directamente a las entradas del **convertidor analógico-digital (ADC)** del microcontrolador principal.

Si bien funcional, este método presentaba desventajas significativas:

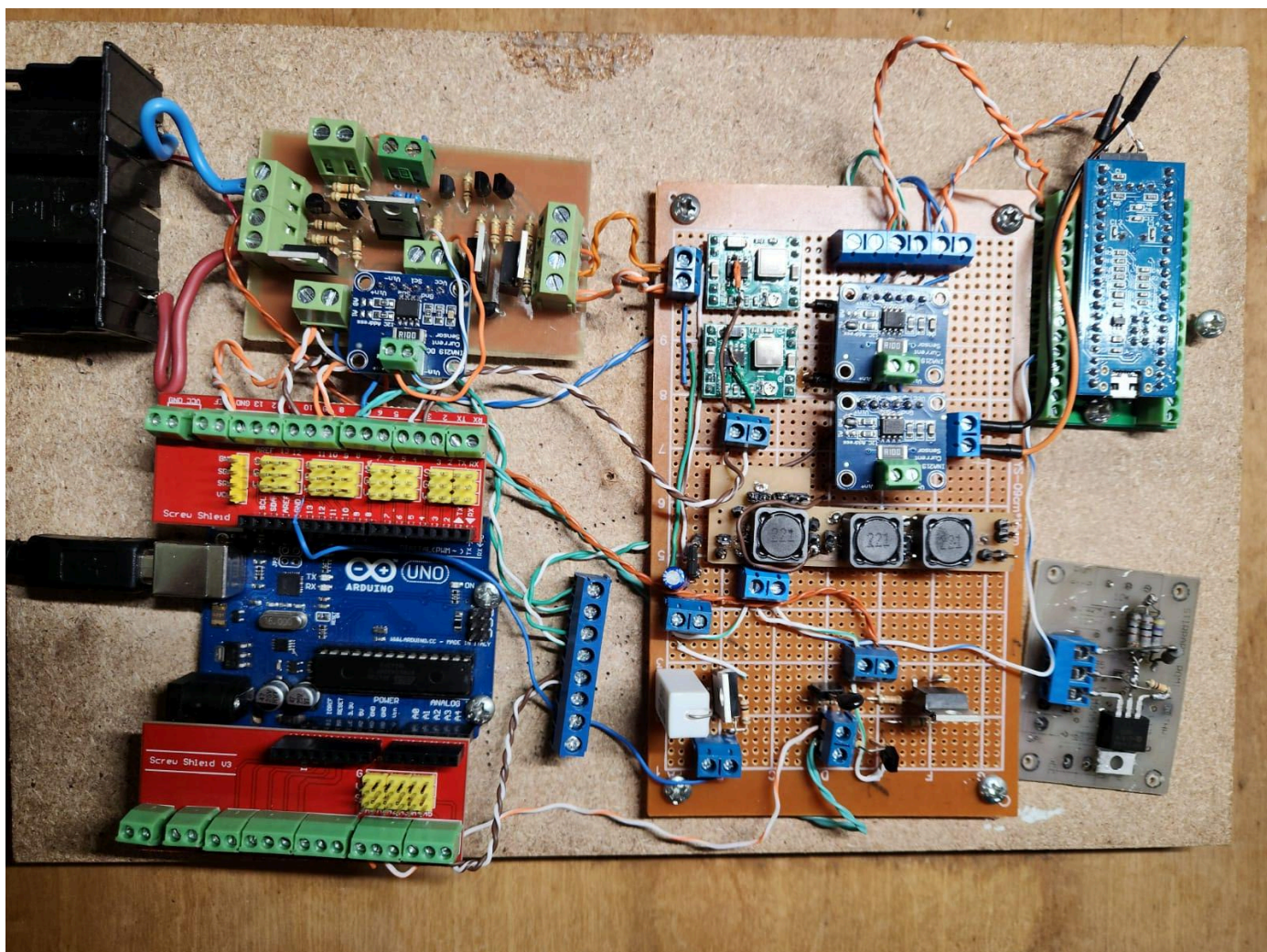
- **Alto Consumo de Pines:** Requería un pin de ADC exclusivo para cada señal, agotando rápidamente los recursos disponibles del microcontrolador.
- **Complejidad del Circuito:** Exigía circuitería de acondicionamiento de señal para cada medición, aumentando el número de componentes y la complejidad del PCB.
- **Susceptibilidad al Ruido:** Las largas pistas analógicas desde los puntos de medición hasta el microcontrolador eran susceptibles a la interferencia electromagnética.

Transición a una Arquitectura Digital con Bus I²C

Para solventar estas limitaciones, se tomó la decisión de migrar hacia una arquitectura de monitoreo digital y distribuida, implementando un **bus de comunicación I²C**.

La pieza central de esta nueva topología es el sensor de corriente y potencia **INA219**. Este dispositivo integra en un solo encapsulado un amplificador de alta precisión y un ADC de 12 bits, capaces de medir la tensión del bus y la caída de tensión en una resistencia shunt para calcular la corriente. El resultado de estas mediciones se transmite de forma digital a través del bus I²C.

La implementación consistió en colocar un sensor INA219 en cada uno de los puntos de interés, permitiendo un monitoreo exhaustivo de las tres entradas, el BMS y las tres salidas de potencia. El microcontrolador, actuando como maestro, se comunica ahora con todos estos sensores (esclavos) utilizando únicamente dos líneas de comunicación (SDA y SCL), simplificando drásticamente la interconexión y liberando la mayoría de los pines que antes se usaban. Esta modificación representó un salto cualitativo en la robustez, precisión y eficiencia del diseño final.



14. Controladores

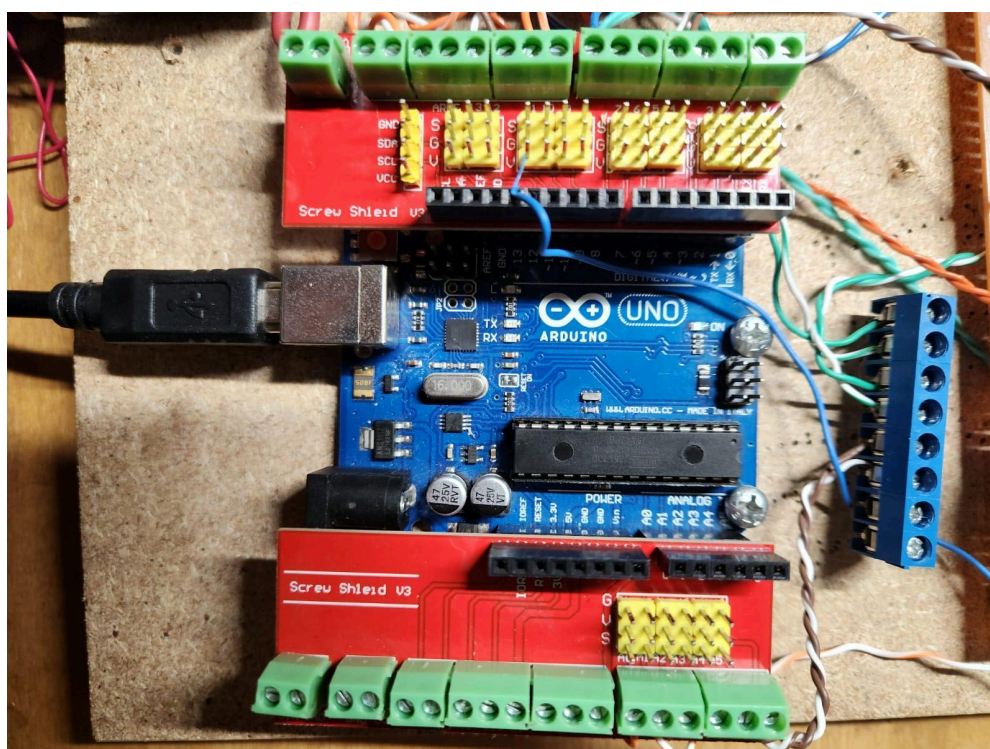
Arquitectura de Control y Microcontroladores Seleccionados

La arquitectura de control del sistema EPS se basa en un enfoque de procesamiento distribuido, utilizando dos microcontroladores específicos que operan en una relación de maestro-esclavo. Para la fase de prototipado, se utilizaron placas de desarrollo comerciales (como Arduino y Blue Pill) que contienen estos chips, ya que resultaba innecesario y poco práctico fabricar una placa a medida solo para la validación del diseño.

ATmega328P (Controlador Lógico Principal)

El cerebro principal del sistema es el microcontrolador de 8 bits **ATmega328P** de Microchip (anteriormente Atmel). Este componente fue elegido por su simplicidad, robustez y el maduro ecosistema de desarrollo que lo rodea.

- **Rol en el Proyecto:** Actúa como el controlador maestro, donde reside la lógica principal del sistema. Es responsable de la toma de decisiones, la ejecución de algoritmos de protección, la gestión de la PDU y la centralización de la comunicación con el usuario.
- **Ecosistema de Desarrollo:** Aunque es un microcontrolador independiente, se programó utilizando el **ecosistema Arduino** (Arduino IDE y sus librerías). Este framework abstrae la complejidad del manejo de registros de bajo nivel y acelera enormemente el desarrollo de la lógica de control y, sobre todo, de la interfaz de comunicación.
- **Capacidades Clave Utilizadas:**
 - **UART:** Se empleó su interfaz serie para crear la consola de monitoreo y comandos, indispensable para la depuración y validación.
 - **GPIO:** Se utilizaron sus pines de entrada/salida de propósito general para manejar las señales de control y comunicarse con el co-procesador STM32.

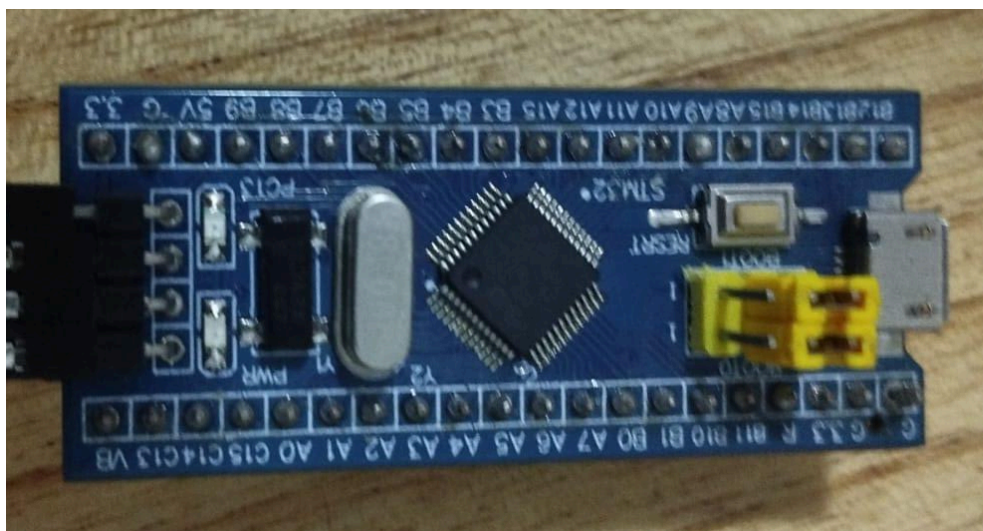


STM32F103C8T6 (Co-Procesador de Hardware)

Para las tareas que exigen un control de hardware de alta velocidad y precisión, se seleccionó el microcontrolador de 32 bits **STM32F103C8T6**, basado en la arquitectura ARM Cortex-M3.

- **Rol en el Proyecto:** Funciona como un co-procesador o esclavo inteligente. Su única misión es ejecutar las tareas de hardware críticas que le ordena el ATmega328P. Esto descarga al controlador principal, permitiéndole centrarse en la lógica de decisión.
- **Ecosistema de Desarrollo:** Se programó utilizando el entorno profesional **STM32CubeIDE**. Esta herramienta permite una configuración detallada y optimizada de los periféricos de hardware del chip.
- **Capacidades Clave Utilizadas:**
 - **Temporizadores Avanzados:** La razón principal de su elección. Sus temporizadores son capaces de generar señales **PWM de alta frecuencia** con gran precisión, algo fundamental para el control eficiente del convertidor boost del PPC.
 - **Periférico I²C por Hardware:** Se utilizó su controlador I²C por hardware para gestionar de forma eficiente la comunicación en el bus con los múltiples sensores **INA219**, recopilando la telemetría y enviándola al ATmega328P cuando este la solicita.

En resumen, la arquitectura combina la simplicidad y rapidez de desarrollo del **ATmega328P** para la lógica general, con la potencia y las capacidades de hardware de alta velocidad del **STM32F103** para las tareas físicas especializadas.



15. Descripción Funcional

La fuente de alimentación está diseñada para satélites pequeños y de bajo costo con demandas de potencia de 1-8.5 W.

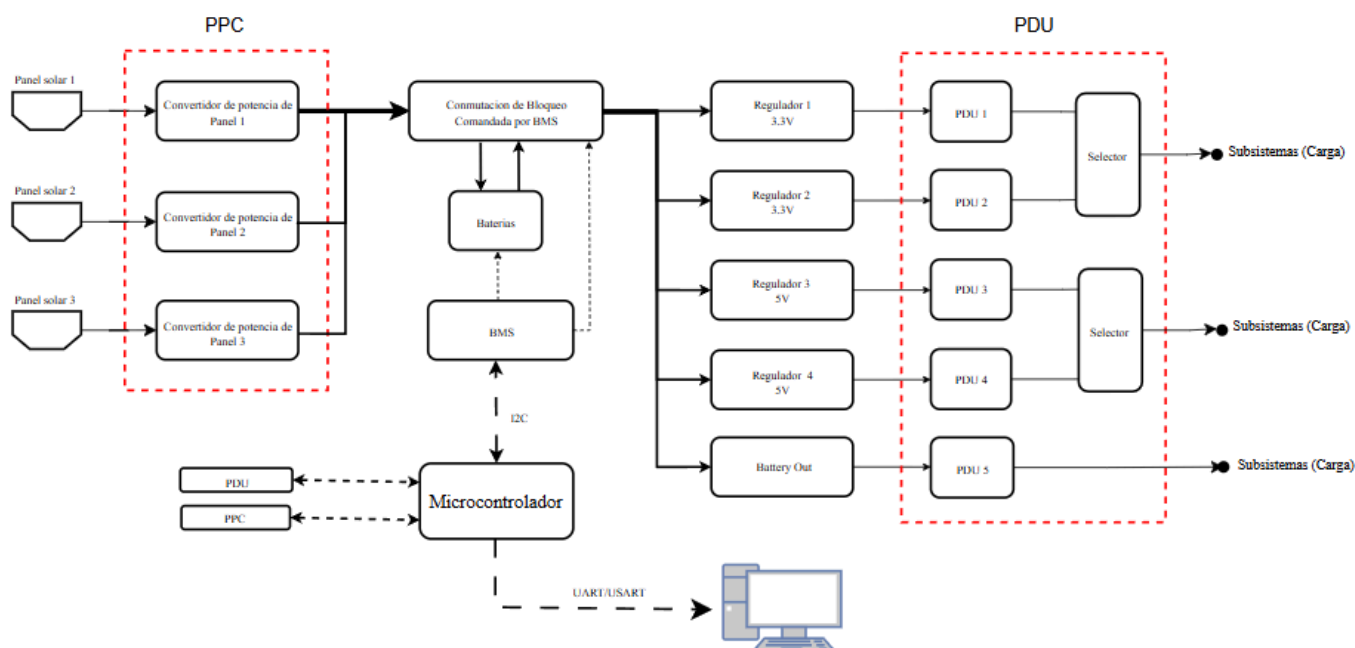
El circuito interactúa con las células fotovoltaicas ubicadas en las diferentes caras del satélite a través de una fuente conmutada de alta eficiencia. Este diseño se implementa con el objetivo de maximizar la potencia de salida y garantizar una carga óptima de las baterías proporcionadas.

15.1. Características Generales

- Posee 3 canales de entrada independientes que pueden proporcionar hasta una potencia total de entrada de 5.25W.
- Soporta dos baterías de 3.7V (2600mAh) en serie.
- Posee seguimiento del punto de máxima potencia usando el algoritmo MPPT.
- Posee protección contra baja y sobre tensión de la batería.
- Posee 3 buses de alimentación regulados: 3.3V@0.6A (x2), 5V@0.23A (x2) y @0.4A.
- Las salidas de 3.3V y 5V poseen redundancia.
- Soporta una Potencia Máxima Total de 7.5W.
- Las fuentes de salida de la PDU Poseen limitadores de corriente.
- Monitoreo del estado de carga y temperatura de las baterías.
- Interfaz de comunicación UART/USART.
- Temperatura de funcionamiento: de -30 a +70 °C.
- Dimensiones de la placa: 96 mm x 90mm x 25.2mm.
- Se ajusta a la norma PC104.

15.2. Diagrama en Bloques

A continuación se muestra el diagrama en bloques de los distintos módulos que posee el sistema EPS



Aclaración: Este diagrama es un esquema básico que describe el comportamiento del sistema, sin embargo, en el apéndice 2, se encuentra una versión más completa que incluye la descripción de los módulos PPC y PDU.

15.3. Microcontrolador

El circuito cuenta con un microcontrolador que controla que las baterías aprovechen al máximo la energía dispuesta por los paneles (MPPT). Además, este registra las tensiones, corrientes y temperaturas del sistema, permite el control de usuario, etc.

Utilizando una interfaz I2C es posible leer las mediciones, controlar el estado de encendido/apagado de los buses del PDU, encender/ apagar el MPPT y establecer/leer parámetros del sistema.

15.4. Paneles solares

El sistema está diseñado para soportar tres canales de entrada fotovoltaica individuales.

En satélites con hasta tres paneles solares en la luz del sol, este permite que el voltaje se ajuste independientemente por cada canal logrando así fijar el punto de potencia máxima exacta en todas las células iluminadas.

Si se utiliza satélites con forma cúbica como un CubeSat, simplemente conectando a cada canal pares de paneles solares montados en caras opuestas se permitirá la conversión individual de la potencia de todas las células en luz solar.

La fuente switching de los PPC está diseñada para manejar hasta 650 mA corriente de entrada. Cada panel debe tener un diodo de protección en serie para evitar que los paneles no iluminados extraigan corriente de los iluminados.

16. Especificaciones

En esta sección se muestran las especificaciones de los distintos módulos de la EPS, en particular se muestran los valores por defecto, sin embargo, existen algunos valores configurables por el usuario (comandables mediante la computadora de abordo), la manera de comandar estos valores se detalla en la sección de **Interfaz de Datos y Comandos**.

16.1. PPC

Photovoltaic Power Converter	Característica	Min	Max	Typ	Unit
Seguimiento del Máximo Punto de Potencia (MPPT)					
Protección contra baja y sobre tensión					
Configuración Boost					
Compatible con configuraciones de baterías 2S1P y 2S2P					
Carga Estándar	Tensión (CCCV)	-	8.2	-	V
	Corriente (CCCV)	-	650	-	mA
	Tiempo de Carga	214	-	-	min

- CCCV: Constant Current Constant Voltaje

16.2. BMS

Battery Management System
Protección por sobre y baja tensión
Protección por sobre corriente
Comunicación I2C
Permite obtener el estado de carga
Monitoreo de temperatura
Permite balancear las baterías
Temperatura Operacional: -40°C a 110°C
Monitoreo del porcentaje de carga

16.3. Reguladores con PDU

Salidas	Característica	Min	Max	Typ	Unit
Cantidad de Salidas: 4+1					
Tensiones: 3.3, 5, Bat					
Salida 1	Tensión	3.2	3.4	3.3	V
	Limitación de Corriente	-	0.75	-	A
Salida 2	Tensión	4.9	5.1	5	V
	Limitación de Corriente	-	0.65	-	A
Salida 3	Tensión	6.6	8.2	7.2	V
	Limitación de Corriente	-	0.41	-	A
Reguladores 1-2-3-4-5	Limitación de Corriente total simultánea	-	850	-	mA
	Tiempo de Respuesta de limitación	-	100	-	ms

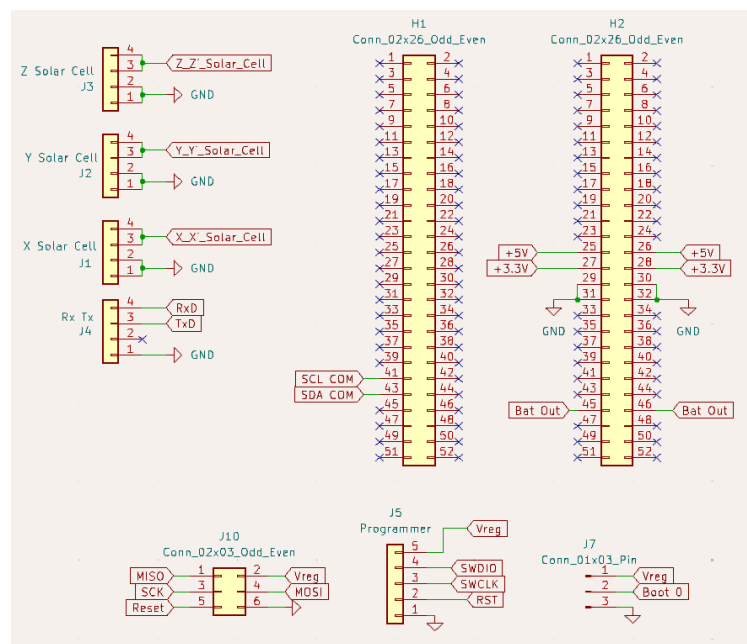
16.4. Paneles – P110 GOMspace

Solar Panels	Min	Max	Typ	Unit
Eficiencia máxima de 30%				
Hasta 2.3W en Orbita LEO				
Dimensiones: 98mm x 98mm x 1.6mm				
Características Eléctricas para Full sunlight in LEO:				
Tensión Óptima	4.64	4.84	-	V
Corriente a Tensión Óptima	490	508		mA
Temperatura Operacional	-40	85	-	°C
Potencia	2270	2400	-	mW
Eficiencia	29.8	30.2	30	%

16.5. Baterías - Battery 18650 Samsung

Battery	Min	Max	Typ	Unit
Tipo: Litio				
Capacidad: 2500mAh				
Configuración: 2 Baterías en Serie				
Dimensiones: Altura=65mm, Diámetro=18.4mm				
Tensión nominal	-	-	3.7	V
Carga Estándar	-	-	4.2	V
CCCV	-	1.3	-	A
Tensión de corte de descarga	3	-	-	V
Tiempo de Carga	180	-	-	min

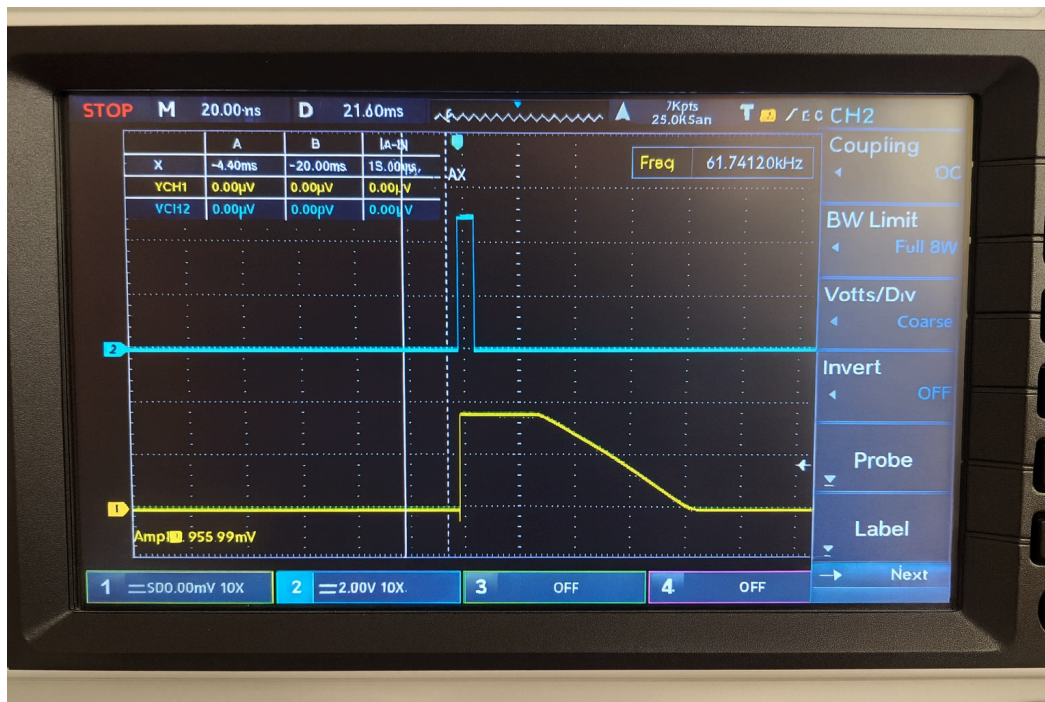
17. Conexiones



18. Desempeño

18.1. Medición de Señales con ADC:

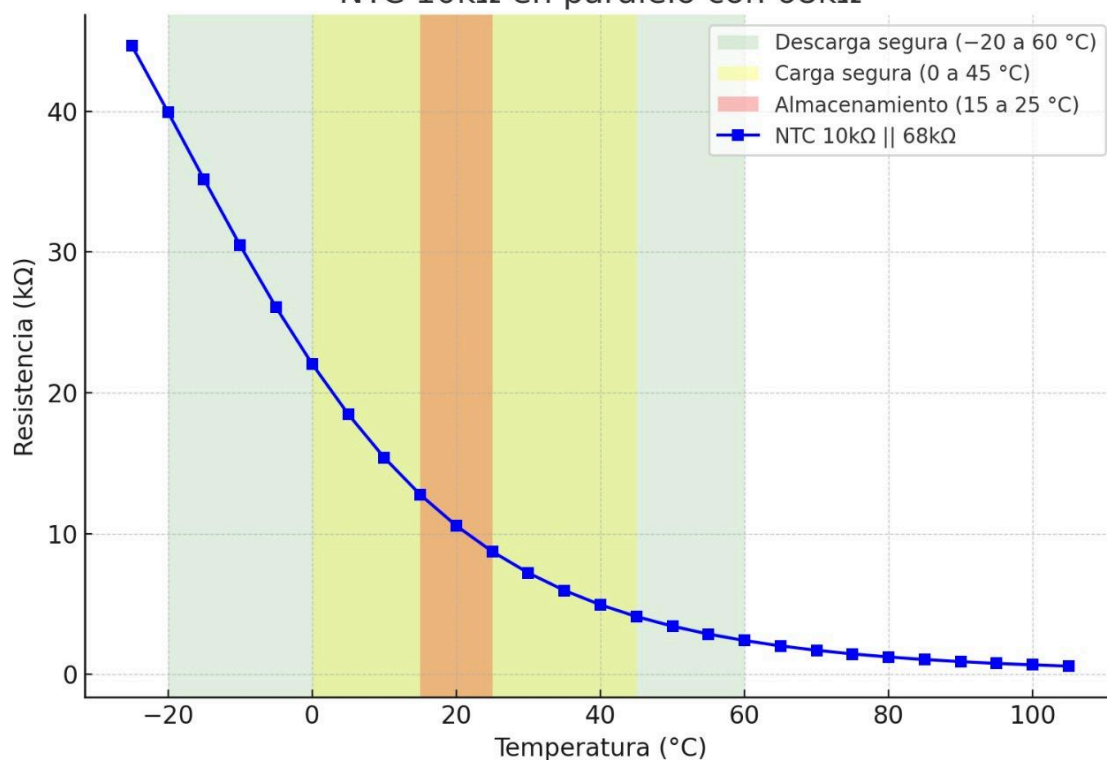
En este grafico en azul se representa la señal de habilitacion del transistor para poder medir la señal con el ADC, en amarillo tenemos la señal a medir, este metodo nos permite consumir menos corriente ya que el uc permanece dormido hasta que tiene que medir tension y sus pines de ADC se quedan apagados.



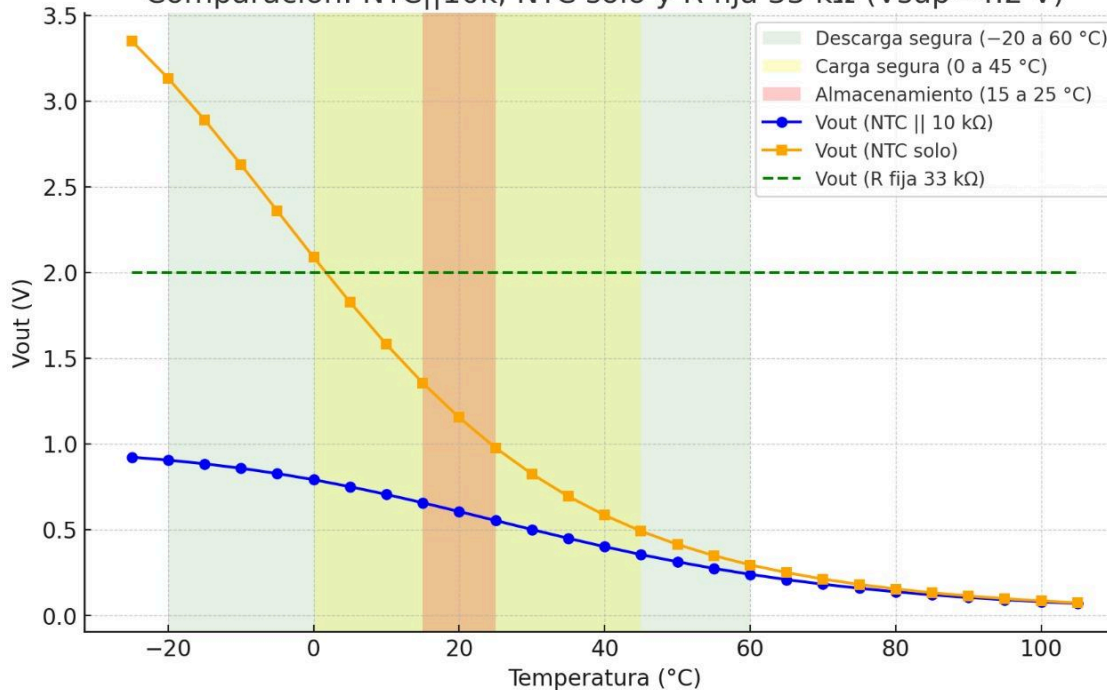
18.2. Medición de Temperatura

En estas curvas vemos la respuesta de la resistencia NTC dentro de los rangos de seguridad para carga y descarga de las baterías.

Resistencia vs. Temperatura (k Ω)
NTC 10k Ω en paralelo con 68k Ω

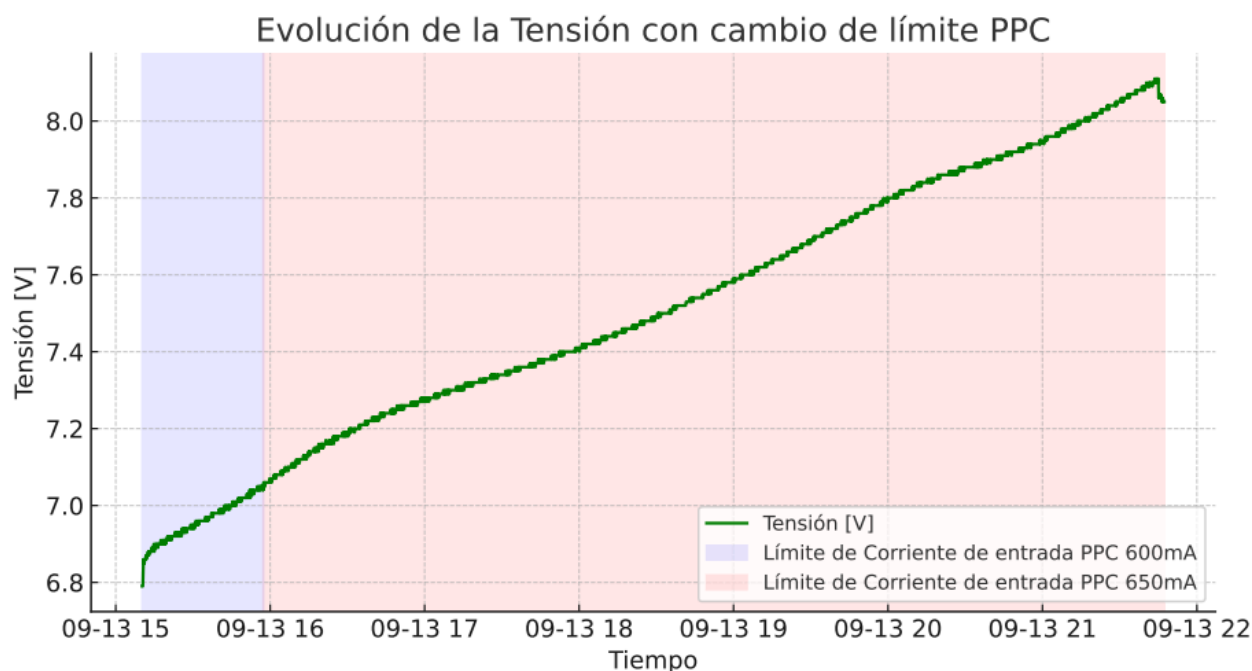


Vout vs Temperatura con Rpull=33 k Ω
Comparación: NTC||10k, NTC solo y R fija 33 k Ω (Vsup=4.2 V)

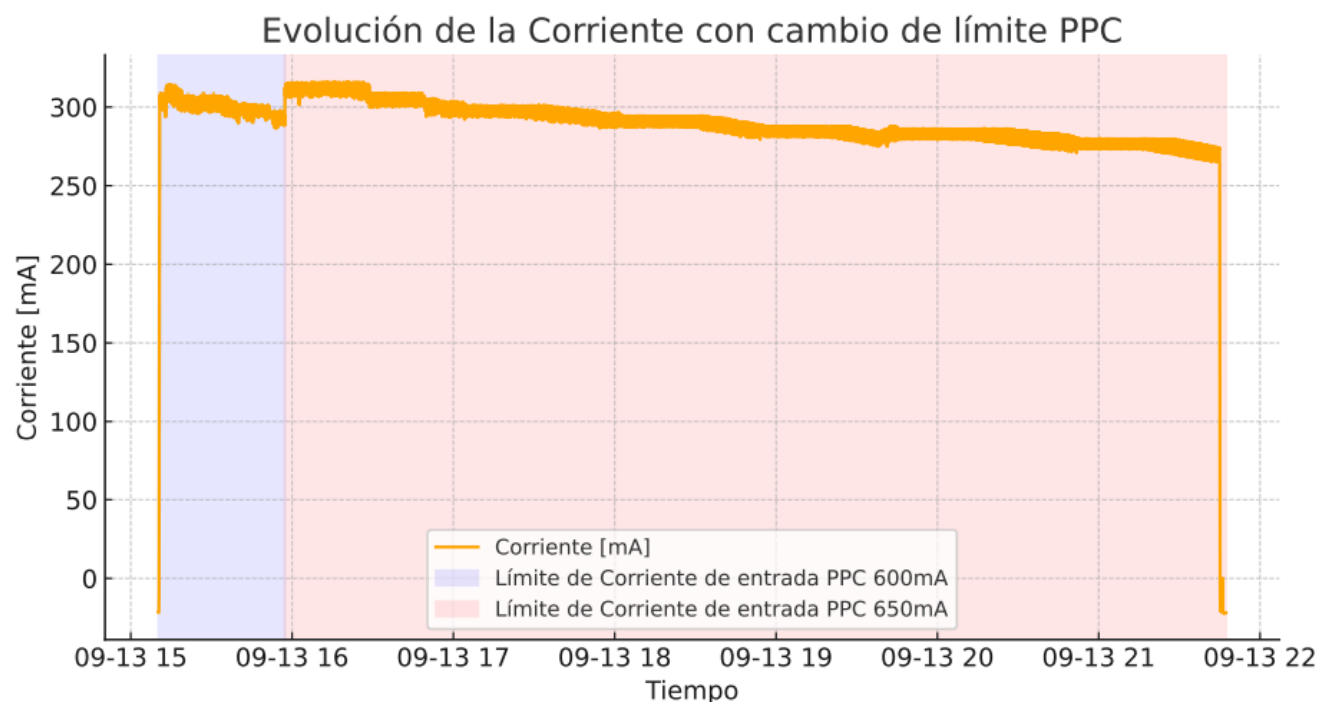


18.3. Carga Completa de las Baterías

En este gráfico vemos que se comienza a cargar las baterías en 3 V y se termina de cargar en cuando llegan a 4.1V. Se puede observar la resistencia parásita de las baterías en las caídas de tensión al dejar de entregarles corriente.



En el siguiente gráfico se ve la corriente que ingresa a las baterías, a la entrada de los paneles tenemos 600 mA y 4.5V y en las baterías tenemos 300mA y 7V en promedio. Por ende a medida que la tensión de las baterías sube la corriente que ingresa va bajando.

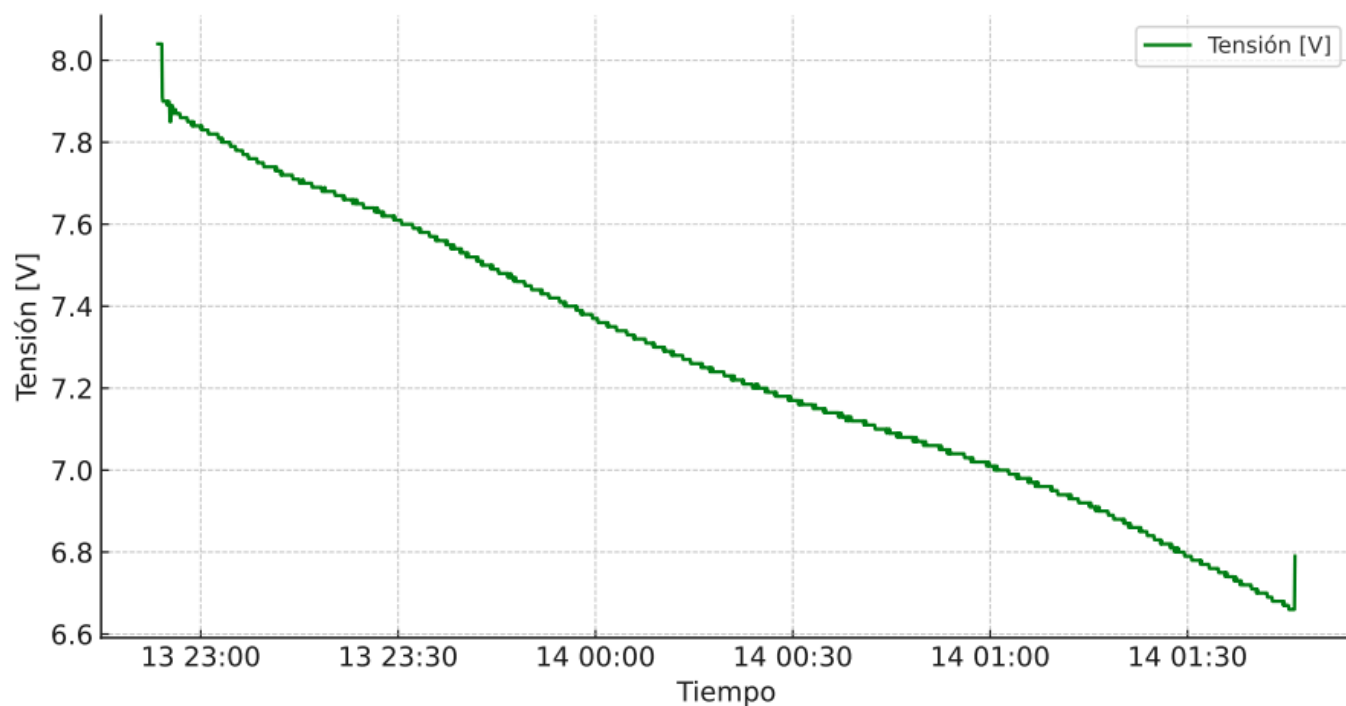


En la Datasheet del fabricante de las celdas Samsung ICR18650-26J tenemos 2,500 mAh entre 2.75 V y 4.2 V en estos experimentos no se realizó la carga completa para no exigir de más a las celdas. Por lo que el resultado de 1910.5 mAh está dentro de lo esperable.

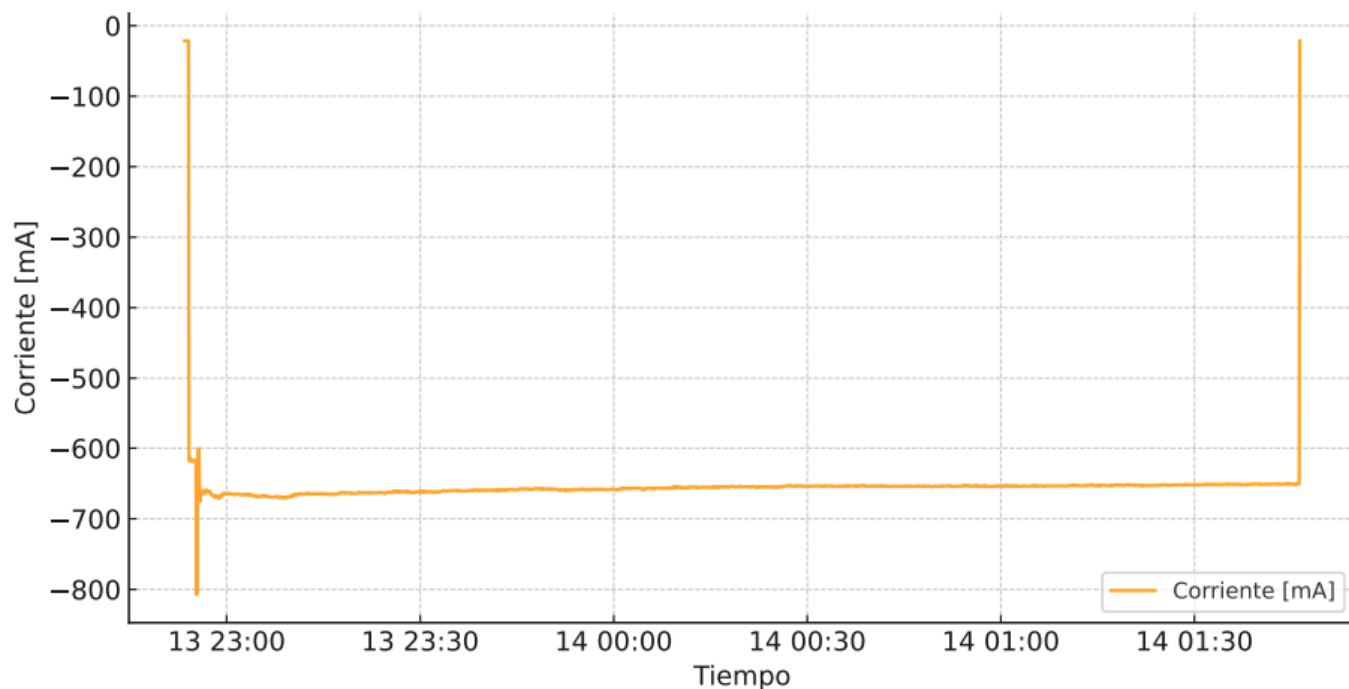


18.4. Descarga Completa de las Baterías

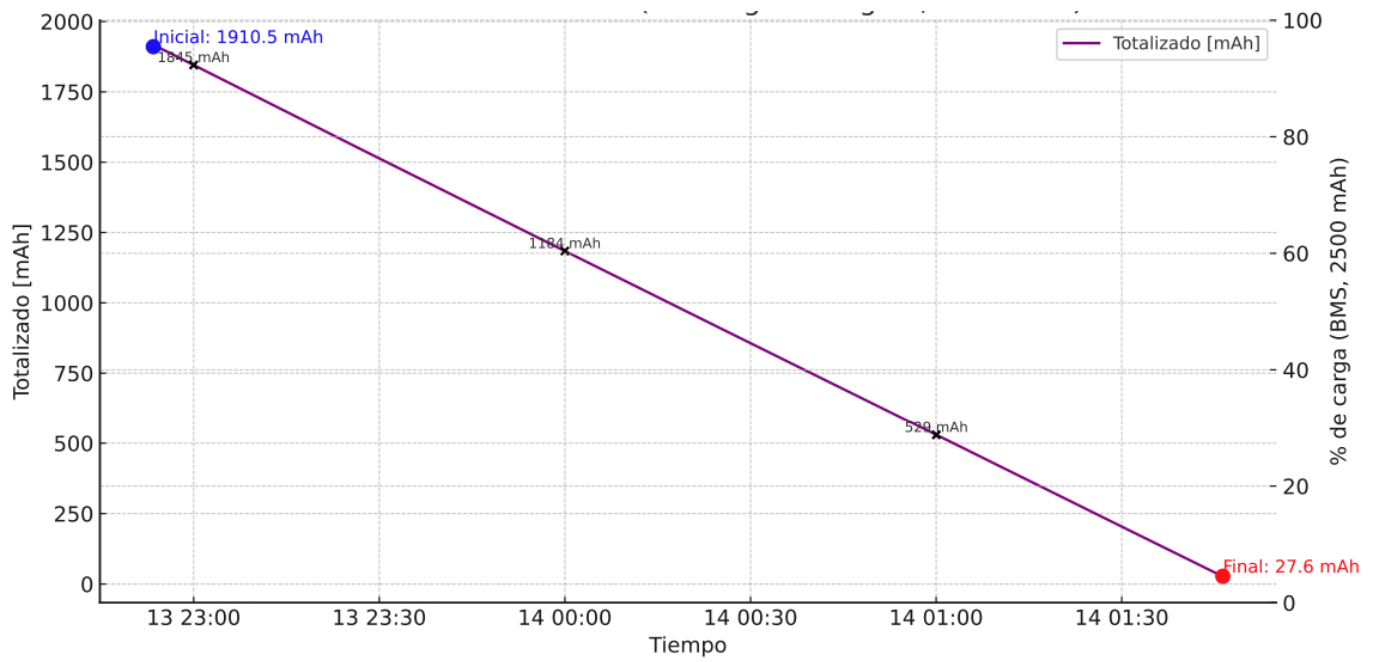
Evolución de la tensión:



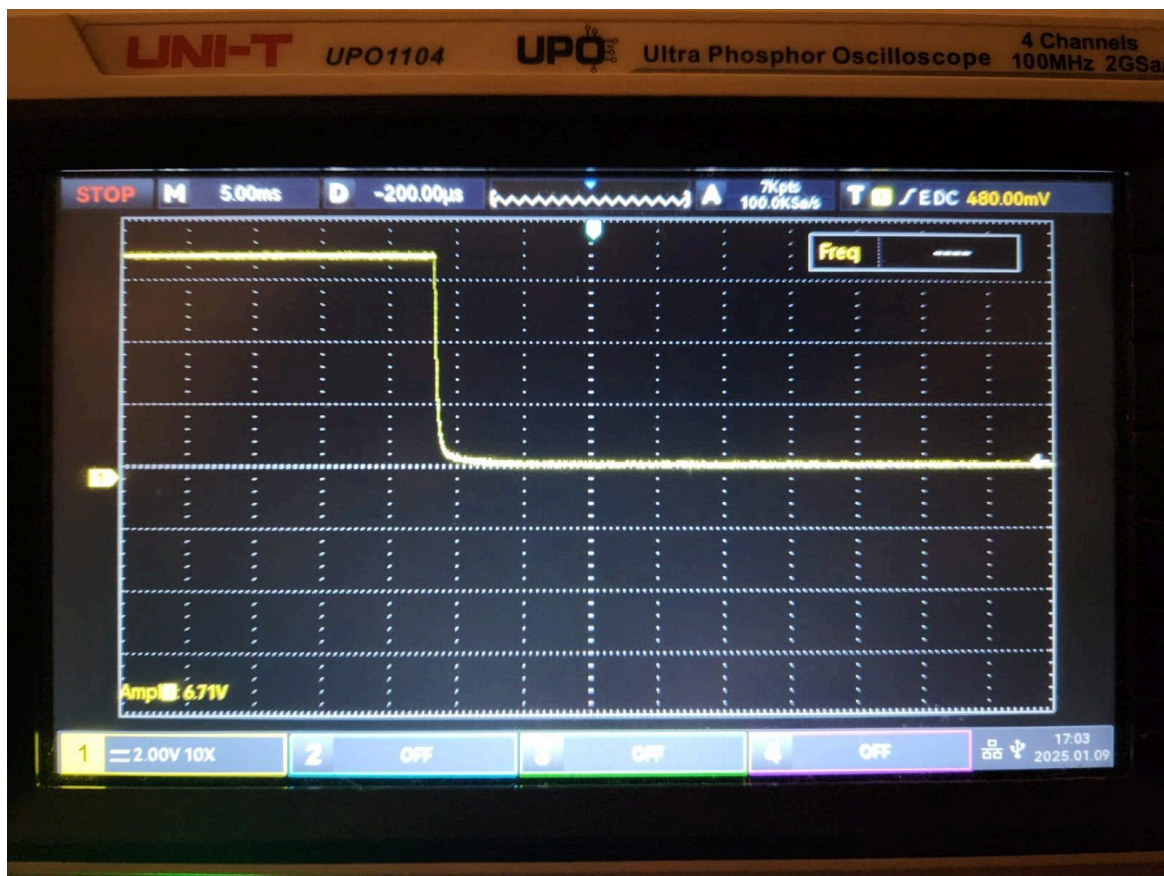
Evolución de la corriente:



Evolución del coulomb counter en mAh

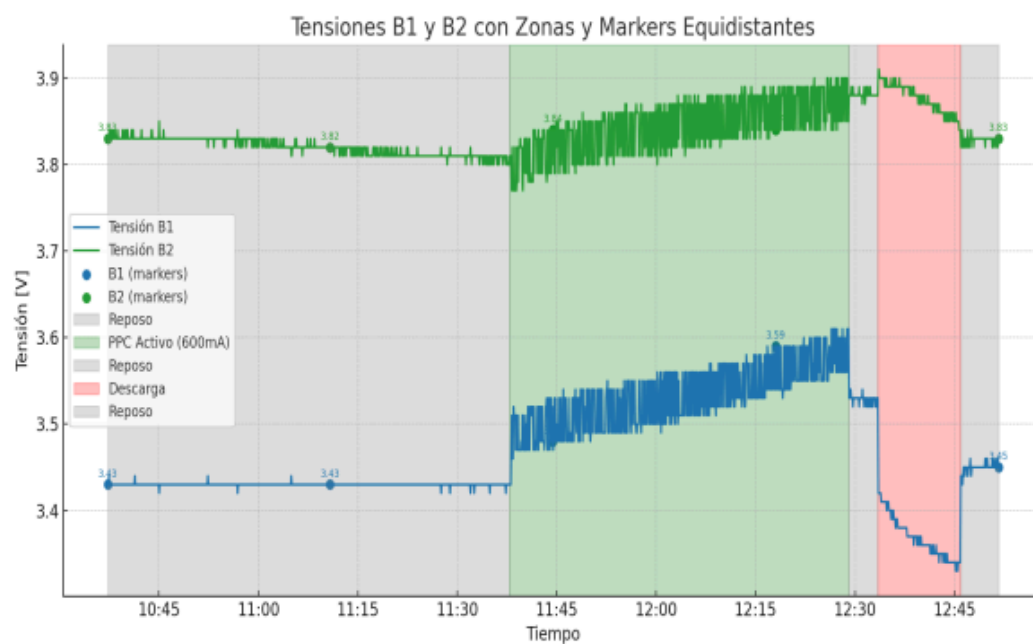


Corte Detectado con Osciloscopio:

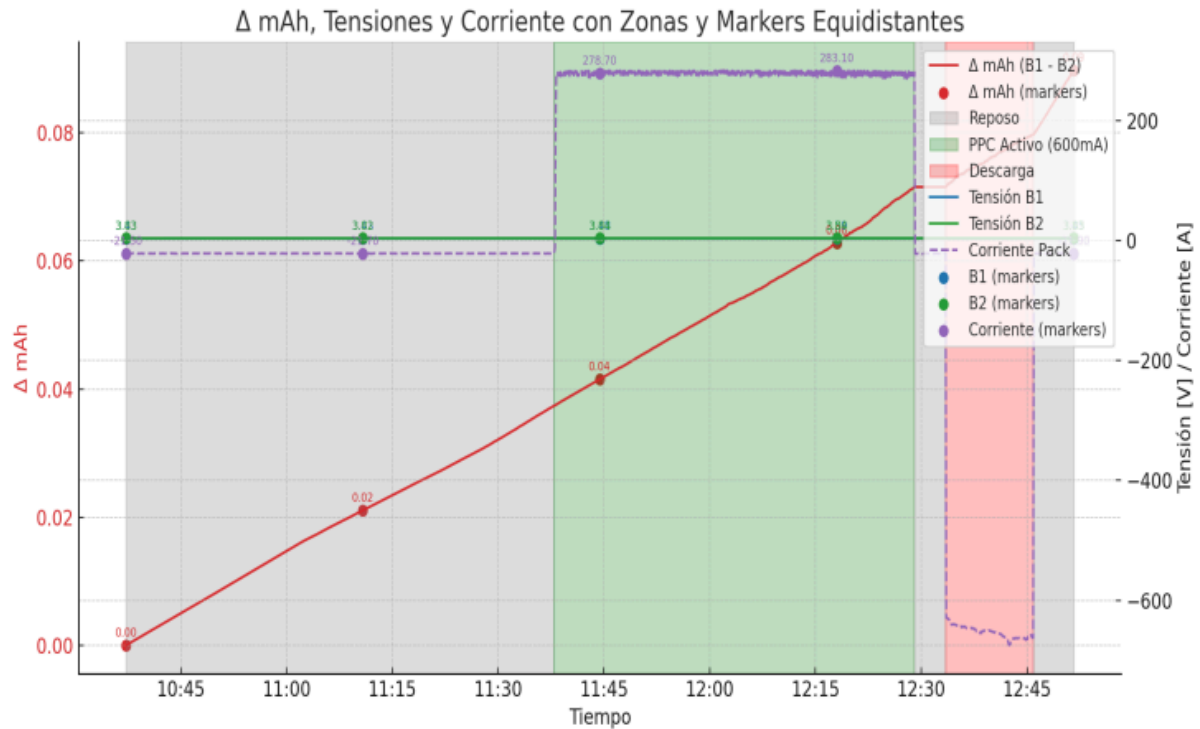


18.5. Balanceo de Baterías:

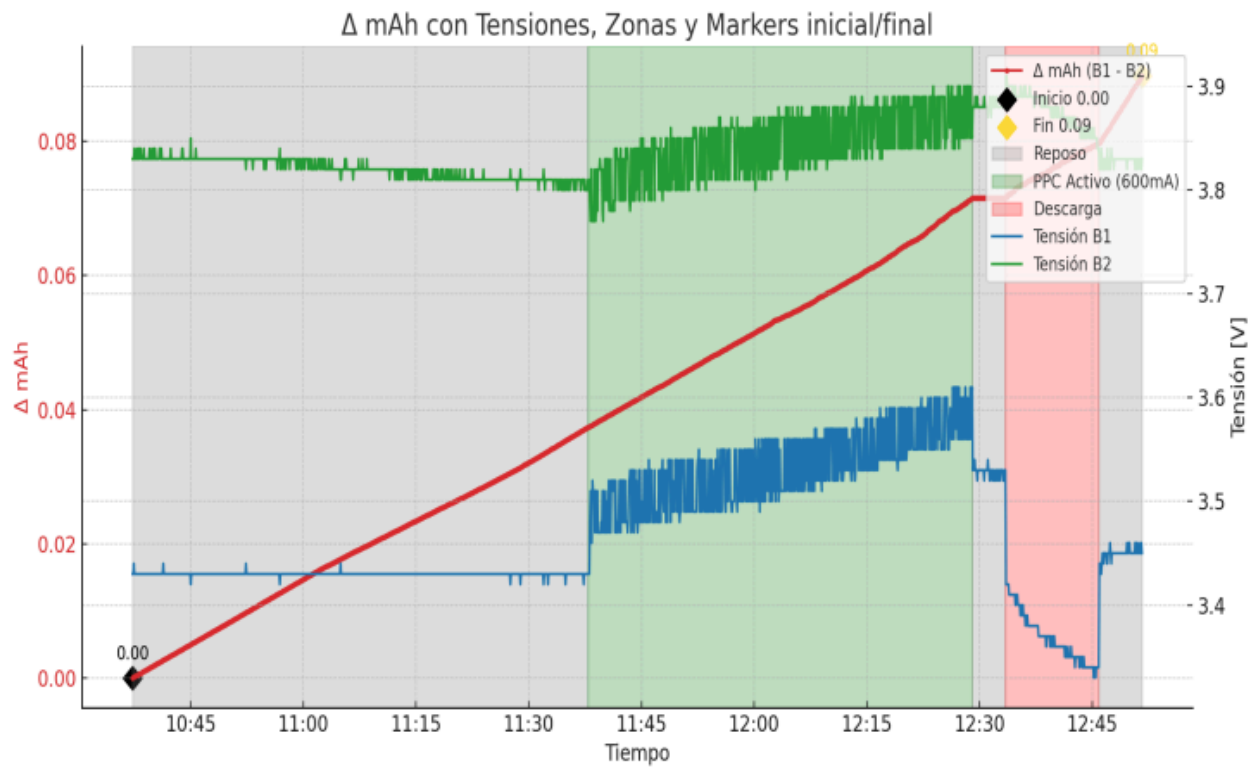
Tensiones de B1 y B2



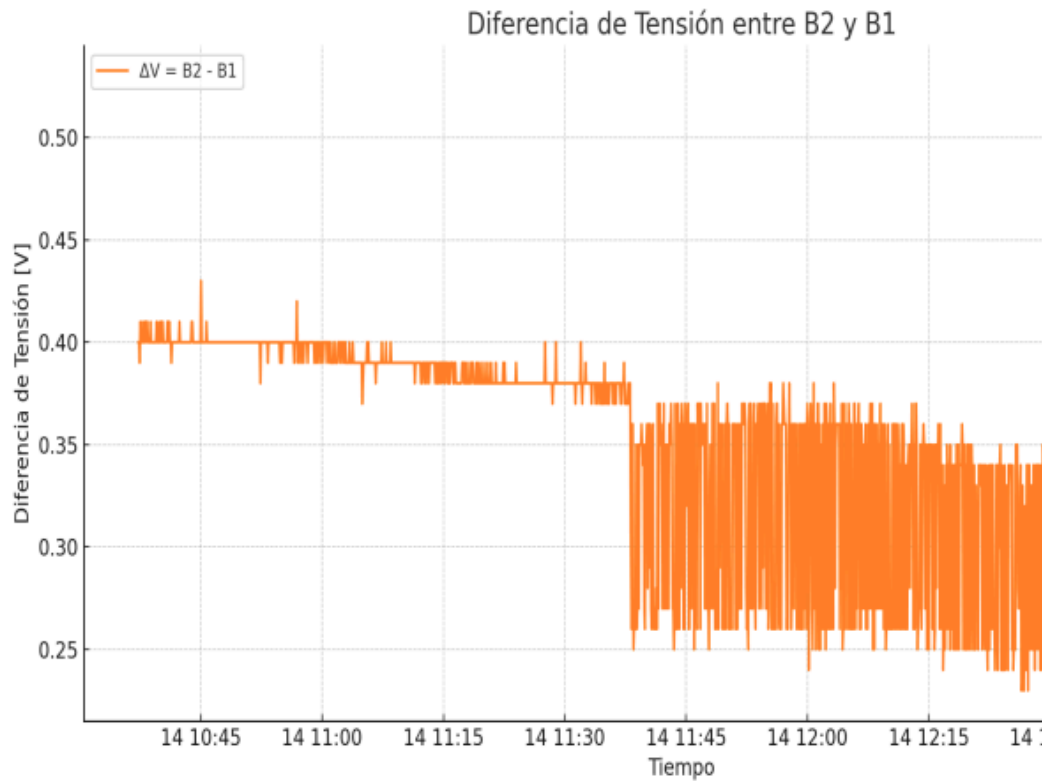
Δ mAh, Tensiones y Corriente



Δ mAh con Tensiones



Diferencia de Tensión entre B2 y B1



Aclaración:

Para todas las pruebas se usaron baterías 18650 de 2500mAh de cada una de marca Samsung



19. Interfaz de control y envío de datos

El sistema cuenta con un microcontrolador que se comunicará con la computadora a bordo del Cubesat mediante comunicación UART/USART, a continuación, se muestra el listado de los comandos:

Cada comando debe ser enviado como una única cadena de caracteres con un delimitador \n al final

19.1. PPC

- MR: devuelve el registro PPC (Read)

Bloque MPPT X:

- MXN: Activa bloque MPPT X (Write)
- MXF: Apaga bloque MPPT X (Write)
- MXV: Obtener voltaje X (Entrega bytes con el Voltaje en mV) (Read)
- MXC: Obtener corriente X (Entrega dos bytes con la corriente en mAh) (Read)

Bloque MPPT Y:

- MYN: Activa bloque MPPT Y (Write)
- MYF: Apaga bloque MPPT Y (Write)
- MYV: Obtener voltaje Y (Entrega bytes con el Voltaje en mV) (Read)
- MYC: Obtener corriente Y (Entrega dos bytes con la corriente en mAh) (Read)

Bloque MPPT Z:

- MZN: Activa bloque MPPT Z (Write)
- MZF: Apaga bloque MPPT Z (Write)
- MZV: Obtener voltaje Z (Entrega bytes con el Voltaje en mV) (Read)
- MZC: Obtener corriente Z (Entrega dos bytes con la corriente en mAh) (Read)

Estado General del Módulo (Read):

R_PPC							
Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0
-	-	PPCXOK	PPCXEN	PPCYOK	PPCYEN	PPCZOK	PPCZEN

PPCXEN:

- 0: PPC X Deshabilitado
- 1: PPC X Habilitado

PPCXOK:

- 0: Indica que se detecta potencia en PPC X cuando PPCXEN está en cero o cuando no se detecta potencia cuando PPCXEN está en uno.
- 1: Indica que se detecta potencia en PPC X cuando PPCXEN está en 1 o cuando no se detecta potencia cuando PPCXEN está en cero.

PPCYEN:

- 0: PPC Y Deshabilitado
- 1: PPC Y Habilitado

PPCYOK:

- 0: Indica que se detecta potencia en PPC Y cuando PPCYEN está en cero o cuando no se detecta potencia cuando PPCYEN está en uno.
- 1: Indica que se detecta potencia en PPC Y cuando PPCYEN está en 1 o cuando no se detecta potencia cuando PPCYEN está en cero.

PPCZEN:

- 0: PPC Z Deshabilitado
- 1: PPC Z Habilitado

PPCZOK:

- 0: Indica que se detecta potencia en PPC Z cuando PPCZEN está en cero o cuando no se detecta potencia cuando PPCZEN está en uno.
- 1: Indica que se detecta potencia en PPC Z cuando PPCZEN está en 1 o cuando no se detecta potencia cuando PPCZEN está en cero.

19.2. BMS

- BS: Devuelve el registro BMS1, BMS2 y BMS3. (Read) (Se mandan en serie los 3 registros)
- B1N: Activa balance de Celda 1 (Write)
- B1F: Desactiva balance de Celda 1 (Write)
- B1V: Obtener Voltaje Celda 1 (Entrega dos bytes con el Voltaje en mV) (Read)
- B1Q: Obtener Carga Celda 1 (Entrega dos bytes con la carga en mAh) (Read)
- B2N: Activa balance de Celda 2 (Write)
- B2F: Desactiva balance de Celda 2 (Write)
- B2V: Obtener Voltaje Celda 2 (Entrega bytes con el Voltaje en mV) (Read)
- B2Q: Obtener Carga Celda 2 (Entrega dos bytes con la carga en mAh) (Read)
- BC: Corriente actual de las baterías. (Entrega dos bytes con la corriente en mA) (Read)
- BT: Temperatura actual de las baterías. (Entrega un byte con la temperatura en °C comenzando en -30°C) (Read)
- BVn: Umbral superior de tensión (Ver apéndice 5) (Write)

Estado General del Módulo 1 (Read):

R_BMS1							
Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0
CELL2UV	CELL1UV	CELL2OV	CELL1OV	DCSH	DCOV	CHSH	CHOV

CHOV:

- 0: No se detecta sobrecorriente en carga.

- 1: Detección de sobrecorriente en carga.

CHSH:

- 0: No se detecta cortocircuito en carga.
- 1: Detección de cortocircuito en carga.

DCOV:

- 0: No se detecta sobrecorriente en descarga.
- 1: Detección de sobrecorriente en descarga.

DCSH:

- 0: No se detecta cortocircuito en descarga.
- 1: Detección de cortocircuito en descarga.

CELL1OV:

- 0: No se detecta sobretensión en celda 1.
- 1: Detección de sobretensión en celda 1.

CELL2OV:

- 0: No se detecta sobretensión en celda 2.
- 1: Detección de sobretensión en celda 2.

CELL1UV:

- 0: No se detecta baja tensión en celda 1.
- 1: Detección de baja tensión en celda 1.

CELL2UV:

- 0: No se detecta baja tensión en celda 2.
- 1: Detección de baja tensión en celda 2.

Estado General del Módulo 2 (Read):

R_BMS2							
Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0
BUT	BOT	RBAL2	RBAL1	BAL2	BAL1	DSG	CHG

CHG:

- 0: Carga de baterías deshabilitada.
- 1: Carga de baterías habilitado.

DSG:

- 0: Descarga de baterías deshabilitado.

- 1: Descarga de baterías habilitado.

BAL1:

- 0: Indica que no se está balanceando la celda 1.
- 1: Indica que se está balanceando la celda 1.

BAL2:

- 0: Indica que no se está balanceando la celda 2.
- 1: Indica que se está balanceando la celda 2.

RBAL1:

- 0: Indica que la celda 1 se encuentra balanceada.
- 1: Indica que se requiere balanceo de la celda 1.

RBAL2:

- 0: Indica que la celda 1 se encuentra balanceada.
- 1: Indica que se requiere balanceo de la celda 1.

BOT:

- 0: No se detecta alta temperatura en baterías.
- 1: Detección de temperatura muy alta en baterías

BUT:

- 0: No se detecta baja temperatura en baterías.
- 1: Detección de temperatura muy baja en baterías

Estado General del Módulo 3 (Read):

R BMS3							
Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0
-	-	-	-	-	-	-	I2COK

I2COK:

- 0: Falla en la comunicación I2C.
- 1: Comunicación I2C OK.

19.3. PDU

Obtención de Registros:

- PR: Devuelve el registro RPDU (Read)

Salida 3.3V:

- P3N: Encendido (Write)
- P3F: Apagado (Write)
- P3S: Alternar con Backup (Write)
- P3V: Obtener Voltaje (Entrega dos bytes con el Voltaje en mV) (Read)
- P3C: Obtener Corriente (Entrega bytes con la corriente en mA) (Read)

Salida 5V:

- P5N: Encendido (Write)
- P5F: Apagado (Write)
- P5S: Alternar con Backup (Write)
- P5V: Obtener Voltaje (Entrega bytes con el Voltaje en mV) (Read)
- P5C: Obtener Corriente (Entrega bytes con la corriente en mA) (Read)

Salida Battery:

- PBN: Encendido (Write)
- PBF: Apagado (Write)
- PBV: Obtener Voltaje (Entrega bytes con el Voltaje en mV) (Read)
- PBC: Obtener Corriente (Entrega bytes con la corriente en mA) (Read)

Estados de las salidas reguladoras del Módulo PDU:

Regulador 3.3V con Redundancia (Read):

R_PDU1							
Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0
PDU2OC	PDU2ON	PDU2VOK	PDU2EN	PDU1OC	PDU1ON	PDU1VOK	PDU1EN

PDU1EN:

- 0: Regulador 1 Deshabilitado
- 1: Regulador 1 Habilitado

PDU1VOK:

- 0: Indica que la tensión a la salida del regulador 1 no está comprendida en el rango entre 3.2 y 3.4 cuando PDU1EN está en uno
- 1: Indica que la tensión a la salida del regulador 1 está comprendida en el rango entre 3.2 y 3.4 cuando PDU1EN está en uno

PDU1ON:

- 0: Indica que la salida del regulador 1 no está siendo usada debido a que no se detecta una corriente distinta de cero.
- 1: Indica que la salida del regulador 1 está siendo usada debido a que se detecta una corriente distinta de cero.

PDU1OC:

- 0: Indica que no hay sobrecorriente en la salida del regulador 1.
- 1: Indica que sobrecorriente en la salida del regulador 1.

PDU2EN:

- 0: Regulador 2 Deshabilitado
- 1: Regulador 2 Habilitado

PDU2VOK:

- 0: Indica que la tensión a la salida del regulador 2 no está comprendida en el rango entre 3.2 y 3.4 cuando PDU2EN está en uno
- 1: Indica que la tensión a la salida del regulador 2 está comprendida en el rango entre 3.2 y 3.4 cuando PDU2EN está en uno

PDU2ON:

- 0: Indica que la salida del regulador 2 no está siendo usada debido a que no se detecta una corriente distinta de cero.
- 1: Indica que la salida del regulador 2 está siendo usada debido a que se detecta una corriente distinta de cero.

PDU2OC:

- 0: Indica que no hay sobrecorriente en la salida del regulador 2.
- 1: Indica que sobrecorriente en la salida del regulador 2.

Regulador 5V con Redundancia (Read):

R_PDU2							
Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0
PDU4OC	PDU4ON	PDU4VOK	PDU4EN	PDU3OC	PDU3ON	PDU3VOK	PDU3EN

PDU3EN:

- 0: Regulador 3 Deshabilitado
- 1: Regulador 3 Habilitado

PDU3VOK:

- 0: Indica que la tensión a la salida del regulador 3 no está comprendida en el rango entre 4.9 y 5.1 cuando PDU3EN está en uno
- 1: Indica que la tensión a la salida del regulador 3 está comprendida en el rango entre 4.9 y 5.1 cuando PDU3EN está en uno

PDU3ON:

- 0: Indica que la salida del regulador 3 no está siendo usada debido a que no se detecta una corriente distinta de cero.
- 1: Indica que la salida del regulador 3 está siendo usada debido a que se detecta una corriente distinta de cero.

PDU3OC:

- 0: Indica que no hay sobrecorriente en la salida del regulador 3.
- 1: Indica que sobrecorriente en la salida del regulador 3.

PDU4EN:

- 0: Regulador 3 Deshabilitado
- 1: Regulador Habilitado

PDU4VOK:

- 0: Indica que la tensión a la salida del regulador 4 no está comprendida en el rango entre 4.9 y 5.1 cuando PDU4EN está en uno
- 1: Indica que la tensión a la salida del regulador 4 está comprendida en el rango entre 4.9 y 5.1 cuando PDU4EN está en uno

PDU4ON:

- 0: Indica que la salida del regulador 4 no está siendo usada debido a que no se detecta una corriente distinta de cero.
- 1: Indica que la salida del regulador 4 está siendo usada debido a que se detecta una corriente distinta de cero.

PDU4OC:

- 0: Indica que no hay sobrecorriente en la salida del regulador 4.
- 1: Indica que sobrecorriente en la salida del regulador 4.

Salida Batería (Read):

R_PDU5							
Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0
-	-	-	-	PDU5OC	PDU5ON	PDU5VOK	PDU5EN

PDU5EN:

- 0: Regulador 5 Deshabilitado
- 1: Regulador 5 Habilitado

PDU5VOK:

- 0: Indica que la tensión a la salida del regulador 5 no está comprendida en el rango entre 6.6 y 8.2 cuando PDU5EN está en uno
- 1: Indica que la tensión a la salida del regulador 5 está comprendida en el rango entre 6.6 y 8.2 cuando PDU5EN está en uno

PDU5ON:

- 0: Indica que la salida del regulador 5 no está siendo usada debido a que no se detecta una corriente distinta de cero.
- 1: Indica que la salida del regulador 5 está siendo usada debido a que se detecta una corriente distinta de cero.

PDU5OC:

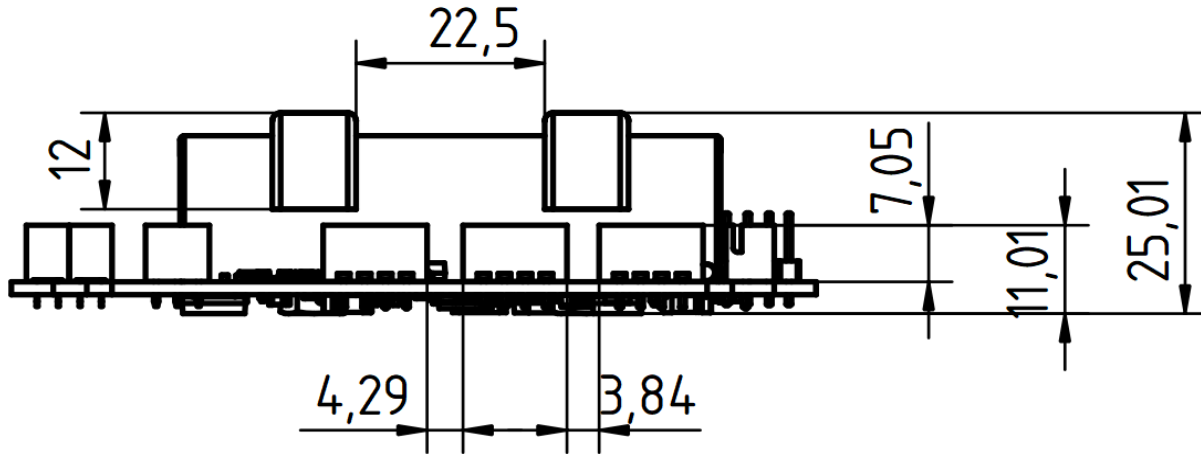
- 0: Indica que no hay sobrecorriente en la salida del regulador 5.
- 1: Indica que sobrecorriente en la salida del regulador 5.

19.4. Aclaraciones:

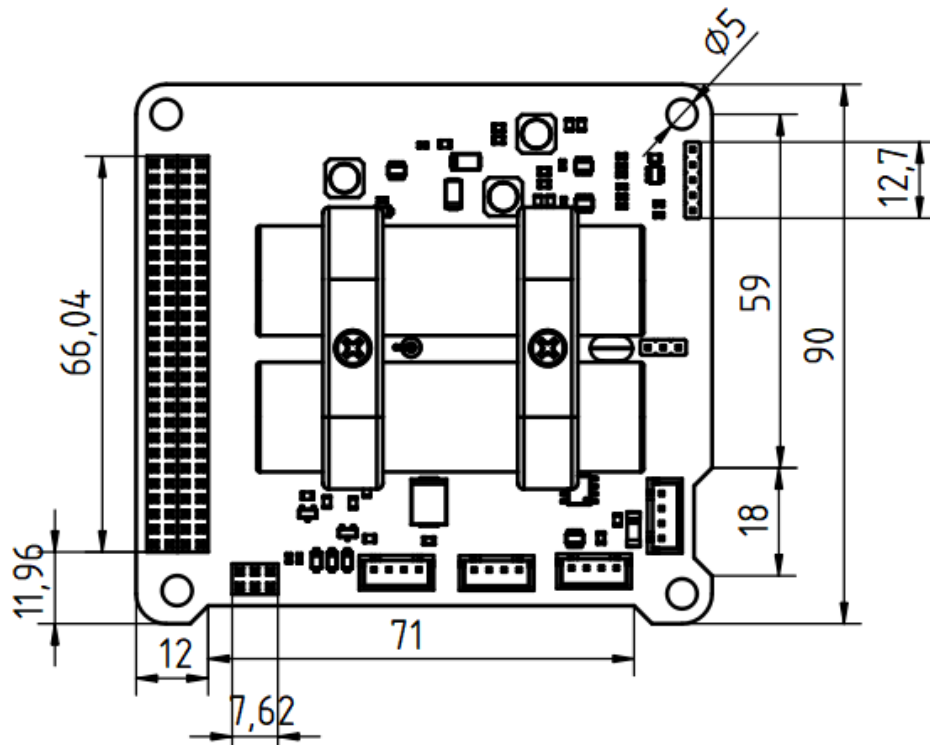
- El sistema puede apagar y encender las distintas partes del sistema si es que así lo requiere de manera automática, es decir que no se requiere de un módulo externo que decida cuándo abrir y cerrar entradas/salidas.
- Sin embargo, el sistema puede ser comandado por la computadora a bordo lo cual es altamente recomendable ya que el monitoreo exhaustivo de todos los módulos son informados a dicha computadora y por lo tanto es la más indicada para tomar una decisión crítica.
- Los registros pueden ser pedidos por la computadora a bordo escribiendo el nombre del registro (Por ejemplo, si se quiere el estado del registro R_PDU, se debe mandar por el puerto serial PR)
- Si bien el sistema puede ser comandado por la computadora de a bordo del Cubesat, esta puede descartar el comando si se dan ciertas condiciones que puedan poner en riesgo el suministro de energía, por ejemplo si se pide habilitar la carga cuando las baterías están en el límite superior configurado, el sistema detectará over voltage y la volverá a deshabilitar inmediatamente (sin embargo, esto no debería pasar ya que la computadora a bordo dispondrá de todos los datos necesarios para conocer el estado de los registros de todos los módulos correspondientes a la EPS y por lo tanto debería saber que la tensión es alta).
- Los estados de los registros y por ende el estado de los módulos del sistema se mandan bajo pedido, el sistema no manda información por sí mismo a la computadora a bordo.
- Los registros de cada módulo, si bien están divididos en distintos registros de 8 bits, el sistema manda por el puerto uart todos los registros del módulo correspondiente de manera consecutiva, solo hace distinción entre los registros de distintos módulos (PPC, BMS y PDU).

20. Dimensiones Físicas

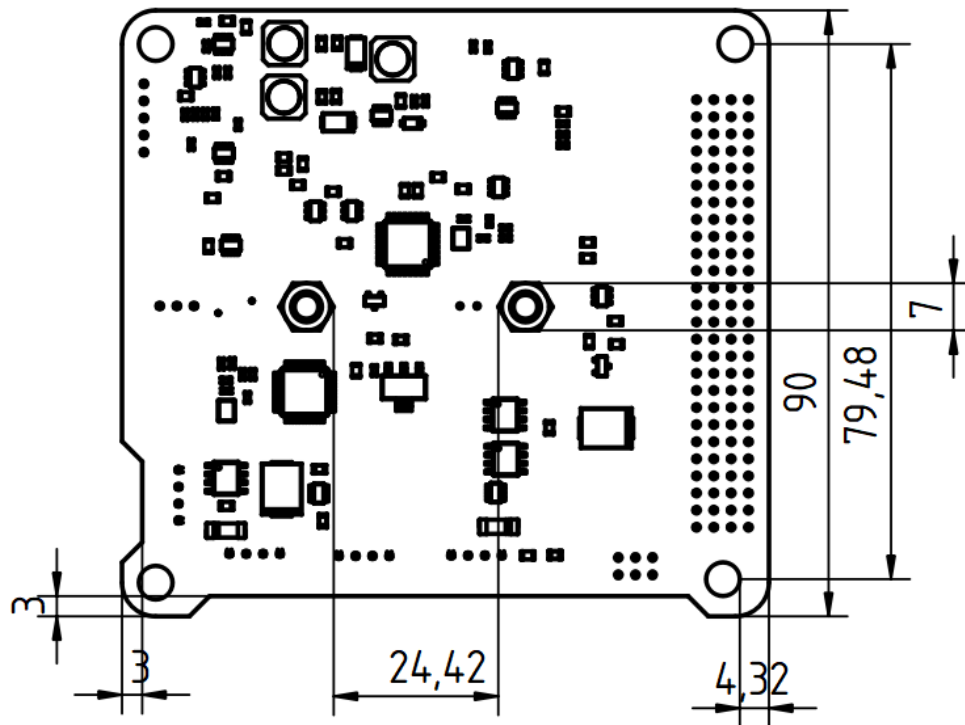
Vista de costado:



Vista de arriba:



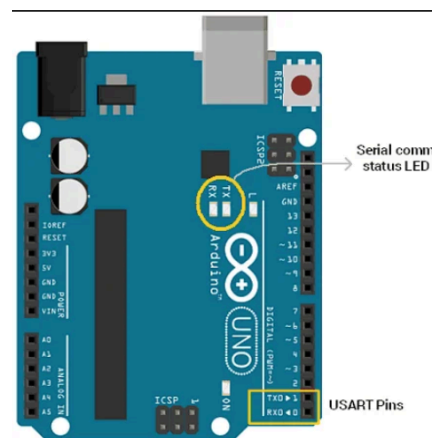
Vista de abajo:



21. Apéndice

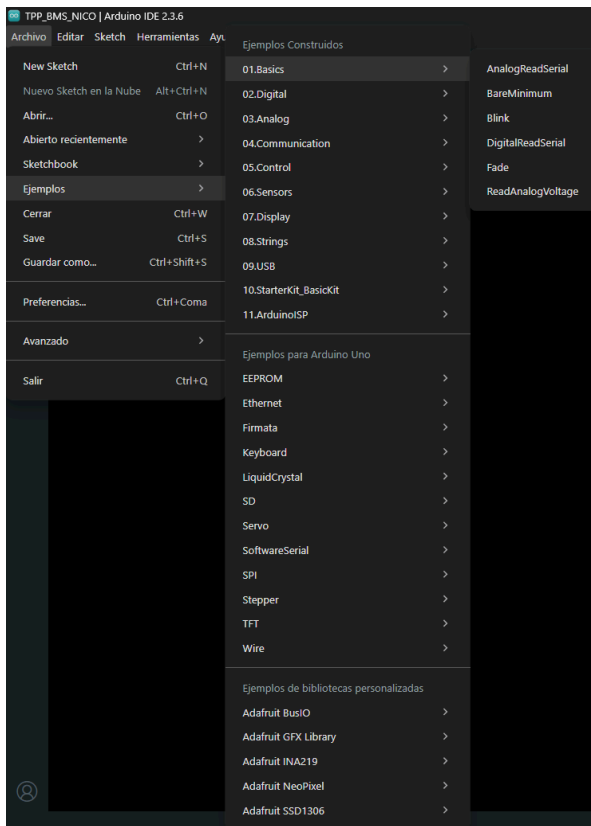
APÉNDICE 1

A continuación se describe cómo usar la comunicación serial (UART), en particular, se muestra la aplicación usando un Arduino, a continuación se muestra los pines correspondientes a los cuales se deberían conectar el sistema EPS

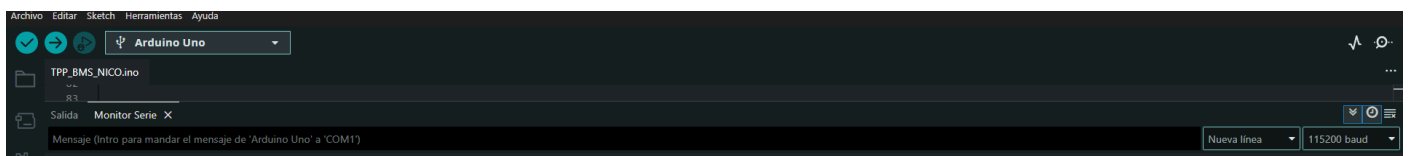


Luego, se debe ingresar a la aplicación de ArduinoIDE, desde la cual se puede abrir cualquier archivo de ejemplo de lectura de Serial, para recibir la información y cargarle (La programación de esto debería depender de la computadora de a bordo que es la encargada de comunicarse con el módulo EPS), sin

embargo, si se quiere hacer pruebas y obtener lecturas provenientes del módulo EPS, esta es una opcion valida

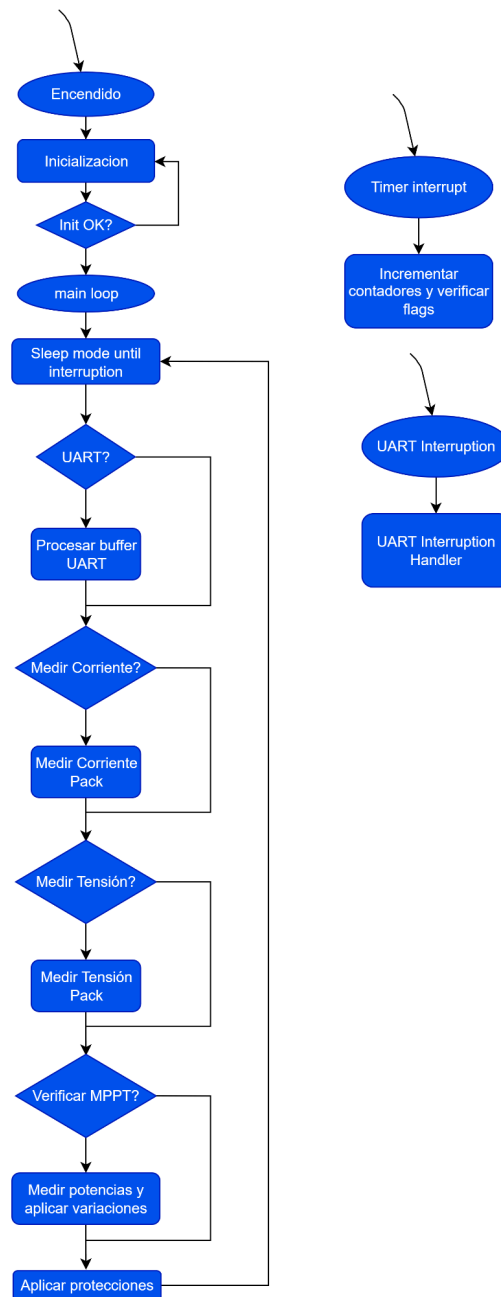


Luego se debe tener en cuenta que los baudios que se deben seleccionar para la comunicación serial es de 115200 en el monitor serial, desde el cual se puede leer los mensajes y escribir los comandos hacia el sistema:



APÉNDICE 2

Diagrama de Flujo del Funcionamiento del BMS:



APÉNDICE 4

CONFIGURACIONES:

Los valores mostrados en la tabla de características generales muestra los valores por defecto, sin embargo, mediante software se pueden modificar diversos umbrales, para que de esta manera el sistema no esté limitado a un solo tipo de batería, las variables con sus respectivos valores se muestran a continuación:

Módulo	Comando	Descripción	Umbral de tensión [V]	Valor de n
--------	---------	-------------	-----------------------	------------

BMS	BVn	Overvoltage Celda 1 y Celda 2	3.9	0x00
			3.95	0x01
			4	0x02
			4.05	0x03
			4.1	0x04
			4.15	0x05
			4.2	0x06
			4.25	0x07
			4.3	0x08
			4.35	0x09
BMS	BVn	Undervoltage Celda 1 y Celda 2	2.95	0x0A
			3	0x0B
			3.05	0x0C
			3.1	0x0D
			3.15	0x0E
			3.2	0x0F
			3.25	0x10
			3.3	0x11
			3.35	0x12
			3.4	0x13

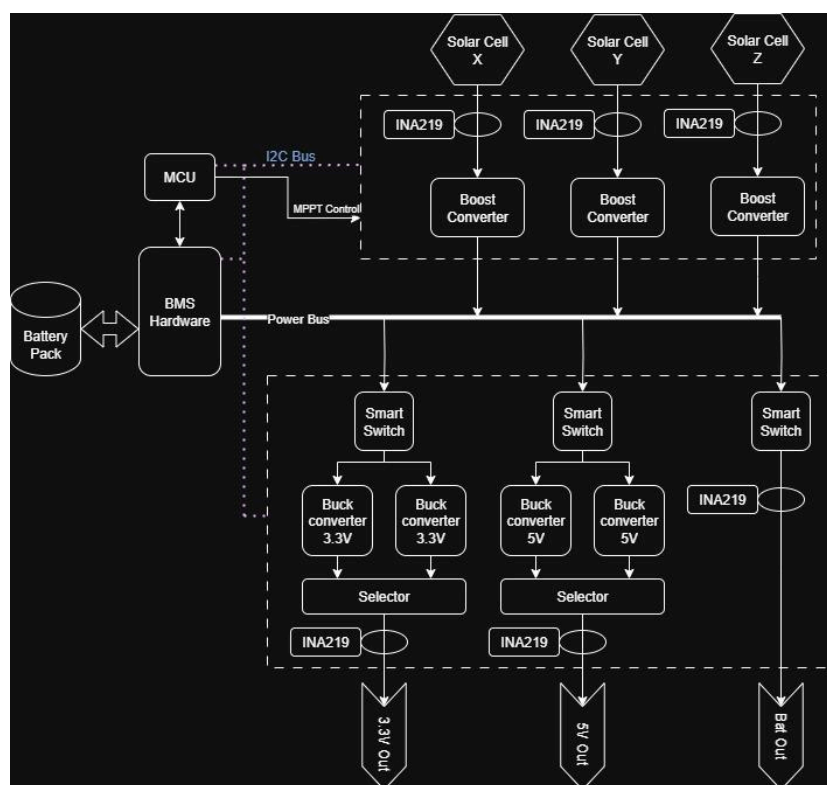
Consideraciones Importantes:

Tener en cuenta que si bien los valores de overvoltage y undervoltage son configurables, estos no deben ser modificados a la ligera ya que por ejemplo, las pruebas del sistema se hicieron con baterías marca samsung 18650, las cuales tienen un rango útil que está comprendido entre 2.5V y 4.2V, sin embargo, estos son valores límites que si se alcanzan se puede generar un daño permanente en las baterías, es por ello, que los valores por defecto de undervoltage es de 3.3V y el de overvoltage es de 4V.

APÉNDICE 5

Información Técnica del Sistema:

En este apéndice se explica más en detalle los distintos módulos del sistema EPS, a continuación se muestra el diagrama de bloques del sistema completo:



PPC:

Como se puede ver del diagrama anterior, la configuración de los elevadores de tensión son del tipo boost, es decir elevan tensión, en particular los paneles solares van a tener un rango aproximado de tensión entre 4.4V y 5.5V, por lo cual para cargar las dos baterías en serie se necesita esta configuración para poder elevar la tensión por encima de los 6V hasta los 8.2V.

Cada convertidor de potencia cuenta con sensores de la familia INA que comunican los datos de tensión y corriente mediante comunicación I2C

PDU:

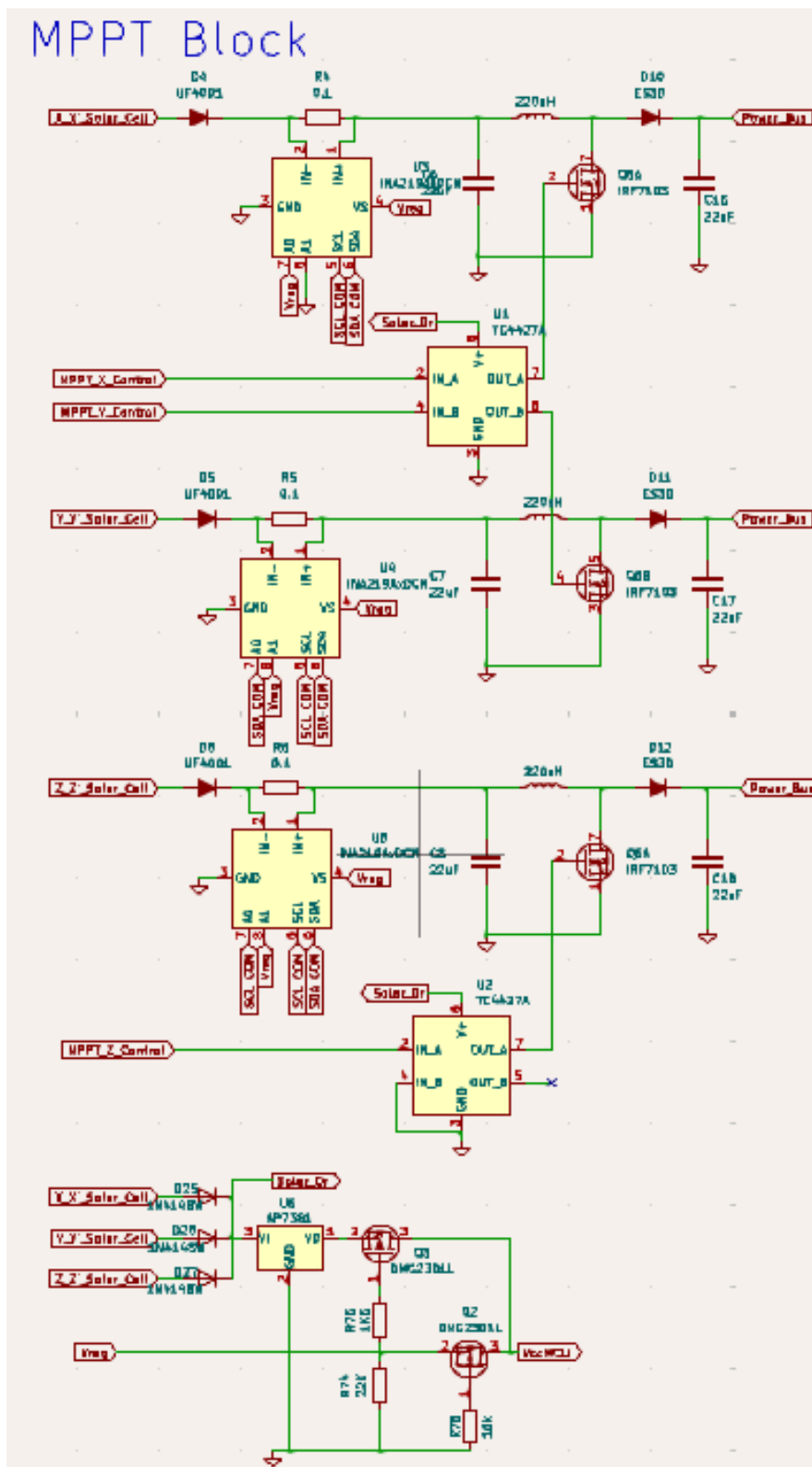
Como se puede ver en el diagrama anterior, las salidas cuentan con reguladores del tipo buck, es decir que disminuyen tensión desde la batería hacia las salidas (de aproximadamente 7.4V a 3.3V y 5V).

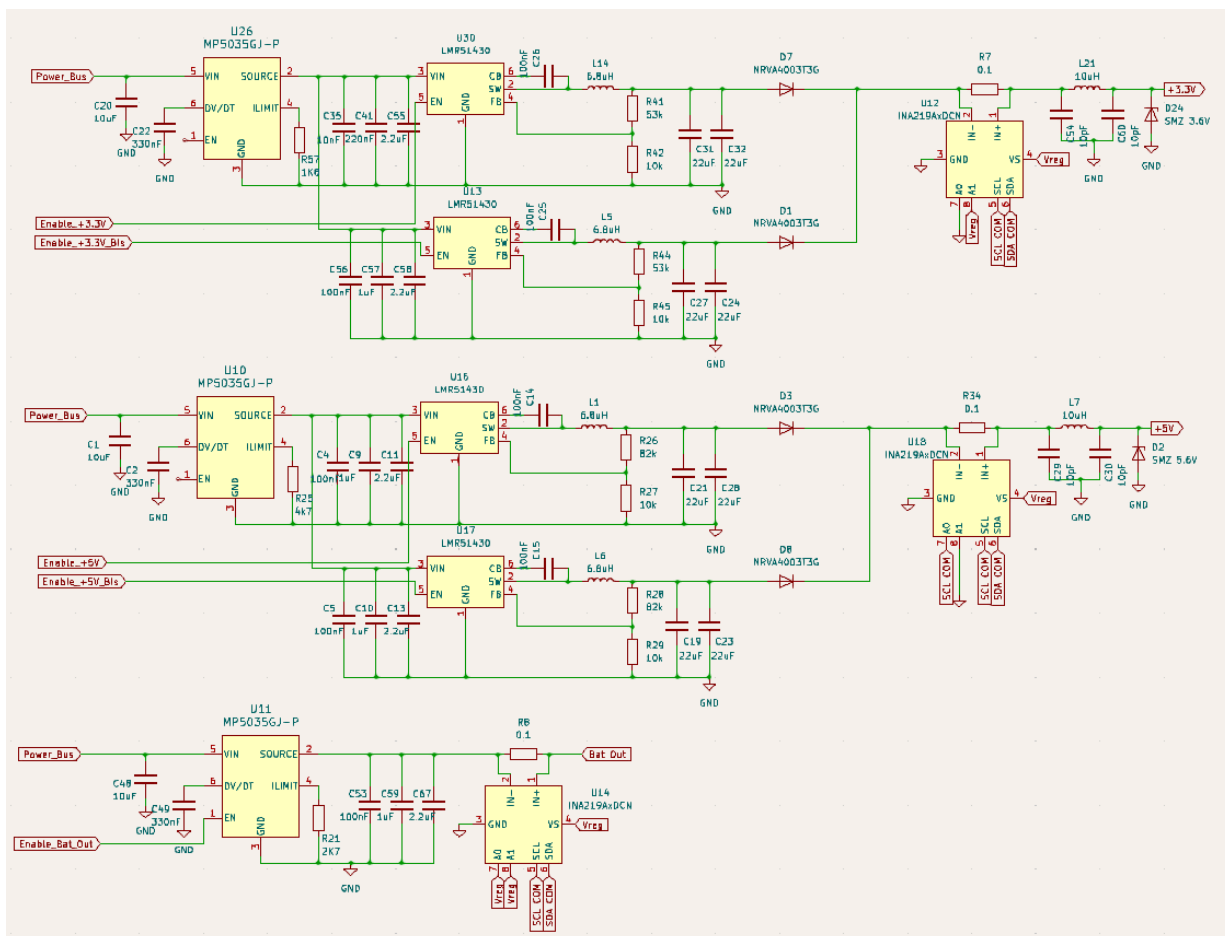
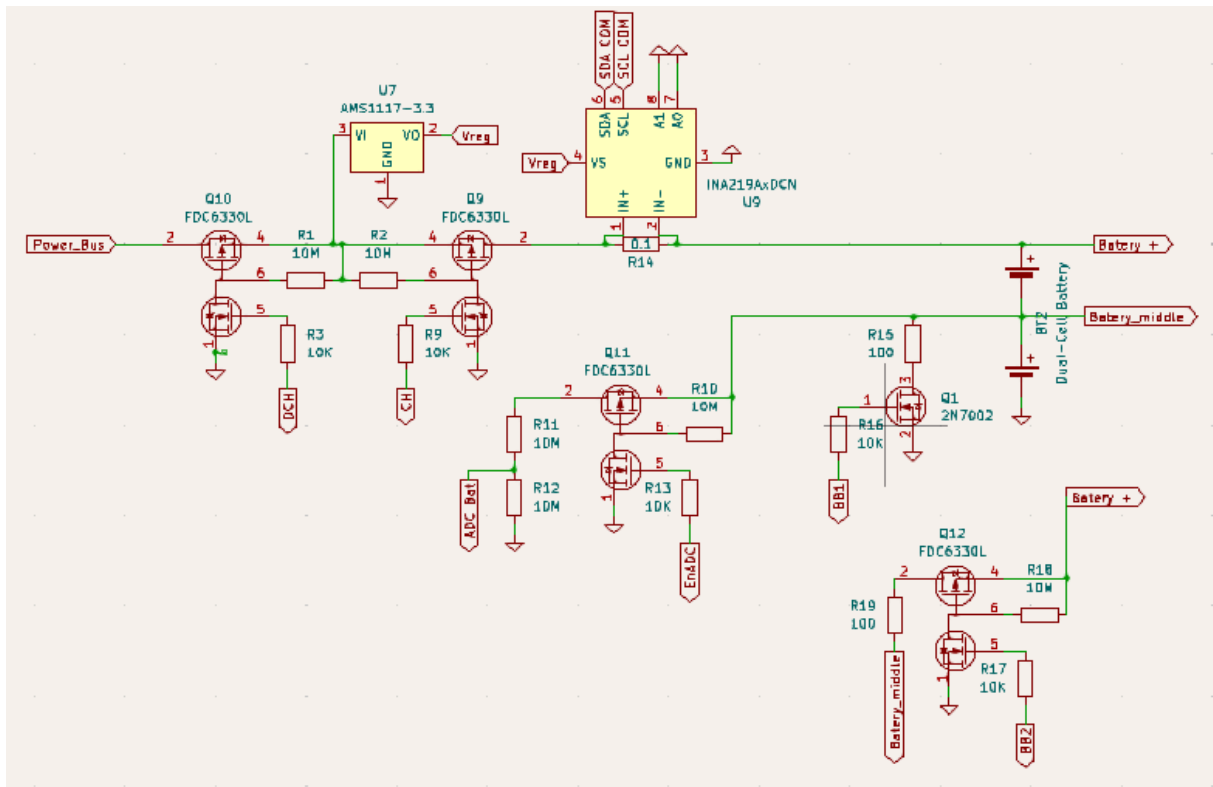
Cada PDU cuenta con sensores de la familia INA que comunican los datos de tensión y corriente mediante comunicación I2C.

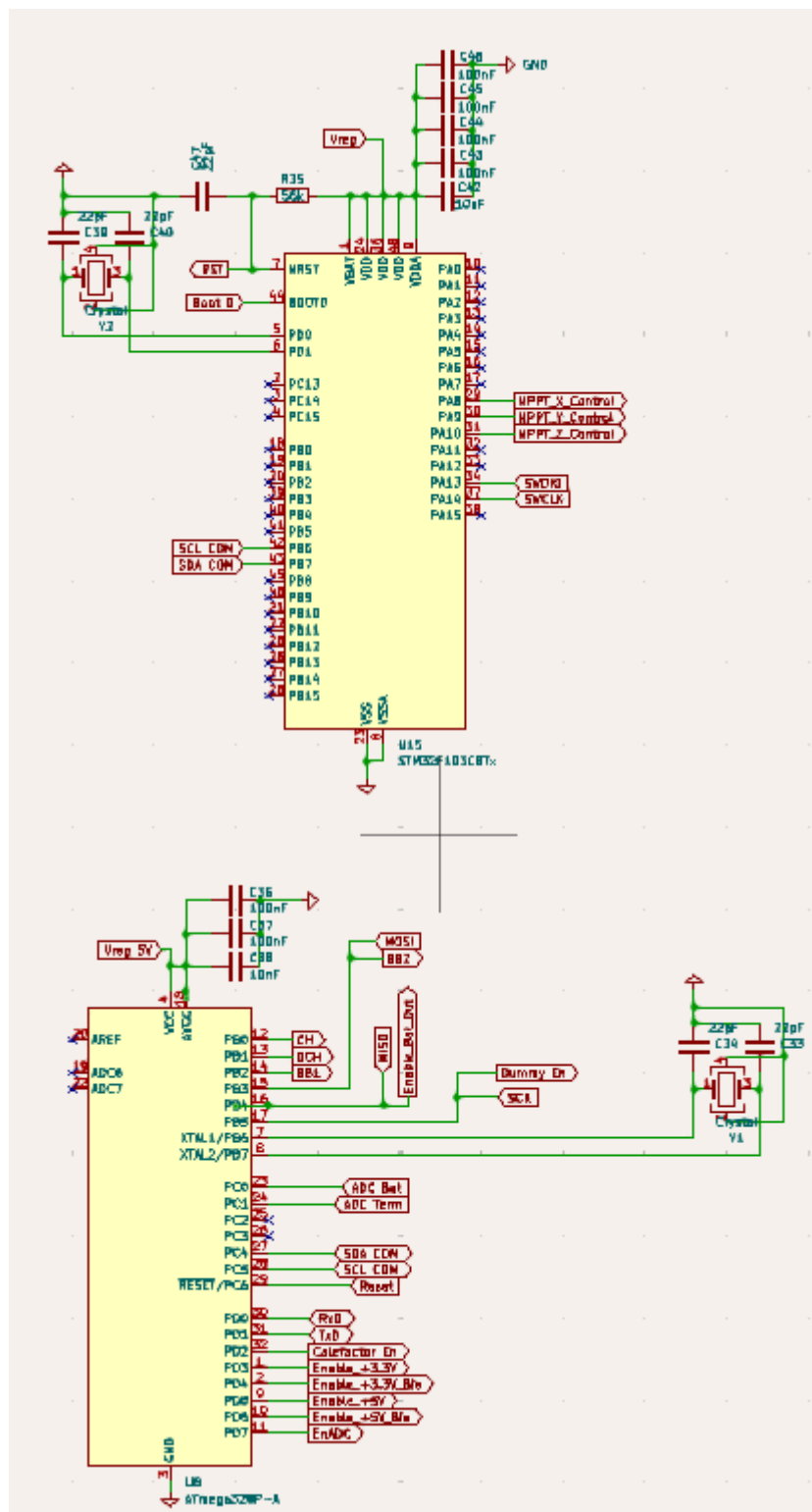
Cada PDU cuenta con un filtro PI que sirve para filtrar las interferencias electromagnéticas provenientes de la computadora de abordo (si es que las hubiese).

Aclaración: como se puede ver en el diagrama, todas las comunicaciones internas del sistema EPS son mediante I2C, sin embargo la comunicación del usuario (computadora de abordo) es UART Serial.

Esquemático del circuito:







22. Fallos Encontrados y Soluciones Implementadas:

- **Fallo Funcional del Primer Prototipo Monolítico:**
 - **Problema:** El diseño inicial, concebido como una solución "todo en uno" y fabricado profesionalmente, falló en las pruebas iniciales. Esto se debió a la complejidad inherente de un sistema monolítico, donde la interdependencia de los componentes dificulta la depuración y la identificación de la causa raíz de los fallos.
 - **Solución:** Se adoptó una **metodología de desarrollo modular e incremental**. El sistema EPS se descompuso en sus bloques funcionales principales (PPC, BMS, PDU), y cada módulo se implementó y probó de forma aislada utilizando prototipos de bajo costo. Esta estrategia permitió una depuración más controlada y económica.
- **Ineficiencia en la Adquisición de Datos (Mapeo Directo por ADC):**
 - **Problema:** El enfoque preliminar utilizaba divisores de tensión y amplificadores operacionales conectados directamente a las entradas ADC del microcontrolador. Esto resultaba en un alto consumo de pines, complejidad del circuito debido a la circuitería de acondicionamiento de señal para cada medición, y susceptibilidad al ruido en las largas pistas analógicas.
 - **Solución:** Se migró a una **arquitectura de monitoreo digital y distribuida basada en el bus I²C**, utilizando sensores de corriente y potencia INA219. Estos dispositivos integran un amplificador de precisión y un ADC de 12 bits, transmitiendo mediciones de forma digital. Esta solución redujo drásticamente el uso de pines del microcontrolador, simplificó la interconexión, aumentó la precisión y la robustez frente al ruido.

23. Aprendizaje

- **Importancia de la Modularidad:** La descomposición del sistema en módulos funcionales es crucial para proyectos complejos con recursos limitados, facilitando la depuración, la iteración y la optimización de cada subsistema de forma independiente.
- **Ventajas del Control Digital:** El uso de protocolos como I²C para la adquisición de datos reduce significativamente el consumo de pines del microcontrolador y la complejidad del cableado, mejorando la escalabilidad y la eficiencia en espacio y peso.
- **Gestión de Restricciones Presupuestarias y de Importación:** La necesidad de reducir costos, junto con los problemas de importación y la dificultad para obtener los integrados necesarios para desarrollar circuitos en el mercado local, impulsó la innovación en la metodología de desarrollo, demostrando que las limitaciones pueden ser catalizadores para soluciones más eficientes y robustas.

24. Resultados del Desarrollo a Nivel técnico:

El sistema EPS final es un prototipo operativo con las siguientes características técnicas destacadas:

- Un **diseño modular** que integra un PPC optimizado con control digital directo, un BMS a medida y una PDU eficiente.
- Implementación de un bus I²C con sensores INA219 para un **monitoreo preciso y escalable** de tensión, corriente y temperatura en puntos críticos del sistema.
- **Gestión Inteligente de Baterías (BMS):** Protección integral contra sobre y baja tensión, sobrecorriente, monitoreo de temperatura, estado de carga y capacidad de balanceo de celdas.
- **Cargador de Baterías con MPPT:** Capacidad para extraer la máxima potencia de los paneles solares mediante el algoritmo MPPT, optimizando la eficiencia de carga.
- **Regulación y Distribución de Energía:** Tres canales de entrada fotovoltaica independientes y tres buses de alimentación regulados (3.3V, 5V) con redundancia en las salidas de 3.3V y 5V, y limitadores de corriente.
- **Interfaz de Comunicación Versátil:** Comunicación UART/USART con la computadora a bordo del CubeSat, permitiendo el comando y la lectura de registros de estado de todos los módulos.

25. Código:

```
#include <Wire.h>
#include <Adafruit_INA219.h>
#include <avr/interrupt.h>
#include <avr/io.h>
#include <avr/sleep.h>
#include <EEPROM.h>
```

```
void ina219Sleep(Adafruit_INA219 &sensor, uint8_t addr);
void ina219Wake(Adafruit_INA219 &sensor, uint8_t addr);
//Variables del sistema
float corriente    = 0.0;
float corriente33  = 0.0;
float corriente5   = 0.0;
```

```
#include <Wire.h>
#include <Adafruit_INA219.h>
```



```
#include <avr/interrupt.h>
#include <avr/io.h>
#include <avr/sleep.h>
#include <EEPROM.h>

void ina219Sleep(Adafruit_INA219 &sensor, uint8_t addr);
void ina219Wake(Adafruit_INA219 &sensor, uint8_t addr);
//Variables del sistema
float corriente = 0.0;
float corriente33 = 0.0;
float corriente5 = 0.0;
float corrienteBat = 0.0;
float tensionINA = 0.0;
float tension33 = 0.0;
float tension5 = 0.0;
float tensionBat = 0.0;
float tensionADC = 0.0;
float tensionB1 = 0.0;
float tensionB2 = 0.0;
float tensionTermistor = 0.0;
float OvervoltageLimit = 0.0;
float UndervoltageLimit = 0.0;

// ===== Estructuras =====
struct BatteryCharge {
    double coulombCount1; // En AVR (Uno), double = float (4 bytes)
    double coulombCount2;
};

struct PDUStatus {
    bool output33Selected;
    bool output33BSelected;
    bool output5Selected;
    bool output5BSelected;
    bool BatOutOn;
};

struct PDUStatus_REG {
    bool PDU1EN;
    bool PDU1OK;
    bool PDU1ON;
```

```
bool PDU10C;  
bool PDU2EN;  
bool PDU20K;  
bool PDU20N;  
bool PDU20C;  
bool PDU3EN;  
bool PDU30K;  
bool PDU30N;  
bool PDU30C;  
bool PDU4EN;  
bool PDU40K;  
bool PDU40N;  
bool PDU40C;  
bool PDU5EN;  
bool PDU50K;  
bool PDU50N;  
bool PDU50C;  
};  
  
struct PPCStatus_REG {  
    bool PPCXEN;  
    bool PPCXOK;  
    bool PPCYEN;  
    bool PPCYOK;  
    bool PPCZEN;  
    bool PPCZOK;  
};  
  
struct BMSStatus {  
    //Byte 1  
    bool dischargeOC;  
    bool chargeOC;  
    bool dischargeSC;  
    bool chargeSC;  
    bool cell1UV;  
    bool cell2UV;  
    bool cell10V;  
    bool cell20V;  
    //Byte 2  
    bool Chg;  
    bool Dch;
```

```
bool Bal1;
bool Bal2;
bool RBal1;
bool RBal2;
bool OvT;
bool UnT;
//Byte 3
bool BatF;
bool BatL;
bool I2CStatus;
};

void printBMSStatus(BMSStatus status){
    Serial.print("Flags: \n Byte1 - Ch0v:"); Serial.print(status.chargeOC);
    Serial.print(" ChSh:"); Serial.print(status.chargeSC);
    Serial.print(" Dc0v:"); Serial.print(status.dischargeOC);
    Serial.print(" DcSh:"); Serial.print(status.dischargeSC);
    Serial.print(" Cell10V:"); Serial.print(status.cell10V);
    Serial.print(" Cell20V:"); Serial.print(status.cell20V);
    Serial.print(" Cell1UV:"); Serial.print(status.cell1UV);
    Serial.print(" Cell2UV:"); Serial.println(status.cell2UV);

    Serial.print("Flags: \n Byte2 - Chg:"); Serial.print(status.Chg);
    Serial.print(" Dch:"); Serial.print(status.Dch);
    Serial.print(" Bal1:"); Serial.print(status.Bal1);
    Serial.print(" Bal2:"); Serial.print(status.Bal2);
    Serial.print(" RBal1:"); Serial.print(status.RBal1);
    Serial.print(" RBal2:"); Serial.print(status.RBal2);
    Serial.print(" OvT:"); Serial.print(status.OvT);
    Serial.print(" UnT:"); Serial.println(status.UnT);

    Serial.print(" I2CStatus:"); Serial.println(status.I2CStatus);
}

class MPPT{
private:

    static constexpr uint8_t MIN_DUTY = 0;
    static constexpr uint8_t MAX_DUTY = 245;
    static constexpr uint8_t STEP      = 1;
    static constexpr float VO_MAX      = 12;
```

```
static constexpr float VD          = 0.8;

bool mpptEn;
bool mpptConstVoltage;
uint8_t INA219addr;
Adafruit_INA219 sensor;
uint8_t duty;           // último duty aplicado
bool lastVariation;    // +1 o -1 (dirección de la última variación)
float lastPower_mW;    // última potencia medida (mW)
float outVoltage;
void applyDuty() {
    if (duty < MIN_DUTY) duty = MIN_DUTY;
    if (duty > MAX_DUTY) duty = MAX_DUTY;
}

public:
    // Constructor: recibe dirección I2C y estado inicial
    MPPT(uint8_t inaAddr, uint8_t duty_init=128)
        : sensor(inaAddr), INA219addr(inaAddr) , duty(duty_init), lastVariation(+1),
        lastPower_mW(NAN) {}

    bool begin() {
        duty = 25;
        applyDuty();
        lastVariation = false;
        if (!sensor.begin()){
            Serial.println("Error al iniciar ina219_MPPTX");
            return false;
        }

        Serial.println("ina219_MPPTX OK");

        sensor.setCalibration_32V_2A();           // ajustá si usás otro rango
        // Semilla de potencia
        float mpptCurrent = getCurrent();
        float mpptVoltage = getBusVoltage();
        ina219Sleep(sensor, INA219addr);
        float pmw = mpptCurrent * mpptVoltage;
        if (isfinite(pmw)) lastPower_mW = pmw;
        else                lastPower_mW = 0.0f;
    }
```

```
    return true;
}

// ----- MÉTODO ÚNICO: P&O en una llamada -----
// Lee potencia y ajusta el duty. Devuelve la potencia actual (mW).
float applyMppt() {

    ina219Wake(sensor, INA219addr);
    float mpptCurrent = getCurrent();
    float mpptVoltage = getBusVoltage();
    ina219Sleep(sensor, INA219addr);

    float PmW = mpptCurrent * mpptVoltage;
    if (!isfinite(PmW)) return lastPower_mW;    // si lectura inválida, no
cambia
    int dutyMaxByVoltage = 0;

    if(mpptConstVoltage)
        dutyMaxByVoltage = (int)(255.0f * (1.0f - OvervoltageLimit / (VO_MAX +
VD)));
    else
        dutyMaxByVoltage = (int)(255.0f * (1.0f - mpptVoltage / (VO_MAX +
VD)));

    if (PmW > lastPower_mW) {
        if(lastVariation == false){
            duty += STEP;
        }else{duty -= STEP;}
    }
    else{
        if(lastVariation == false){
            duty -= STEP;
            lastVariation = true;
        }else{
            duty += STEP;
            lastVariation = false;
        }
    }
    lastPower_mW = PmW; // Actualizar `prevPower` con el nuevo valor
```

```
if (duty > MAX_DUTY){duty = MAX_DUTY;}
if (duty > dutyMaxByVoltage){duty = dutyMaxByVoltage;}

applyDuty();
lastPower_mW = PmW;
return PmW;
}

// ----- GETTERS de las variables agregadas -----

uint8_t getDuty() const { return duty; }
void setDuty(uint8_t dutyToSet){ duty = dutyToSet; }
int8_t getLastVariation() const{ return lastVariation; } // +1 / -1
float getLastPower_mW() const { return lastPower_mW; }

// Encender / apagar el MPPT
void enable() { mpptEn = true; duty = (MAX_DUTY + MIN_DUTY) / 2; }
void disable() { mpptEn = false; }
bool isEnabled() const { return mpptEn; }

// Obtener dirección del INA219
void setAddress(uint8_t addr){INA219addr = addr;}
uint8_t getAddress() const { return INA219addr; }

// Lecturas con el sensor
void setConstCurrentMode(){mpptConstVoltage=false;}
void setConstVoltageMode(){mpptConstVoltage=true;}
float getBusVoltage() { return sensor.getBusVoltage_V(); }
float getCurrent() { return sensor.getCurrent_mA(); }
float getPower() { return sensor.getPower_mW(); }
bool isOK(){return (isEnabled() && getPower() > 50) || (!isEnabled() &&
getPower() < 50);}

};

class Thermistor{
private:
float Temp;
float Voltage;
bool Charge;
```

```
bool Discharge;
bool HeatUp;

static constexpr float MAIN_VOLTAGE = 4.2;
static constexpr uint8_t CHARGE_TOP_TEMP = 45;
static constexpr uint8_t CHARGE_BOTTOM_TEMP = 0;
static constexpr uint8_t DISCHARGE_TOP_TEMP = 60;
static constexpr uint8_t DISCHARGE_BOTTOM_TEMP = -20;
static constexpr uint8_t STORAGE_TOP_TEMP = 25;
static constexpr uint8_t STORAGE_BOTTOM_TEMP = 15;
static constexpr float RPULL = 33000;
static constexpr float RP = 10000;
static constexpr float R25 = 10000;
static constexpr float BETA = 3880.0;
static constexpr float T0 = 298.15;

float adaptVoltage(float B1Voltage, float TVoltage){ return TVoltage*(
MAIN_VOLTAGE / B1Voltage ); }

public:
bool getCharge(){return Charge;}
bool getDischarge(){return Discharge;}
float getTemp(){return Temp;}
bool measureTemp(float B1Voltage, float TVoltage){
    if (TVoltage <= 0.0f || TVoltage >= MAIN_VOLTAGE) return false;

    float paralelo = RPULL*(TVoltage /(B1Voltage-TVoltage));
    //Serial.print("Paralelo: ");
    //Serial.println(paralelo, 5);
    float ntc = 1.0f/((1.0f)/paralelo -(1.0f/RP));
    //Serial.print("ntc: ");
    //Serial.println(ntc, 5);
    if (ntc <= 0.0f) return false;

    float tempC = 1/(1/T0 + 1/BETA * logf(ntc/R25)) - 273;
    if (isnan(tempC) || isinf(tempC)) return false;

    Temp = tempC;
    return true;
}
```



```
bool applyThermalProtection(){  
  
    if(Temp < CHARGE_BOTTOM_TEMP || Temp > CHARGE_TOP_TEMP)  
        Charge = false;  
    else  
        Charge = true;  
  
    if(Temp < DISCHARGE_BOTTOM_TEMP || Temp > DISCHARGE_TOP_TEMP)  
        Discharge = false;  
    else  
        Discharge = true;  
    if(Temp < CHARGE_BOTTOM_TEMP)  
        HeatUp = true;  
    else if(Temp > (CHARGE_BOTTOM_TEMP + 5))  
        HeatUp = false;  
  
    return true;  
}  
  
bool getHeatUp(){return HeatUp;}  
  
};  
  
// Objetos y constantes  
  
static constexpr uint8_t BMS_I2C_ADDR = 0x40;  
static constexpr uint8_t OUT33_I2C_ADDR = 0x44;  
static constexpr uint8_t OUT5_I2C_ADDR = 0x43;  
static constexpr uint8_t BATOUT_I2C_ADDR = 0x45;  
static constexpr uint8_t MPPTX_I2C_ADDR = 0x41;  
static constexpr uint8_t MPPTY_I2C_ADDR = 0x46;  
static constexpr uint8_t MPPTZ_I2C_ADDR = 0x47;  
  
Adafruit_INA219 ina219BMS(BMS_I2C_ADDR);  
Adafruit_INA219 ina219_33Out(OUT33_I2C_ADDR);  
Adafruit_INA219 ina219_50Out(OUT5_I2C_ADDR);  
Adafruit_INA219 ina219_BatOut(BATOUT_I2C_ADDR);
```

```
Adafruit_INA219 ina219_MPPTX(MPPTX_I2C_ADDR);

void ina219Sleep(Adafruit_INA219 &sensor, uint8_t addr) {
    Wire.beginTransaction(addr);
    Wire.write(0x00); // Registro de configuración
    Wire.write(0x00); // MSB
    Wire.write(0x00); // LSB -> Power-Down
    Wire.endTransmission();
}

// === Despertar el sensor ===
// Incluye el delay necesario (~1 ms con configuración por defecto de
Adafruit)
void ina219Wake(Adafruit_INA219 &sensor, uint8_t addr) {
    sensor.begin(addr);
    delay(1); // esperar al menos un ciclo de conversión (~1 ms en modo
default)
}

// ****Prototipos****//
// ==== Setup / Inicialización ====
void setupProtection();
void setupBalanceo();
void setupPDU();
void setupTimer1();

// ==== EEPROM ====
void saveAllToEEPROM(int base = 0);
void loadAllFromEEPROM(int base = 0);
void saveOvervoltage(int base = 0);
void loadOvervoltage(int base = 0);
void saveUndervoltage(int base = 0);
void loadUndervoltage(int base = 0);
void saveBatteryCharge(int base = 0);
void loadBatteryCharge(int base = 0);
void savePDUStatus(int base = 0);
void loadPDUStatus(int base = 0);

// ==== Protecciones ====
void checkCellVoltageProtection(float cell1_V, float cell2_V);
void checkCellChargeCurrentProtection(float current);
```

```
void checkCellDischargeCurrentProtection(float current);
void applyProteccion();

// ==== UART / Comandos ====
void identificarSerial();

const int VOLTAGE_ENABLE_PIN          = 7;
const int ADC_DIVISOR_PIN             = A0; // Entrada del divisor
bateria
const int ADC_THERMISTOR_PIN          = A1; // Entrada del
divisor termistor

/***** Pines de control *****/
// ==== Habilitacion de carga y descarga
const uint8_t CHARGE_ENABLE_PIN      = 8;
const uint8_t DISCHARGE_ENABLE_PIN   = 9;

// ==== Balanceo por comando
const uint8_t BAL1_PIN               = 10; // Celda 1
const uint8_t BAL2_PIN               = 11; // Celda 2

// ==== Habilitaciones de Bucks

const uint8_t ENABLE_3_OUT           = 3;
const uint8_t ENABLE_3_BIS_OUT       = 4;

const uint8_t ENABLE_5_OUT           = 5;
const uint8_t ENABLE_5_BIS_OUT       = 6;

const uint8_t ENABLE_BAT_OUT         = 12;

// ==== Calefactor
const uint8_t ENABLE_CALEFACTOR       = 2;
const uint8_t ENABLE_DUMMIE          = 13;

/*****CONSTANTES*****/
// Umbrales de protección [V]
const float OVERVOLTAGE_LIMIT        = 4.20; // Overvoltage default
(No se usa)
```

```
const float UNDERVOLTAGE_LIMIT      = 3.30;
const float HYSTERESIS                = 0.175; // 100 mV de margen para
evitar oscilación
const float OVERCURRENT_IN_CHARGE    = 1200.0;
const float OVERCURRENT_IN_DISCHARGE = -1200.0;
const float SHORTCIRCUIT_IN_CHARGE   = 1400.0;
const float SHORTCIRCUIT_IN_DISCHARGE = -1400.0;
const float BALANCE_DELTA_V          = 0.08; // 80 mV
const float BALANCE_MIN_CELL_V       = 3.8;

//Constantes de tiempo
const uint8_t HICCUPTIME              = 20;
const uint8_t OCTIME                  = 20;
const uint8_t VOLTAGE_SAMPLE_TIME     = 50;
const uint8_t MPPT_APPLYING_TIM       = 2;

// Configuración ADC
const float REFERENCIA_INTERNA        = 1.1; // 1.1V interna
const float DIVISOR_FACTOR            = 4.328; // Factor del divisor
resistivo

// Constantes del circuito
const float RESBAL                    = 100; // 100 ohm de
resistencia de balanceo

/*****Variables
globales*****/

// status bms
BMSStatus BMS;

PDUStatus_REG PDUStatus_REG;
PPCStatus_REG PPCStatus_REG;

//Carga-descarga BMS flags
// Control de corriente BMS FLAGS:
// Control de tension BMS FLAGS:

//Balance
bool balance1_request = false; // pedido del usuario por serial
```

```
bool balance2_request = false;

// Control de corriente acumuladores:
uint8_t overcurrentChargeCounter = 0;
uint8_t overcurrentDischargeCounter = 0;

// Control de Salidas FLAGS:
bool Out330n = false;
bool Out50n = false;
PDUStatus pdu{};
bool BatOutOn = false;

//Control de flujo
volatile bool medirCorriente = false;
volatile bool medirVoltajes = false;
volatile bool aplicarMppt = false;
volatile bool guardarStatus = false;

//Control de tiempo
volatile unsigned int eepromCounter = 0;
volatile unsigned int tickCounter = 0;
unsigned long lastMeasureTime = 0;

//objetos y estructuras
BatteryCharge battery{};

//MPPTs
MPPT mpptX(0x41, 0);
MPPT mpptY(0x41, 0);
MPPT mpptZ(0x41, 0);
int statusPPC = 0;
int statusPDU = 0;

//Termistor
Thermistor Thermistor;
```

```
/*
*****
*****/

/******Initial
SETUP******/

void setup() {
  Serial.begin(115200);
  setupPDU();

  BMS.dischargeOC = false;
  BMS.chargeOC = false;
  BMS.dischargeSC = false;
  BMS.chargeSC = false;
  BMS.cell11UV = false;
  BMS.cell12UV = false;
  BMS.cell110V = false;
  BMS.cell120V = false;
  BMS.Chg = true;
  BMS.Dch = true;
  BMS.Bal1 = false;
  BMS.Bal2 = false;
  BMS.RBal1 = false;
  BMS.RBal2 = false;
  BMS.OvT = false;
  BMS.UnT = false;
  BMS.BatF = false;
  BMS.BatL = false;
  BMS.I2CStatus = false;
  battery.coulombCount1 = 0;
  battery.coulombCount2 = 0;

  loadOvervoltage();

  Serial.println("Setup in ");
  Wire.begin();
  mpptX.begin();

  Serial.println("Init INA ");

  while (!ina219BMS.begin()) { Serial.println("Error al iniciar
ina219BMS");}
```

```
//ina219BMS.setCalibration_32V_2A();
Serial.println("ina219BMS OK");
while (!ina219_330Out.begin()) { Serial.println("Error al iniciar
ina219_330Out");}
//ina219BMS.setCalibration_32V_2A();
Serial.println("ina219_330Out OK");
/*
while (!ina219_MPPTX.begin()) { Serial.println("Error al iniciar
ina219_MPPTX");}
//ina219BMS.setCalibration_32V_2A();
Serial.println("ina219_MPPTX OK");
*/

ina219Sleep(ina219BMS,BMS_I2C_ADDR);
ina219Sleep(ina219_330Out,OUT33_I2C_ADDR);
ina219Sleep(ina219_MPPTX,MPPTX_I2C_ADDR);

pinMode(VOLTAGE_ENABLE_PIN, OUTPUT);
digitalWrite(VOLTAGE_ENABLE_PIN, LOW);

// Configurar referencia ADC a 1.1V interna
analogReference(INTERNAL);

setupTimer1();
setupProtection();
setupBalanceo();

set_sleep_mode(SLEEP_MODE_IDLE); // Modo IDLE: deja funcionando el timer
y el I2C
sei(); // Habilita interrupciones globales

//loadAllFromEEPROM(); //carga valores guardados en EEPROM
if( !isfinite(battery.coulombCount1) || !isfinite(battery.coulombCount2)
){
    battery.coulombCount1 = 0;
    battery.coulombCount2 = 0;
}
lastMeasureTime = millis();
}
```



```
/*
*****
*****MAIN LOOP*****
*/

void loop() {
    //Serial.println("1 ");

    sleep_mode(); // El micro duerme hasta que el Timer1 lo despierte
    //Serial.println("2 ");
    // --- Se ejecuta al despertar ---

    while (Serial.available() > 0) {
        identificarSerial();
    }

    //Serial.println("3 ");
    if (medirCorriente){
        // Tiempo transcurrido desde última medición (en segundos)
        unsigned long now = millis();
        double deltaT_s = (now - lastMeasureTime) / 1000.0;
        lastMeasureTime = now;

        ina219Wake(ina219BMS, BMS_I2C_ADDR);
        corriente = ina219BMS.getCurrent_mA();
        ina219Sleep(ina219BMS, BMS_I2C_ADDR);

        checkCellChargeCurrentProtection(corriente);
        checkCellDischargeCurrentProtection(corriente);
        applyProteccion();
        // Coulomb counting: corriente[mA] * tiempo[s]
        if(digitalRead(BAL1_PIN)){
            battery.coulombCount1 += (corriente - tensionB1 / RESBAL) * deltaT_s;
        }else{
            battery.coulombCount1 += corriente * deltaT_s;
        }
        if(digitalRead(BAL2_PIN)){
            battery.coulombCount2 += (corriente - tensionB2 / RESBAL) * deltaT_s;
        }else{
            battery.coulombCount2 += corriente * deltaT_s;
        }
        medirCorriente = false;
    }
}
```

```
//Serial.println("4 ");

if(aplicarMppt){

    if ((tensionB1 >= OvervoltageLimit - 0.1) || (tensionB2 >=
OvervoltageLimit - 0.1) || BMS.Chg == false) {
        mpptX.setConstVoltageMode();
    }
    else if ((tensionB1 <= (OvervoltageLimit -0.1 - HYSTERESIS)) &&
(tensionB1 <= (OvervoltageLimit -0.1 - HYSTERESIS))) {
        mpptX.setConstCurrentMode();
    }

    mpptX.applyMppt();

    Wire.beginTransaction(17);

    // Escribe el byte a enviar
    Wire.write(mpptX.getDuty());
    Wire.write(12);
    Wire.write(25);
    // Termina la transmisión
    Wire.endTransmission();
    ina219Wake(ina219BMS, BMS_I2C_ADDR);
    ina219Wake(ina219_330Out, OUT33_I2C_ADDR);

    if(mpptX.getLastPower_mW()-ina219_330Out.getCurrent_mA()*ina219_330Out.getBusVo
ltage_V() > 0 && BMS.Chg == false){
        digitalWrite(ENABLE_DUMMIE, HIGH);
    }else{
        digitalWrite(ENABLE_DUMMIE, LOW);
    }
    ina219Sleep(ina219BMS, BMS_I2C_ADDR);
    ina219Sleep(ina219_330Out, OUT33_I2C_ADDR);

    aplicarMppt = false;
}

/*
```

```
*/  
  
//Serial.println("5");  
  
if (medirVoltajes){  
  
    // Medir tensión con ina219BMS  
    ina219Wake(ina219BMS, BMS_I2C_ADDR);  
    tensionINA = ina219BMS.getBusVoltage_V();  
    ina219Sleep(ina219BMS, BMS_I2C_ADDR);  
  
    // Activar divisor resistivo  
    digitalWrite(VOLTAGE_ENABLE_PIN, HIGH);  
  
    if(digitalRead(BAL1_PIN)){ //Desactivar balanceos para medir  
        digitalWrite(BAL1_PIN , LOW);  
    }  
    if(digitalRead(BAL2_PIN)){  
        digitalWrite(BAL2_PIN , LOW);  
    }  
    delay(5); // Esperar estabilización mínima  
    int adcDiv = analogRead(ADC_DIVISOR_PIN);  
    // Calcular tensión medida  
    tensionADC = ((float)adcDiv / 1023.0) * REFERENCIA_INTERNA;  
    adcDiv = analogRead(ADC_THERMISTOR_PIN);  
    // Calcular tensión medida  
    tensionTermistor = ((float)adcDiv / 1023.0) * REFERENCIA_INTERNA;  
  
    digitalWrite(VOLTAGE_ENABLE_PIN, LOW);  
    tensionB1 = DIVISOR_FACTOR * tensionADC;  
    if (tensionB1 < 0) tensionB1 = 0; //idk pero por las dudas  
    tensionB2 = tensionINA - tensionB1;  
    if(tensionB2 < 0) tensionB2 = 0;  
  
    //informarStatus();  
  
    // Curvas de descarga y Carga  
  
    relevarCurva();  
}
```

```
// END Curvas de descarga y Carga

/*****Ejecuto protecciones*****/
checkCellVoltageProtection(tensionB1, tensionB2);
//
Termistor.measureTemp(tensionB1, tensionTermistor);
Termistor.applyThermalProtection();
applyProteccion();

medirVoltajes = false;

}

//Serial.println("6");

if (guardarStatus){
    //saveBatteryCharge();
    guardarStatus = false;
}

//Serial.println("7 ");
}

/*****
*****
*****/
/*****SETUP*****/
/
void setupProtection() {
    pinMode(CHARGE_ENABLE_PIN, OUTPUT);
    pinMode(DISCHARGE_ENABLE_PIN, OUTPUT);
    pinMode(ENABLE_CALEFACTOR, OUTPUT);
    pinMode(ENABLE_DUMMIE, OUTPUT);
    digitalWrite(CHARGE_ENABLE_PIN, HIGH); // HIGH = habilitado
    digitalWrite(DISCHARGE_ENABLE_PIN, HIGH); // HIGH = habilitado
    digitalWrite(ENABLE_CALEFACTOR, LOW); // HIGH = habilitado
    digitalWrite(ENABLE_DUMMIE, LOW); // HIGH = habilitado
}
```

```
void setupBalanceo(){
    // Balanceo
    pinMode(BAL1_PIN, OUTPUT);
    pinMode(BAL2_PIN, OUTPUT);
    digitalWrite(BAL1_PIN, LOW);
    digitalWrite(BAL2_PIN, LOW);
}

void setupPDU(){

    pinMode(ENABLE_3_OUT, OUTPUT);
    pinMode(ENABLE_3_BIS_OUT, OUTPUT);
    pinMode(ENABLE_5_OUT, OUTPUT);
    pinMode(ENABLE_5_BIS_OUT, OUTPUT);
    pinMode(ENABLE_BAT_OUT, OUTPUT);

    digitalWrite(ENABLE_3_OUT, HIGH);
    digitalWrite(ENABLE_3_BIS_OUT, LOW);
    digitalWrite(ENABLE_5_OUT, LOW);
    digitalWrite(ENABLE_5_BIS_OUT, LOW);
    digitalWrite(ENABLE_BAT_OUT, LOW);

}

void setupTimer1() {
    cli(); // Desactiva interrupciones mientras se configura

    TCCR1A = 0;
    TCCR1B = 0;

    TCCR1B |= (1 << WGM12); // Modo CTC
    TCCR1B |= (1 << CS11) | (1 << CS10); // Prescaler 64

    OCR1A = 24999; // 100 ms (16MHz / 64 / 10Hz =
25000)

    TIMSK1 |= (1 << OCIE1A); // Habilita interrupción por
comparación
```

```
sei(); // Habilita interrupciones
}
/*****INTERUPCIONES*****/
ISR(TIMER1_COMPA_vect) {
    medirCorriente = true;
    eepromCounter++;
    tickCounter++;
    if ((tickCounter % MPPT_APPLYING_TIM) == 0){
        aplicarMppt = true;
    }
    if (tickCounter >= VOLTAGE_SAMPLE_TIME) { // VOLTAGE_SAMPLE_TIME * 100ms
        medirVoltajes = true;
        tickCounter = 0;
    }
    if (eepromCounter >= 18000){
        eepromCounter = 0;
        guardarStatus = true;
    }
}

/*****EEPROM*****/
*****/
// =====
// Helpers internos: calculan offsets LOCALMENTE
// Orden fijo: OvervoltageLimit -> BatteryCharge -> PDUStatus
// =====
static inline void calcOffsets(int base, int &off0V, int &offUV, int
&offBat, int &offPDU) {
    int off = base;
    off0V = off;                off +=
sizeof(OvervoltageLimit);
    offUV = off;                off +=
sizeof(UndervoltageLimit);
    offBat = off;                off += sizeof(BatteryCharge);
    offPDU = off;
}

// ===== Guardar / Cargar todo =====
void saveAllToEEPROM(int base = 0) {
    int off0V, offUV, offBat, offPDU;
```

```
calcOffsets(base, offOV, offUV, offBat, offPDU);

EEPROM.put(offOV, OvervoltageLimit);
EEPROM.put(offUV, UndervoltageLimit);
EEPROM.put(offBat, battery);
EEPROM.put(offPDU, pdu);
}

void loadAllFromEEPROM(int base = 0) {
    int offOV, offUV, offBat, offPDU;
    calcOffsets(base, offOV, offUV, offBat, offPDU);

    EEPROM.get(offOV, OvervoltageLimit);
    EEPROM.get(offUV, UndervoltageLimit);
    EEPROM.get(offBat, battery);
    EEPROM.get(offPDU, pdu);
}

// ===== Guardar / Cargar cada bloque por separado =====
void saveOvervoltage(int base = 0) {
    int offOV, offUV, offBat, offPDU;
    calcOffsets(base, offOV, offUV, offBat, offPDU);
    EEPROM.put(offOV, OvervoltageLimit);
}

void loadOvervoltage(int base = 0) {
    int offOV, offUV, offBat, offPDU;
    calcOffsets(base, offOV, offUV, offBat, offPDU);
    EEPROM.get(offOV, OvervoltageLimit);
}

void loadUndervoltage(int base = 0) {
    int offOV, offUV, offBat, offPDU;
    calcOffsets(base, offOV, offUV, offBat, offPDU);
    EEPROM.get(offUV, UndervoltageLimit);
}

void saveUndervoltage(int base = 0) {
    int offOV, offUV, offBat, offPDU;
    calcOffsets(base, offOV, offUV, offBat, offPDU);
    EEPROM.put(offUV, UndervoltageLimit);
}
```



```
}

void saveBatteryCharge(int base = 0) {
    int off0V, offUV, offBat, offPDU;
    calcOffsets(base, off0V, offUV, offBat, offPDU);
    EEPROM.put(offBat, battery);
}

void loadBatteryCharge(int base = 0) {
    int off0V, offUV, offBat, offPDU;
    calcOffsets(base, off0V, offUV, offBat, offPDU);
    EEPROM.get(offBat, battery);
}

void savePDUStatus(int base = 0) {
    int off0V, offUV, offBat, offPDU;
    calcOffsets(base, off0V, offUV, offBat, offPDU);
    EEPROM.put(offPDU, pdu);
}

void loadPDUStatus(int base = 0) {
    int off0V, offUV, offBat, offPDU;
    calcOffsets(base, off0V, offUV, offBat, offPDU);
    EEPROM.get(offPDU, pdu);
}

/*****PROTECCIONES*****/
// Función de protección BMS
void checkCellVoltageProtection(float cell1_V, float cell2_V) {

    // Sobretensión celda 1
    if (cell1_V >= OvervoltageLimit) {
        BMS.cell10V = true;
    }
    else if (cell1_V <= (OvervoltageLimit - HYSTERESIS)) {
        BMS.cell10V = false;
    }

    // Sobretensión celda 2
    if (cell2_V >= OvervoltageLimit) {
```

```
BMS.cell20V = true;
}
else if (cell2_V <= (OvervoltageLimit - HYSTERESIS)) {
    BMS.cell20V = false;
}

// Subtensión celda 1
if (cell1_V <= UndervoltageLimit) {
    BMS.cell1UV = true;
}
else if (cell1_V >= (UndervoltageLimit + HYSTERESIS)) {
    BMS.cell1UV = false;
}

// Subtensión celda 2
if (cell2_V <= UndervoltageLimit) {
    BMS.cell2UV = true;
}
else if (cell2_V >= (UndervoltageLimit + HYSTERESIS)) {
    BMS.cell2UV = false;
}
}

void checkCellChargeCurrentProtection(float current){
    if(current < OVERCURRENT_IN_CHARGE && !BMS.chargeSC && !BMS.chargeOC){
//Si esta todo bien sale directamente y resetea contador
        overcurrentChargeCounter = 0;
        return;
    }else if(BMS.chargeSC || BMS.chargeOC){ //si no esta todo bien y salto
algun flag
        if (overcurrentChargeCounter <= 0) overcurrentChargeCounter =
HICCUPTIME; // seed
        if (overcurrentChargeCounter > 0) overcurrentChargeCounter--;
        if(overcurrentChargeCounter <= 0){ //si se acaba reseteo flags
            BMS.chargeSC = false;
            BMS.chargeOC = false;
        }
    }else if(current >= SHORTCIRCUIT_IN_CHARGE){ //umbral de cortocircuito
corta directamente
        BMS.chargeSC = true;
        overcurrentChargeCounter = HICCUPTIME;
    }
}
```

```
    }else{
        overcurrentChargeCounter++;
        if(overcurrentChargeCounter >= OCTIME){ //si llego a 20 corto la
carga
            BMS.chargeOC = true;
            overcurrentChargeCounter = HICCUPTIME;
        }
    }

    return;
}

void checkCellDischargeCurrentProtection(float current){
    if(current > OVERCURRENT_IN_DISCHARGE && !BMS.dischargeSC &&
!BMS.dischargeOC){ //Si esta todo bien sale directamente y resetea contador
        overcurrentDischargeCounter = 0;
        return;
    }else if(BMS.dischargeSC || BMS.dischargeOC){ //si no esta todo bien y
salto algun flag
        if (overcurrentDischargeCounter <= 0) overcurrentDischargeCounter =
HICCUPTIME; // seed
        if (overcurrentDischargeCounter > 0) overcurrentDischargeCounter--;
        if(overcurrentDischargeCounter <= 0){ //si se acaba reseteo flags
            BMS.dischargeSC = false;
            BMS.dischargeOC = false;
        }
    }else if(current <= SHORTCIRCUIT_IN_DISCHARGE){ //umbral de
cortocircuito corta directamente
        BMS.dischargeSC = true;
        overcurrentDischargeCounter = HICCUPTIME;
    }else{
        overcurrentDischargeCounter++;
        if(overcurrentDischargeCounter >= OCTIME){ //si llego a 20 corto la
carga
            BMS.dischargeOC = true;
            overcurrentDischargeCounter = HICCUPTIME;
        }
    }

    return;
}
```

```
void applyPDU() {
    // --- Salida 3.3V ---
    if (!Out33On) {
        digitalWrite(ENABLE_3_OUT, LOW);
        digitalWrite(ENABLE_3_BIS_OUT, LOW);
    } else {
        if (pdu.output33Selected && !pdu.output33BSelected) {
            digitalWrite(ENABLE_3_OUT, HIGH);
            digitalWrite(ENABLE_3_BIS_OUT, LOW);
        } else if (!pdu.output33Selected && pdu.output33BSelected) {
            digitalWrite(ENABLE_3_OUT, LOW);
            digitalWrite(ENABLE_3_BIS_OUT, HIGH);
        } else {
            // caso inválido: ambos true o ambos false
            digitalWrite(ENABLE_3_OUT, HIGH);
            digitalWrite(ENABLE_3_BIS_OUT, LOW);
        }
    }
}

// --- Salida 5V ---
if (!Out50On) {
    digitalWrite(ENABLE_5_OUT, LOW);
    digitalWrite(ENABLE_5_BIS_OUT, LOW);
} else {
    if (pdu.output5Selected && !pdu.output5BSelected) {
        digitalWrite(ENABLE_5_OUT, HIGH);
        digitalWrite(ENABLE_5_BIS_OUT, LOW);
    } else if (!pdu.output5Selected && pdu.output5BSelected) {
        digitalWrite(ENABLE_5_OUT, LOW);
        digitalWrite(ENABLE_5_BIS_OUT, HIGH);
    } else {
        // caso inválido
        digitalWrite(ENABLE_5_OUT, LOW);
        digitalWrite(ENABLE_5_BIS_OUT, LOW);
    }
}

// --- Salida Bat ---
if (BatOutOn) {
    digitalWrite(ENABLE_BAT_OUT, HIGH);
}
```

```
} else {  
    digitalWrite(ENABLE_BAT_OUT, LOW);  
}  
}  
  
void applyProteccion(){  
    if(BMS.dischargeOC || BMS.dischargeSC || BMS.cell11UV || BMS.cell12UV)  
        BMS.Dch = false;  
    else  
        BMS.Dch = true;  
    if(BMS.chargeOC || BMS.chargeSC || BMS.cell11OV || BMS.cell12OV){  
        BMS.Chg = false;  
    }  
    else{  
        BMS.Chg = true;  
    }  
    digitalWrite(CHARGE_ENABLE_PIN, BMS.Chg ? HIGH : LOW);  
    digitalWrite(DISCHARGE_ENABLE_PIN, BMS.Dch ? HIGH : LOW);  
    // --- Control de BALANCEO (aplicación SIEMPRE) ---  
  
    //bool allowBal1 = !cell11OV && !cell11UV;    // regla básica: no balancear  
    //si OV/UV  
    //bool allowBal2 = !cell12OV && !cell12UV;  
    // (opcional) bloquear si hay OC/SC globales:  
    // allowBal1 &= !(chargeSC || chargeOC || dischargeSC || dischargeOC);  
    // allowBal2 &= !(chargeSC || chargeOC || dischargeSC || dischargeOC);  
  
    bool bal1_on = balance1_request; // && allowBal1;  
    bool bal2_on = balance2_request; // && allowBal2;  
  
    digitalWrite(BAL1_PIN, bal1_on ? HIGH : LOW);  
    digitalWrite(BAL2_PIN, bal2_on ? HIGH : LOW);  
  
    applyPDU();  
  
    if(Termistor.getHeatUp())  
        digitalWrite(ENABLE_CALEFACTOR, HIGH); // HIGH = habilitado  
    else  
        digitalWrite(ENABLE_CALEFACTOR, LOW); // HIGH = habilitado  
  
    // Debug
```

```
if(medirVoltajes && 0){
    Serial.print("Flags - Ch0v:"); Serial.print(BMS.chargeOC);
    Serial.print(" ChSh:"); Serial.print(BMS.chargeSC);
    Serial.print(" Dc0v:"); Serial.print(BMS.dischargeOC);
    Serial.print(" DcSh:"); Serial.print(BMS.dischargeSC);
    Serial.print(" Cell10V:"); Serial.print(BMS.cell10V);
    Serial.print(" Cell20V:"); Serial.print(BMS.cell20V);
    Serial.print(" Cell1UV:"); Serial.print(BMS.cell1UV);
    Serial.print(" Cell2UV:"); Serial.println(BMS.cell2UV);

    Serial.print("Charge: "); Serial.println(BMS.Chg ? "ON" : "OFF");
    Serial.print("Discharge: "); Serial.println(BMS.Dch ? "ON" : "OFF");
}
/*
// === NUEVO LOG SIEMPRE (balanceo) ===
Serial.print(F("Balance req B1:")); Serial.print(balance1_request);
Serial.print(F(" B2:")); Serial.println(balance2_request);
Serial.print(F("Balance appl B1:")); Serial.print(bal1_on ? "ON" :
"OFF");
Serial.print(F(" B2:")); Serial.println(bal2_on ? "ON" : "OFF");
*/
}
/*****UART*****/

void relevarCurva(){
    Serial.println(tensionINA);
    Serial.println(corriente);
    Serial.println(battery.coulombCount1);
}

void relevarCurva2(){
    Serial.println(tensionB1);
    Serial.println(tensionB2);
    Serial.println(battery.coulombCount1);
    Serial.println(battery.coulombCount2);
    Serial.println(corriente);
}

void informarStatus(){
    Serial.println("|*****|");
    /*****Informe corriente*****/
}
```

```
Serial.print("Corriente (mA): ");
Serial.println(corriente);
/*****Informe cargas*****/
Serial.print("Carga acumulada B1 (mAh): ");
Serial.println(battery.coulombCount1 / 3600.0, 5);
Serial.print("Carga acumulada B2 (mAh): ");
Serial.println(battery.coulombCount2 / 3600.0, 5);
/*****Informe tensiones*****/
Serial.print("Tensión ina219BMS [V]: ");
Serial.println(tensionINA);
Serial.print("Tensión B1 [V]: ");
Serial.println(tensionB1, 4);
Serial.print("Tensión B2 [V]: ");
Serial.println(tensionB2, 4);
Serial.print("Tensión Termistor [V]}: ");
Serial.println(tensionTermistor, 4);
Serial.print("Temperatura [°C]: ");
Serial.println(Termistor.getTemp(), 3);

// PDU Status - 3.3 Select:255 3.3B Select:255 3.3 ON:0 5 Select:255 5B
Select:255 5 ON:0Balance: OK

Serial.print("PDU Status - 3.3 Select:");
Serial.print(pdu.output33Selected);
Serial.print(" 3.3B Select:"); Serial.print(pdu.output33BSelected);
Serial.print(" 3.3 ON:"); Serial.print(Out33On);
Serial.print(" 5 Select:"); Serial.print(pdu.output5Selected);
Serial.print(" 5B Select:"); Serial.print(pdu.output5BSelected);
Serial.print(" 5 ON:"); Serial.println(Out5On);

/*****Necesidad de balance*****/
Serial.print("Balance: ");
if(tensionB1 - tensionB2 > BALANCE_DELTA_V && tensionB1 >
BALANCE_MIN_CELL_V && !balance2_request)
    if(balance1_request)
        Serial.println("Balance B1 in progress");
    else
        Serial.println("Balance B1 needed");
else if(tensionB2 - tensionB1 > BALANCE_DELTA_V && tensionB2 >
BALANCE_MIN_CELL_V && !balance1_request)
    if(balance2_request)
```

```
        Serial.println("Balance B2 in progress");
    else
        Serial.println("Balance B2 needed");
    else{
        if(balance1_request == true || balance2_request == true){
            balance1_request = false;
            balance2_request = false;
            Serial.println("DONE");
        }
        Serial.println("OK");
    }

    Serial.print("Duty: ");
    Serial.println(mpptX.getDuty());
    Serial.print("P (mW): ");
    Serial.println(mpptX.getLastPower_mW());
    Serial.println("|*****|");

}

void identificarSerial(){
    String cmd = Serial.readStringUntil('\n');
    cmd.trim();
    int len = cmd.length();
    if (len < 2)
        return;

    switch(cmd[0]){
        case 'M': //Todo lo referido a MPPT
            if (len > 3){
                Serial.println("Invalid Command");
                return;
            }
            switch(cmd[1]){ //identifico siguiente byte
                case 'R': // registro de estado
                    if (len > 2){
                        Serial.println("Invalid Command");
                        return;
                    }
                }else
                    if(mpptX.isOK())                {PPCStatus_REG.PPCXOK = true;}
            }
    }
}
```



```
if(mpptX.isEnabled()) {PPCStatus_REG.PPCXEN = true;}
if(mpptY.isOK()) {PPCStatus_REG.PPCYOK = true;}
if(mpptY.isEnabled()) {PPCStatus_REG.PPCYEN = true;}
if(mpptZ.isOK()) {PPCStatus_REG.PPCZOK = true;}
if(mpptZ.isEnabled()) {PPCStatus_REG.PPCZEN = true;}
Serial.println("Registro mppt:");
Serial.print("PPCXOK");
Serial.println(PPCStatus_REG.PPCXOK);
Serial.print("PPCXEN");
Serial.println(PPCStatus_REG.PPCXEN);
Serial.print("PPCYOK");
Serial.println(PPCStatus_REG.PPCYOK);
Serial.print("PPCYEN");
Serial.println(PPCStatus_REG.PPCYEN);
Serial.print("PPCZOK");
Serial.println(PPCStatus_REG.PPCZOK);
Serial.print("PPCZEN");
Serial.println(PPCStatus_REG.PPCZEN);
break;

case 'X': // referido al par de paneles del eje X
    switch(cmd[2]){
        case 'N': // encender mppt x
            Serial.println("MPPT X: ON");
            mpptX.enable();
            break;
        case 'F': // apagar mppt x
            Serial.println("MPPT X: OFF");
            mpptX.disable();
            break;
        case 'V': // tension mppt x
            Serial.print("Tensión mppt X [V]: ");
            Serial.println(mpptX.getBusVoltage(), 4);
            break;
        case 'C': // corriente mppt x
            Serial.print("Corriente mppt X [mA]: ");
            Serial.println(mpptX.getCurrent(), 4);
            break;
        default: // cualquier cotra cosa es invalido
            Serial.println("Invalid Command");
            return;
    }
}
```

```
        break;
    }
    break;
    case 'Y': // referido al par de paneles del eje Y
    switch(cmd[2]){
        case 'N': // encender mppt y
        Serial.println("MPPT Y: ON");
        break;
        case 'F': // apagar mppt y
        Serial.println("MPPT Y: OFF");
        break;
        case 'V': // tension mppt y
        Serial.println("VY: ");
        break;
        case 'C': // corriente mppt y
        Serial.println("VY: ");
        break;
        default: // cualquier cotra cosa es invalido
        Serial.println("Invalid Command");
        return;
        break;
    }
    break;
    case 'Z': // referido al par de paneles del eje Z
    switch(cmd[2]){
        case 'N': // encender mppt z
        Serial.println("MPPT Z: ON");
        break;
        case 'F': // apagar mppt z
        Serial.println("MPPT Z: OFF");
        break;
        case 'V': // tension mppt z
        Serial.println("VZ: ");
        break;
        case 'C': // corriente mppt z
        Serial.println("VZ: ");
        break;
        default: // cualquier cotra cosa es invalido
        Serial.println("Invalid Command");
        return;
        break;
    }
}
```

```
    }  
    break;  
    default:  
        Serial.println("Invalid Command");  
        break;  
    }  
    break;  
case 'P': //Todo lo referido a PDU  
    if (len > 3){  
        Serial.println("Invalid Command");  
        return;  
    }  
    switch(cmd[1]){ //identifico siguiente byte  
        case 'R': // registro de estado PDU  
            if (len > 2){  
                Serial.println("Invalid Command");  
                return;  
            }else  
                if(corrienteBat < 1000)  
{PDUStatus_REG.PDU10C = true;}  
                if(corrienteBat > 10)  
{PDUStatus_REG.PDU10N = true;}  
                if(tensionBat > 6.6 && tension5 < 8.2)  
{PDUStatus_REG.PDU10K = true;}  
                if(digitalRead(ENABLE_5_BIS_OUT))  
{PDUStatus_REG.PDU1EN = true;}  
  
                if(corriente5 < 1000 && pdu.output5BSelected)  
{PDUStatus_REG.PDU20C = true;}  
                if(corriente5 > 10 && pdu.output5BSelected)  
{PDUStatus_REG.PDU20N = true;}  
                if(tension5 > 4.9 && tension5 < 5.1)  
{PDUStatus_REG.PDU20K = true;}  
                if(digitalRead(ENABLE_5_BIS_OUT))  
{PDUStatus_REG.PDU2EN = true;}  
                if(corriente5 < 1000 && pdu.output5Selected)  
{PDUStatus_REG.PDU30C = true;}  
                if(corriente5 > 10 && pdu.output5Selected)  
{PDUStatus_REG.PDU30N = true;}  
                if(tension5 > 4.9 && tension5 < 5.1)  
{PDUStatus_REG.PDU30K = true;}
```

```
        if(digitalRead(ENABLE_5_OUT))
{PDUStatus_REG.PDU3EN = true;}

        if(corriente33 < 1000 && pdu.output33BSelected)
{PDUStatus_REG.PDU40C = true;}
        if(corriente33 > 10 && pdu.output33BSelected)
{PDUStatus_REG.PDU40N = true;}
        if(tension33 > 3.2 && tension33 < 3.4)
{PDUStatus_REG.PDU40K = true;}
        if(digitalRead(ENABLE_3_BIS_OUT))
{PDUStatus_REG.PDU4EN = true;}
        if(corriente33 < 1000 && pdu.output33Selected)
{PDUStatus_REG.PDU50C = true;}
        if(corriente33 > 10 && pdu.output33Selected)
{PDUStatus_REG.PDU50N = true;}
        if(tension33 > 3.2 && tension33 < 3.4)
{PDUStatus_REG.PDU50K = true;}
        if(digitalRead(ENABLE_3_OUT))
{PDUStatus_REG.PDU5EN = true;}


Serial.println("Registro mppt:");
Serial.print("PDU1EN");
Serial.println(PDUStatus_REG.PDU1EN);
Serial.print("PDU10K");
Serial.println(PDUStatus_REG.PDU10K);
Serial.print("PDU10N");
Serial.println(PDUStatus_REG.PDU10N);
Serial.print("PDU10C");
Serial.println(PDUStatus_REG.PDU10C);

Serial.print("PDU2EN");
Serial.println(PDUStatus_REG.PDU2EN);
Serial.print("PDU20K");
Serial.println(PDUStatus_REG.PDU20K);
Serial.print("PDU20N");
Serial.println(PDUStatus_REG.PDU20N);
Serial.print("PDU20C");
Serial.println(PDUStatus_REG.PDU20C);

Serial.print("PDU3EN");
```

```
Serial.println(PDUStatus_REG.PDU3EN);
Serial.print("PDU3OK");
Serial.println(PDUStatus_REG.PDU3OK);
Serial.print("PDU3ON");
Serial.println(PDUStatus_REG.PDU3ON);
Serial.print("PDU3OC");
Serial.println(PDUStatus_REG.PDU3OC);

Serial.print("PDU4EN");
Serial.println(PDUStatus_REG.PDU4EN);
Serial.print("PDU4OK");
Serial.println(PDUStatus_REG.PDU4OK);
Serial.print("PDU4ON");
Serial.println(PDUStatus_REG.PDU4ON);
Serial.print("PDU4OC");
Serial.println(PDUStatus_REG.PDU4OC);

Serial.print("PDU5EN");
Serial.println(PDUStatus_REG.PDU5EN);
Serial.print("PDU5OK");
Serial.println(PDUStatus_REG.PDU5OK);
Serial.print("PDU5ON");
Serial.println(PDUStatus_REG.PDU5ON);
Serial.print("PDU5OC");
Serial.println(PDUStatus_REG.PDU5OC);

break;

case '3': // referido a la salida de 3.3V
    switch(cmd[2]){
        case 'N': // encender mppt x
            Serial.println("3.3V Output: ON");
            Out33On = true;
            if((pdu.output33Selected == 0 && pdu.output33BSelected == 0)
|| (pdu.output33Selected == 1 && pdu.output33BSelected == 1)){
                pdu.output33Selected = 1;
                pdu.output33BSelected = 0;
                savePDUStatus();
            }
            break;
```

```
case 'F': // apagar mppt x
    Serial.println("3.3V Outout: OFF");
    Out33On = false;
break;
case 'S': // tension mppt x
    Serial.println("3.3V: Alternar backup");
    if(pdu.output33Selected == 1 && pdu.output33BSelected == 0){
        pdu.output33Selected = 0;
        pdu.output33BSelected = 1;
        savePDUStatus();
    }else if(pdu.output33Selected == 0 && pdu.output33BSelected
== 1){
        pdu.output33Selected = 1;
        pdu.output33BSelected = 0;
        savePDUStatus();
    }
break;
case 'V': // tension mppt x
    Serial.print("V 3.3: ");
    ina219Wake(ina219_33Out, OUT33_I2C_ADDR);
    tension33 = ina219_33Out.getBusVoltage_V();
    ina219Sleep(ina219_33Out, OUT33_I2C_ADDR);
    Serial.println(tension33);
break;
case 'C': // corriente mppt x
    Serial.print("C 3.3: ");
    ina219Wake(ina219_33Out, OUT33_I2C_ADDR);
    corriente33 = ina219_33Out.getCurrent_mA();
    ina219Sleep(ina219_33Out, OUT33_I2C_ADDR);
    Serial.println(corriente33);
break;
default: // cualquier cotra cosa es invalido
    Serial.println("Invalid Command");
    return;
break;
}
break;
case '5': // referido a la salida de 5V
switch(cmd[2]){
    case 'N': // encender mppt x
        Serial.println("5V Output: ON");
```

```
        Out5On = true;
        if((pdu.output5Selected == false && pdu.output5BSelected ==
false) || (pdu.output5Selected == true && pdu.output5BSelected == true)){
            pdu.output5Selected = true;
            pdu.output5BSelected = false;
            savePDUStatus();
        }
        break;
    case 'F': // apagar mppt x
        Serial.println("5V Outout: OFF ");
        Out5On = false;
        break;
    case 'S': // tension mppt x
        Serial.println("Alternar backup ");
        if(pdu.output5Selected == true && pdu.output5BSelected ==
false){
            pdu.output5Selected = false;
            pdu.output5BSelected = true;
            savePDUStatus();
        }else if(pdu.output5Selected == false && pdu.output5BSelected
== true){
            pdu.output5Selected = true;
            pdu.output5BSelected = false;
            savePDUStatus();
        }
        break;
    case 'V': // tension mppt x
        Serial.print("V 5: ");
        ina219Wake(ina219_5Out, OUT5_I2C_ADDR);
        tension5 = ina219_5Out.getBusVoltage_V();
        ina219Sleep(ina219_5Out, OUT5_I2C_ADDR);
        Serial.println(tension5);
        break;
    case 'C': // corriente mppt x
        Serial.print("C 5: ");
        ina219Wake(ina219_5Out, OUT5_I2C_ADDR);
        corriente5 = ina219_5Out.getCurrent_mA();
        ina219Sleep(ina219_5Out, OUT5_I2C_ADDR);
        Serial.println(corriente5);
        break;
    default: // cualquier cotra cosa es invalido
```

```
        Serial.println("Invalid Command ");
        return;
        break;
    }
    break;
    case 'B': // referido a la salida de Bat+
    switch(cmd[2]){
        case 'N': // encender mppt x
            Serial.println("Bat Output: ON");
            BatOutOn = true;
            break;
        case 'F': // apagar mppt x
            Serial.println("Bat Outout: OFF");
            BatOutOn = false;
            break;
        case 'V': // tension mppt x
            Serial.println("V Bat: ");
            ina219Wake(ina219_BatOut, BATOUT_I2C_ADDR);
            tensionBat = ina219_BatOut.getBusVoltage_V();
            ina219Sleep(ina219_BatOut, BATOUT_I2C_ADDR);

            Serial.println(tensionBat);
            break;
        case 'C': // corriente mppt x
            Serial.println("C Bat: ");
            ina219Wake(ina219_BatOut, BATOUT_I2C_ADDR);
            corrienteBat = ina219_BatOut.getCurrent_mA();
            ina219Sleep(ina219_BatOut, BATOUT_I2C_ADDR);

            Serial.println(corrienteBat);
            break;
        default: // cualquier cotra cosa es invalido
            Serial.println("Invalid Command");
            return;
            break;
    }
    break;
    default:
        Serial.println("Invalid Command");
        break;
    }
}
```



```
break;
case 'B': //Todo lo referido a BMS
    if (len > 3){
        Serial.println("Invalid Command");
        return;
    }
    switch(cmd[1]){ //identifico siguiente byte
        case 'S': // registro de estado
            if (len > 2){
                Serial.println("Invalid Command");
                return;
            }else{
                printBMSStatus(BMS);
            }
            break;
        case 'C': // Corriente de las baterias
            if (len > 2){
                Serial.println("Invalid Command");
                return;
            }else{
                Serial.print("Corriente Baterias:");
                Serial.println(corriente);
            }
            break;
        case 'T': // Temperatura de las baterias
            if (len > 2){
                Serial.println("Invalid Command");
                return;
            }else{
                Serial.print("Temperatura [°C]: ");
                Serial.println(Termistor.getTemp(), 3);
            }
            break;
        /*
        case '%': // Porcentaje de carga de celdas
            if (len > 2){
                Serial.println("Invalid Command");
                return;
            }else{
                Serial.println("Porcentaje de carga de las baterias:");
            }
        */
    }
```

```
        break;
    */
    case '1': // referido a la bateria 1
        switch(cmd[2]){
            case 'N': // encender Balance 1
                balance1_request = true;
                Serial.println("Bal Bat 1: ON");
                break;
            case 'F': // apagar Balance 1
                balance1_request = false;
                Serial.println("Bal Bat 1: OFF");
                break;
            case 'V': // tension Bateria 1
                Serial.print("Tensión B1 [V]: ");
                Serial.println(tensionB1, 4);
                break;
            case 'Q': // Carga Bateria 1
                Serial.print("Carga B1 (mAh): ");
                Serial.println(battery.coulombCount1 / 3600.0, 5);
                break;
            default: // cualquier cotra cosa es invalido
                Serial.println("Invalid Command");
                return;
                break;
        }
        break;
    case '2': // referido a la bateria 2
        switch(cmd[2]){
            case 'N': // encender Balance 2
                balance2_request = true;
                Serial.println("Bal Bat 2: ON");
                break;
            case 'F': // apagar Balance 2
                balance2_request = false;
                Serial.println("Bal Bat 2: OFF");
                break;
            case 'V': // tension Bateria 2
                Serial.print("Tensión B2 [V]: ");
                Serial.println(tensionB2, 4);
                break;
            case 'Q': // Carga Bateria 2
```

```
Serial.print("Carga B2 (mAh): ");  
Serial.println(battery.coulombCount2 / 3600.0, 5);  
break;  
default: // cualquier cotra cosa es invalido  
Serial.println("Invalid Command");  
return;  
break;  
}  
break;  
case 'V': // Umbral de sobretension  
switch(cmd[2]){  
case '0':  
Serial.print("OV Voltage: ");  
OvervoltageLimit = 3.90;  
Serial.println(OvervoltageLimit, 4);  
saveOvervoltage();  
break;  
case '1':  
Serial.print("OV Voltage: ");  
OvervoltageLimit = 3.95;  
Serial.println(OvervoltageLimit, 4);  
saveOvervoltage();  
break;  
case '2':  
Serial.print("OV Voltage: ");  
OvervoltageLimit = 4.0;  
Serial.println(OvervoltageLimit, 4);  
saveOvervoltage();  
break;  
case '3':  
Serial.print("OV Voltage: ");  
OvervoltageLimit = 4.05;  
Serial.println(OvervoltageLimit, 4);  
saveOvervoltage();  
break;  
case '4':  
Serial.print("OV Voltage: ");  
OvervoltageLimit = 4.1;  
Serial.println(OvervoltageLimit, 4);  
saveOvervoltage();  
break;
```

```
case '5':
Serial.print("OV Voltage: ");
OvervoltageLimit = 4.15;
Serial.println(OvervoltageLimit, 4);
saveOvervoltage();
break;
case '6':
Serial.print("OV Voltage: ");
OvervoltageLimit = 4.2;
Serial.println(OvervoltageLimit, 4);
saveOvervoltage();
break;
case '7':
Serial.print("OV Voltage: ");
OvervoltageLimit = 4.25;
Serial.println(OvervoltageLimit, 4);
saveOvervoltage();
break;
case '8':
Serial.print("OV Voltage: ");
OvervoltageLimit = 4.3;
Serial.println(OvervoltageLimit, 4);
saveOvervoltage();
break;
case '9':
Serial.print("OV Voltage: ");
OvervoltageLimit = 4.35;
Serial.println(OvervoltageLimit, 4);
saveOvervoltage();
break;
case 'A':
Serial.print("UV Voltage: ");
UndervoltageLimit = 2.95;
Serial.println(UndervoltageLimit, 4);
saveUndervoltage();
break;
case 'B':
Serial.print("UV Voltage: ");
UndervoltageLimit = 3.0;
Serial.println(UndervoltageLimit, 4);
saveUndervoltage();
```

```
break;
case 'C':
Serial.print("Uv Voltage: ");
UndervoltageLimit = 3.05;
Serial.println(UndervoltageLimit, 4);
saveUndervoltage();
break;
case 'D':
Serial.print("Uv Voltage: ");
UndervoltageLimit = 3.1;
Serial.println(UndervoltageLimit, 4);
saveUndervoltage();
break;
case 'E':
Serial.print("Uv Voltage: ");
UndervoltageLimit = 3.15;
Serial.println(UndervoltageLimit, 4);
saveUndervoltage();
break;
case 'F':
Serial.print("Uv Voltage: ");
UndervoltageLimit = 3.2;
Serial.println(UndervoltageLimit, 4);
saveUndervoltage();
break;
case '10':
Serial.print("Uv Voltage: ");
UndervoltageLimit = 3.25;
Serial.println(UndervoltageLimit, 4);
saveUndervoltage();
break;
case '11':
Serial.print("Uv Voltage: ");
UndervoltageLimit = 3.3;
Serial.println(UndervoltageLimit, 4);
saveUndervoltage();
break;
case '12':
Serial.print("Uv Voltage: ");
UndervoltageLimit = 3.35;
Serial.println(UndervoltageLimit, 4);
```

```
        saveUndervoltage();  
        break;  
    case '13':  
        Serial.print("Uv Voltage: ");  
        UndervoltageLimit = 3.4;  
        Serial.println(UndervoltageLimit, 4);  
        saveUndervoltage();  
        break;  
    default:  
        Serial.println("Invalid Command");  
        return;  
        break;  
    }  
    break;  
default:  
    Serial.println("Invalid Command");  
    break;  
}  
break;  
default:  
    Serial.println("Invalid Command");  
    break;  
}  
}
```

26. Conclusión

El desarrollo de este Sistema de Gestión de Energía (EPS) para CubeSat fue un proceso iterativo y de gran aprendizaje, marcado por la superación de desafíos técnicos que llevaron a una optimización profunda del diseño.